

CHAPTER 3: CLASSIFICATION (PHÂN LỚP DỮ LIỆU)

Mục lục

3.1. What Is Machine Learning? (Học máy là gì?)	2
3.2. Các loại học máy	3
3.3. Hai loại bài toán dự đoán chính trong học có giám sát: Classification & Numeric Prediction	5
3.3.1. Classification (Phân loại/ phân lớp dữ liệu)	5
3.3.2. Numeric Prediction (Dự đoán số) / Regression (Hồi quy)	6
3.4. Classification—Model Construction, Validation and Testing (Xây dựng, xác thực và kiểm tra mô hình)	6
3.5. Decision Tree (Cây quyết định)	9
3.5.1. Định nghĩa Cây quyết định	9
3.5.2. How to Handle Continuous-Valued Attributes? (Xử lý thuộc tính giá trị liên tục)	10
3.5.3. Splitting Measures: Information Gain (Entropy) (Thước đo phân chia: Độ lợi thông tin)	12
3.5.3.1 Entropy (Information Theory)	12
3.5.3.2. Định nghĩa chính thức cách tính Information Gain để chọn thuộc tính tốt nhất	14
3.5.4. Gain Ratio: A Refined Measure for Attribute Selection (Tỷ lệ độ lợi: Một thước đo cải tiến)	16
3.5.5. Gini index	17
3.5.6. So sánh các độ đo chọn thuộc tính	19
3.5.7. Các thuật toán và độ đo khác của cây quyết định	19
3.5.7.1. ID3 (Iterative Dichotomiser 3)	19
3.5.7.2. C4.5	19
3.5.7.3. C5.0	20
3.5.7.4. CHAID (Chi-squared Automatic Interaction Detection)	20
3.5.7.5. C-SEP	20
3.5.7.6. MDL (Minimal Description Length)	20
3.5.7.7. Multivariate splits	20
3.5.7.8. Measure nào tốt nhất?	21
3.6. Bayes Classification Methods (Các phương pháp phân loại Bayes)	21
3.6.1. Bayes' Theorem	21
3.6.2. Naïve Bayes Classifier	22
3.6.3. Strength vs. Weakness (Điểm mạnh và điểm yếu)	26
3.7. Lazy vs. Eager Learning (Học lười và Học háo hức)	27
3.7.1. Lazy Learner: Instance-Based Methods (Các phương pháp dựa trên mẫu)	27
3.7.2. The k-Nearest Neighbor Algorithm (Thuật toán k-NN)	27
3.7.3. Case-Based Reasoning (CBR) (Suy luận dựa trên tình huống)	31
3.8. Linear Classifiers	32
3.8.1. Giới thiệu bài toán Hồi quy (Regression)	32
3.8.2. Hồi quy tuyến tính (Linear Regression)	33
3.8.3. Logistic Regression (Hồi quy Logistic)	35
3.8.4. Gradient Descent (Giảm theo Gradient)	37
3.9. Support Vector Machine (SVM) (Máy Vector Hỗ trợ)	38
3.9.1. Mở đầu	38
3.9.2. Tính toán margin trên Hyperplane	43

3.10. Model Evaluation and Selection (Đánh giá và lựa chọn mô hình)	47
3.10.1. Confusion Matrix (Ma trận nhầm lẫn)	48
3.10.2. Phương pháp Holdout	51
3.10.3. Phương pháp Cross-Validation (Xác thực chéo) (k-fold)	51
3.10.4. Phương pháp Bootstrap	51
3.10.5. ROC Curves (Đường cong ROC)	51
3.11. Techniques to Improve Classification Accuracy	53
3.11.1. Ensemble Methods: Increasing the Accuracy (Phương pháp tổ hợp/dàn hợp xướng)	53
3.11.2. Bagging: Bootstrap Aggregation	54
3.11.3. Boosting	55
3.11.4. Adaboost (Freund and Schapire, 1997)	56
3.11.5. Gradient Boosting (Tăng cường Gradient)	57
3.11.6. Random Forest	58
3.11.7. Tóm tắt lại ưu và nhược điểm của các phương pháp Ensemble hiện đại (Random Forest và XGBoost)	60
3.11.8. Classification of Class-Imbalanced Data Sets (Phân loại tập dữ liệu mất cân bằng)	60
3.12. Phụ lục	63
3.12.1. Tham số (Parameters) là gì?	63
3.12.2. Siêu tham số (Hyperparameters) là gì?	64

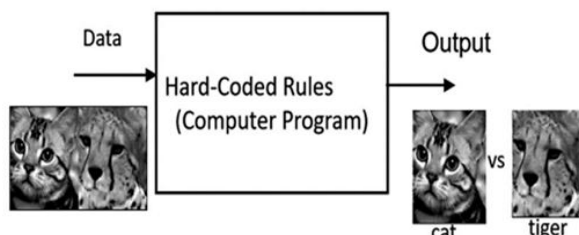
3.1. What Is Machine Learning? (Học máy là gì?)

- **Lập trình truyền thống:**

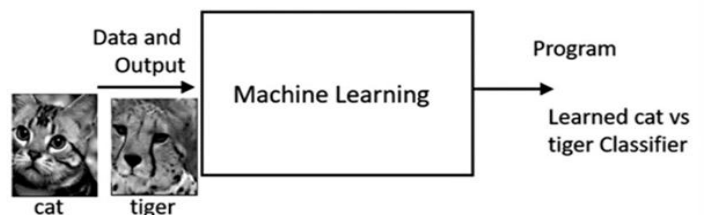
- **Data + Rules = Output.**

- Giải thích: **Lập trình viên phải tự viết ra các quy tắc một cách rõ ràng, tường minh.**
 - Ví dụ, để phân biệt mèo và hổ, bạn phải viết code kiểu như: *if (có vằn) then là hổ; if (kêu meo meo) then là mèo;....*
 - Máy tính chỉ làm theo luật bạn đặt ra.

Traditional Programming : Write Computer Programs using hard-coded (fixed) rules



Machine Learning : Learn Rules by looking at some training data



- **Học máy (Machine Learning):**

- **Data + Output = Rules.**

- Giải thích: Thay vì tự viết luật, bạn **cung cấp cho máy tính rất nhiều ví dụ.**
 - Bạn đưa cho nó hàng ngàn ảnh con mèo (Data) và nói "đây là mèo" (Output). Tương tự với ảnh con hổ.
 - Máy tính sẽ tự "học" và tìm ra các quy tắc (Rules) để phân biệt chúng.
 - Kết quả của quá trình học này là một **"mô hình"** (ví dụ: Learned cat vs tiger Classifier).

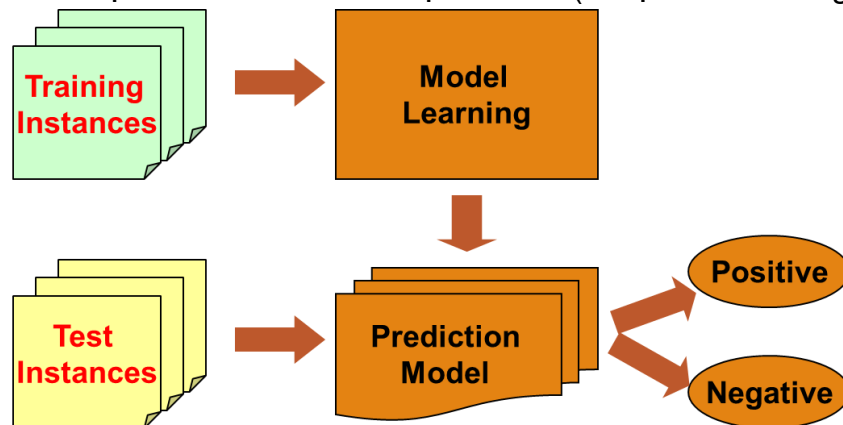
3.2. Các loại học máy

- **Học có giám sát (Supervised Learning (classification)):**

- dữ liệu huấn luyện đã **được gán nhãn sẵn** (labeled data). Giống như có một "người thầy" chỉ cho máy tính biết đâu là câu trả lời đúng.
- Máy học cách **dự đoán dựa trên các ví dụ đã biết**, ví dụ: phân loại hình ảnh (ảnh này là chó, ảnh kia là mèo) hoặc dự đoán giá nhà dựa trên dữ liệu lịch sử.
- New data is classified **based on the models built from the training set**.
- Trong thực tế, loại này thường dùng cho các nhiệm vụ như classification (phân loại) và regression (hồi quy).

- **Quy trình làm việc:**

- **Training Instances (Dữ liệu huấn luyện):** Dữ liệu đã được gán nhãn.
- **Model Learning (Học mô hình):** Thuật toán sẽ học các quy luật từ dữ liệu này để tạo ra một **Prediction Model (Mô hình dự đoán)**.
- **Test Instances (Dữ liệu kiểm tra):** Đưa một dữ liệu mới (chưa có nhãn) vào mô hình.
- Mô hình sẽ dự đoán nhãn cho dữ liệu mới đó (ví dụ: Positive/Negative, Yes/No).

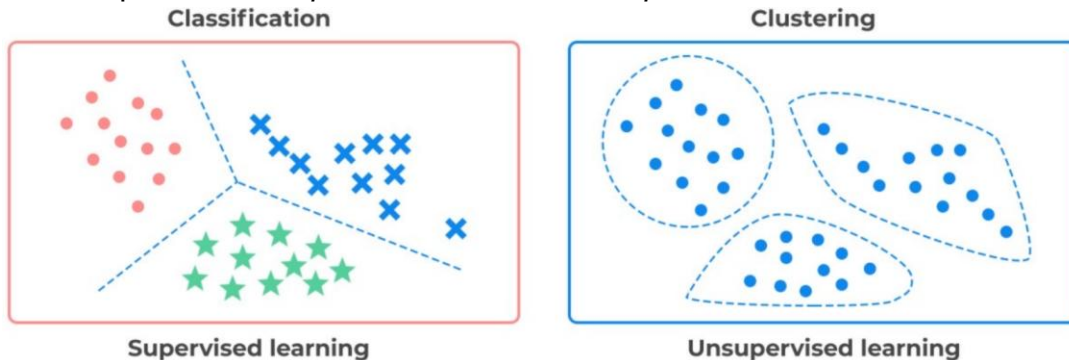


Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No

- Các cột Outlook, Temp, Humidity, Windy là các **đặc trưng (features)** của dữ liệu.
- Cột Play Golf (Chơi golf) chính là **nhãn (label)**. Nhãn này cho biết với điều kiện thời tiết như vậy thì kết quả là "Yes" (có đi chơi) hay "No" (không đi chơi).

- **Học không có giám sát (Unsupervised Learning (clustering)):**

- Dữ liệu không có nhãn, và **máy phải tự tìm ra cấu trúc hoặc mẫu/cụm (clusters) ẩn** trong dữ liệu, chẳng hạn như nhóm các điểm dữ liệu tương tự (clustering) hoặc giảm chiều dữ liệu (dimensionality reduction).
- Máy tự phân nhóm mà không gán nhãn cụ thể – con người có thể diễn giải sau.
- Giống như bạn đưa cho một đứa trẻ một rổ đầy các loại trái cây và bảo nó tự phân loại. Nó có thể sẽ nhóm các quả tròn lại với nhau, các quả dài lại với nhau, hoặc các quả màu đỏ lại với nhau... mà không cần ai dạy trước "đây là quả táo" hay "đây là quả chuối".
- Là loại phổ biến cho *phân tích dữ liệu khám phá*.



- **Học tăng cường (Reinforcement Learning):**

- Đây là học qua tương tác với môi trường, nơi **agent (đại diện) thực hiện hành động, nhận phần thưởng (reward) nếu đúng hoặc phạt (penalty) nếu sai**, nhằm tối ưu hóa tổng phần thưởng dài hạn.
- Ví dụ: AI chơi game như AlphaGo học qua thử và sai.
- **Loại này thường dùng cho robot hoặc xe tự lái.**

- **Học bán giám sát (Semi-supervised Learning):**

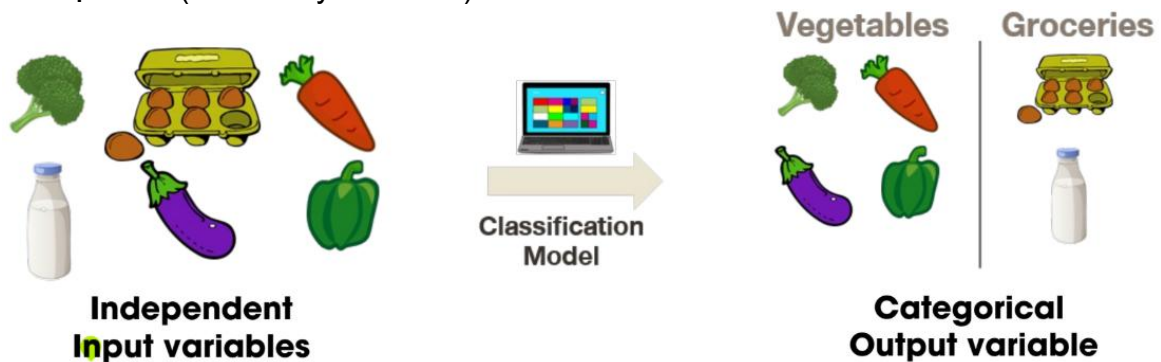
- Kết hợp một **lượng nhỏ dữ liệu có nhãn với lượng lớn dữ liệu không nhãn** để huấn luyện mô hình.
- Điều này hữu ích khi việc gán nhãn tốn kém. Đây thường là cách tiếp cận thực tế để tiết kiệm chi phí.

	Overview	Process	Subtypes	Examples
Supervised learning	Majority of algorithms. Machine is trained using well-labeled data; inputs and outputs are matched.	Mapping function takes inputs and matches to outputs, creating a target function	Classification Regression	Decision tree, Random forest, SVM, K-NN, Neural network, Linear regression, Logistic regression, ...
Unsupervised learning	Unlabeled data (inputs only) is analyzed. Learning happens without supervision	Inputs are used to create a model of the data	Clustering Association Dimensionality reduction	K-Means, C-Means, Hierarchy, Gaussian Mixture Apriori, FP-Growth PCA, LDA,...
Semi-Supervised learning	Some data is labeled, some not. Goal: better results than labeled data alone. Good for real world data.	Combination of above processes	All the above	Self-training, Mixture models, Semi-supervised SVM,...

3.3. Hai loại bài toán dự đoán chính trong học có giám sát: Classification & Numeric Prediction

3.3.1. Classification (Phân loại/ phân lớp dữ liệu)

- mục tiêu là dự đoán nhãn lớp (categorical class labels) cho dữ liệu mới. **Các nhãn lớp này là rời rạc (discrete) hoặc danh nghĩa (nominal)**, nghĩa là chúng thuộc về một tập hợp hữu hạn các danh mục không có thứ tự số học.
- Cách hoạt động:**
 - Dựa trên tập huấn luyện (training set):** Bạn có một tập dữ liệu với các mẫu (samples) đã được gán nhãn sẵn. Mỗi mẫu bao gồm các **thuộc tính (features)** và một **nhãn lớp (class label)** được gán trong thuộc tính phân loại (classifying attribute).
 - Xây dựng mô hình:** Một mô hình học máy được huấn luyện để tìm ra mối quan hệ giữa các thuộc tính và nhãn lớp.
 - Dự đoán dữ liệu mới:** Mô hình sử dụng các mối quan hệ đã học để gán nhãn lớp cho dữ liệu mới (chưa thấy trước đó).



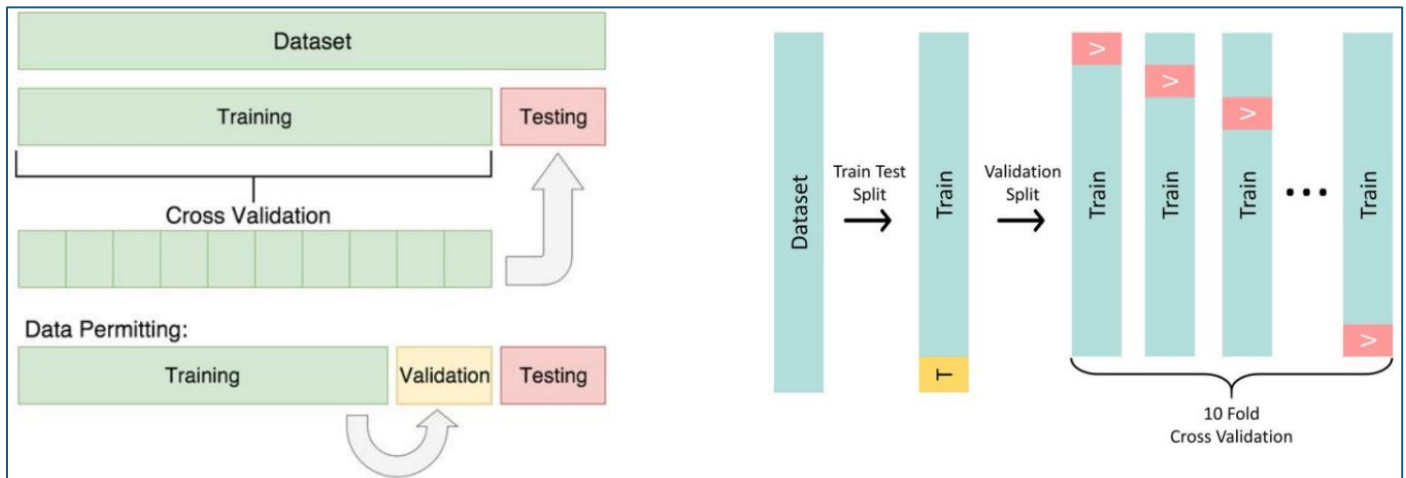
- Ví dụ:**
 - Phân loại email là "spam" hoặc "không spam" dựa trên nội dung email (features: số lượng từ khóa, độ dài email, v.v.).
 - Dự đoán loại hoa (setosa, versicolor, virginica) trong bộ dữ liệu Iris dựa trên các đặc điểm như chiều dài cánh hoa, chiều rộng lá.
 - Xác định một bệnh nhân có mắc bệnh hay không (có/không) dựa trên các triệu chứng.
- Output** (kết quả dự đoán) là **rời rạc**, ví dụ: các lớp như {0, 1} hoặc {A, B, C}.
- Some Applications of Machine Learning Classification Problems**
 - Image classification:** Phân loại hình ảnh
 - Fraud detection:** Phát hiện gian lận
 - Document classification:** Phân loại tài liệu
 - Spam filtering:** Lọc thư rác
 - Facial recognition:** Nhận diện khuôn mặt
 - Voice recognition:** Nhận diện giọng nói
 - Medical diagnostic test:** Xét nghiệm chẩn đoán y khoa
 - Customer behavior prediction:** Dự đoán hành vi khách hàng
 - Product categorization:** Phân loại sản phẩm
 - Malware classification:** Phân loại phần mềm độc hại

3.3.2. Numeric Prediction (Dự đoán số) / Regression (Hồi quy)

- Mục tiêu: Dự đoán một giá trị **liên tục (continuous)**.
- **Cách hoạt động:**
 - **Dựa trên tập huấn luyện:** Tập dữ liệu bao gồm các mẫu với các thuộc tính (features) và giá trị mục tiêu (target value) là số thực (liên tục).
 - **Xây dựng mô hình:** Mô hình học máy được huấn luyện để **tìm ra hàm (function) ánh xạ từ các thuộc tính đến giá trị liên tục**. Hàm này thường là một hàm liên tục, ví dụ, một đường thẳng trong Linear Regression hoặc một hàm phức tạp hơn.
 - **Dự đoán dữ liệu mới:** Mô hình dự đoán **giá trị số cho dữ liệu mới** dựa trên các thuộc tính của nó.
- Đầu ra của mô hình là một con số.
- **Ví dụ:**
 - Dự đoán giá nhà dựa trên diện tích, số phòng, và vị trí (giá có thể là 500 triệu, 501.2 triệu, hoặc bất kỳ số thực nào).
 - Dự đoán nhiệt độ ngày mai dựa trên dữ liệu thời tiết lịch sử (ví dụ: 25.7°C).
 - Dự đoán doanh thu bán hàng của một cửa hàng dựa trên chi tiêu quảng cáo và số lượng khách hàng.

3.4. Classification—Model Construction, Validation and Testing (Xây dựng, xác thực và kiểm tra mô hình)

- **Chia tập dữ liệu trong học máy**



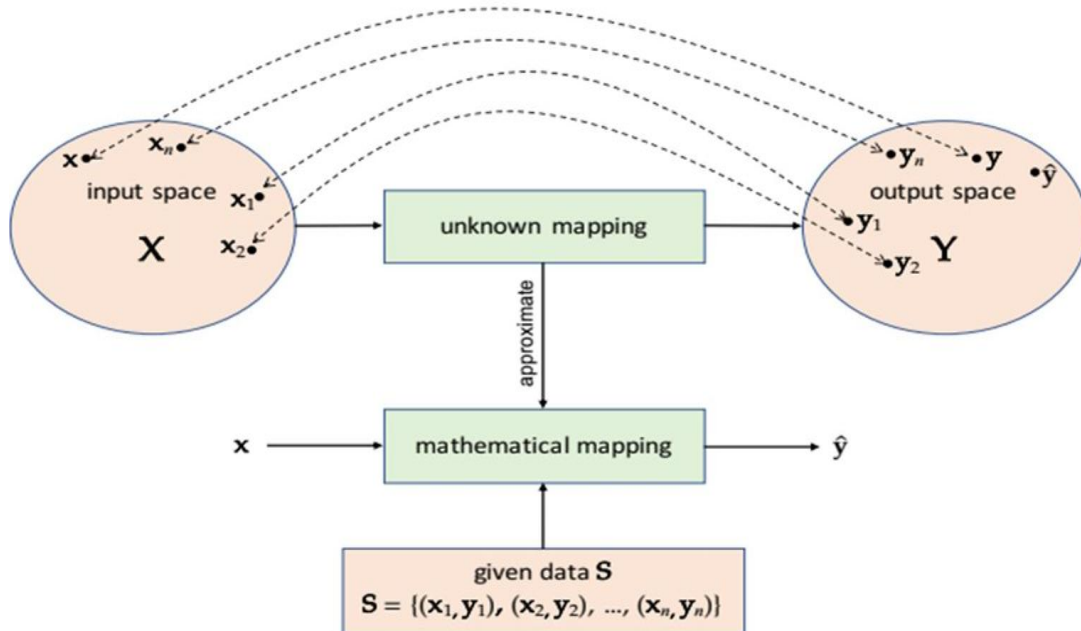
- **Train-Test Split (Chia Train và Test)**
 - Tập dữ liệu ban đầu (dataset) được chia thành hai phần chính:
 - **Training (Huấn luyện):** Phần lớn dữ liệu (thường 70-80%) được dùng để huấn luyện mô hình học máy. Mô hình học các quy tắc hoặc mẫu từ dữ liệu này.
 - **Testing (Kiểm tra):** Phần còn lại (thường 20-30%) được dùng để đánh giá hiệu suất mô hình sau khi huấn luyện, đảm bảo mô hình tổng quát hóa tốt trên dữ liệu mới.
 - **Ứng dụng:** Phương pháp này đơn giản và phổ biến, nhưng nhược điểm là hiệu suất có thể phụ thuộc vào cách chia ngẫu nhiên. Nếu tập test không đại diện, kết quả có thể không chính xác.
 - "Không đại diện" (unrepresentative) nghĩa là tập test không phản ánh đúng sự đa dạng hoặc phân phối của toàn bộ dữ liệu gốc (dataset).

- **Cross Validation (Xác Thực Chéo)**
 - Cross-validation là kỹ thuật cải tiến để đánh giá mô hình bằng cách chia tập dữ liệu thành **k** phần (folds) bằng nhau. Trong hình ảnh, bạn thấy **10-fold Cross Validation**, nghĩa là tập dữ liệu được chia thành 10 phần.
 - Lần lượt, mỗi phần được dùng làm **validation set** (tập xác nhận), trong khi các phần còn lại dùng làm **training set**.
 - Quy trình này lặp lại **k lần** (ở đây là 10 lần), và hiệu suất trung bình từ các lần lặp được tính để đánh giá mô hình.
 - **Lợi ích:** Giảm thiểu rủi ro do chia ngẫu nhiên trong train-test split, **tận dụng toàn bộ dữ liệu để huấn luyện và đánh giá, giúp đánh giá mô hình chính xác hơn.**
 - **Ứng dụng:** Thường dùng khi tập dữ liệu nhỏ, ví dụ: đánh giá mô hình phân loại hình ảnh hoặc dự đoán giá nhà.
- **Data Permitting with Validation (Chia Dữ Liệu với Sự Xác Nhận)**
 - Đây là một cách tiếp cận mở rộng của train-test split, nơi tập dữ liệu được chia thành ba phần:
 - **Training (Huấn luyện):** Dùng để huấn luyện mô hình.
 - **Validation (Xác nhận):** Dùng để điều chỉnh siêu tham số (hyperparameters) hoặc chọn mô hình tốt nhất trong quá trình huấn luyện.
 - **Testing (Kiểm tra):** Dùng để đánh giá cuối cùng, không được dùng trong quá trình huấn luyện hoặc điều chỉnh.
 - **Lợi ích:** Tách validation giúp tránh overfitting (quá khớp) với tập test, đảm bảo đánh giá cuối cùng khách quan.
 - **Ứng dụng:** Phổ biến trong các dự án lớn, ví dụ: huấn luyện mô hình nhận diện khuôn mặt hoặc dự đoán hành vi khách hàng.

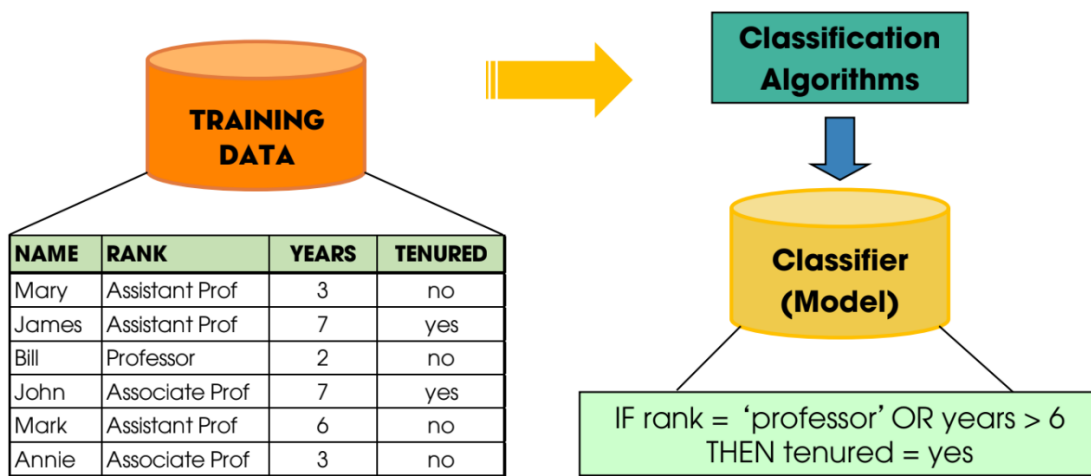
Chi tiết hơn về phương pháp chia dữ liệu thứ 3:

• Model Construction and Training

- **Model (Mô hình):** Là kết quả của quá trình học, có thể ở nhiều dạng như cây quyết định, bộ quy tắc, công thức toán học...
- **Assumption (Giả định):** Mỗi mẫu dữ liệu thuộc về một lớp đã được xác định trước (class label).
- **Training Set (Tập huấn luyện):** Là tập dữ liệu $S = \{(x_1, y_1), (x_2, y_2), \dots\}$ được dùng để xây dựng mô hình. Ở đây x là các đặc trưng và y là nhãn tương ứng.

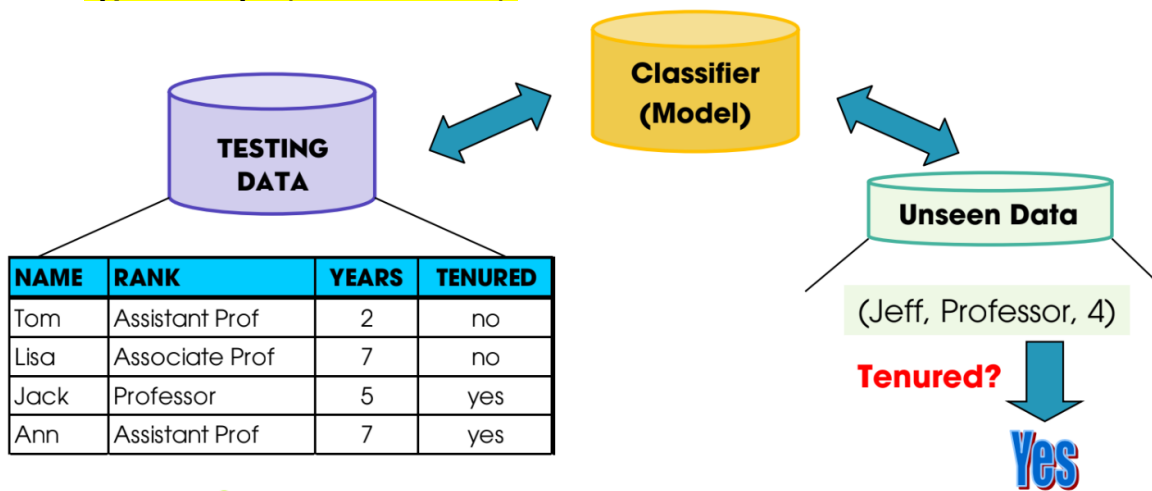


- Ta có một tập dữ liệu đầu vào X và đầu ra tương ứng Y.
- Giữa chúng tồn tại một mối quan hệ "bí ẩn" (unknown mapping) mà chúng ta không biết.
- Mục tiêu của học máy là từ tập dữ liệu huấn luyện S, học ra một mathematical mapping (hàm toán học) để **xấp xỉ (approximate)** tốt nhất mối quan hệ bí ẩn kia. Hàm này chính là mô hình dự đoán của chúng ta.



• Model Validation and Testing (Xác thực và Kiểm tra mô hình):

- **Test (Kiểm tra):** Sau khi xây dựng mô hình từ tập huấn luyện (training set), ta cần đánh giá xem nó hoạt động tốt đến đâu trên dữ liệu mới mà nó chưa từng thấy. Dữ liệu này gọi là **tập kiểm tra (test set)**.
- **Accuracy (Độ chính xác):** Là tỷ lệ phần trăm các mẫu trong tập test được mô hình phân loại đúng.
- **Quan trọng:** Tập test phải **độc lập (independent)** với tập train. Nếu bạn dùng chính dữ liệu đã học để kiểm tra, mô hình sẽ đạt điểm rất cao nhưng đó là "học vẹt", không có ý nghĩa thực tế.
- **Validation (Xác thực):** Nếu bạn dùng tập test để tinh chỉnh các tham số của mô hình (ví dụ: chọn mô hình A thay vì B vì A cho kết quả tốt hơn trên tập test), thì tập đó được gọi là **tập xác thực (validation set)**.



- **Model Deployment (Triển khai mô hình):** Nếu độ chính xác của mô hình đủ tốt và chấp nhận được, ta sẽ mang nó đi sử dụng trong thực tế để phân loại các dữ liệu hoàn toàn mới.

3.5. Decision Tree (Cây quyết định)

- Là một kỹ thuật phân lớp dữ liệu.

3.5.1. Định nghĩa Cây quyết định

- Là một cấu trúc phân cấp của các **node** và các **nhánh**

- **3 loại node**

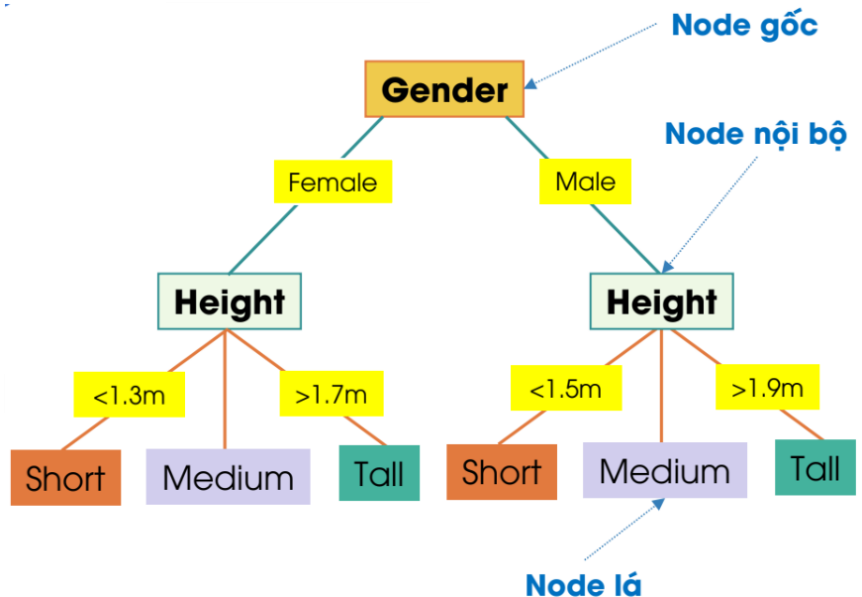
- Node gốc
- Node nội bộ: tên thuộc tính
- Node lá: tên lớp (class label)

- **Nhánh:** giá trị của thuộc tính

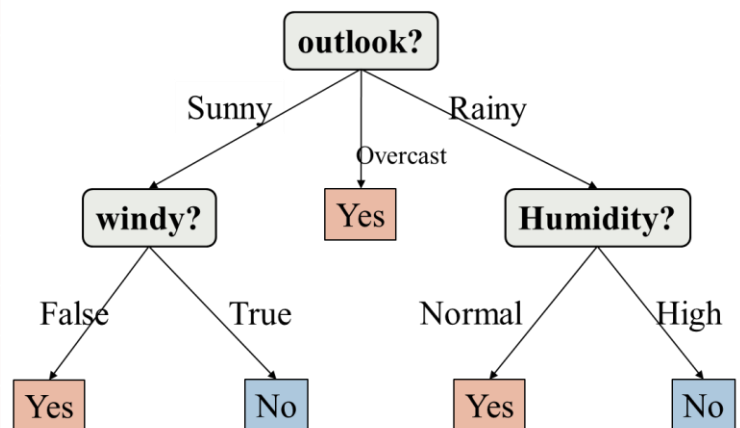
- **Decision tree construction**

(Xây dựng cây quyết định):

- Là một quá trình "**top-down, recursive, divide-and-conquer**" (từ trên xuống, đệ quy, chia để trị).
 - **Từ trên xuống:** Bắt đầu từ câu hỏi quan trọng nhất ở gốc (root).
 - **Chia để trị:** Tại mỗi bước, thuật toán chọn một đặc trưng (ví dụ: Outlook) để chia bộ dữ liệu thành các tập con nhỏ hơn.
 - **Đệ quy:** Quá trình này lặp lại cho mỗi tập con cho đến khi đạt điều kiện dừng (ví dụ: tất cả các mẫu trong tập con đều thuộc một lớp).



Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No



- **Cách dùng cây:** Để dự đoán cho một ngày mới, ví dụ: Outlook=Sunny, Temp=Hot, Humidity=High, Windy=True.
 - Bắt đầu từ gốc: Outlook là Sunny -> đi theo nhánh Sunny.
 - Đến nút windy?: Windy là True -> đi theo nhánh True.
 - Đến nút lá No.
 - Kết luận: Không đi chơi golf.
- **Lưu ý:** Nhánh Overcast đi thẳng đến Yes. Điều này có nghĩa là nếu thời tiết là Overcast, ta luôn đi chơi golf, bất kể các yếu tố khác.
- **Làm sao để chọn thuộc tính tốt nhất để phân chia tại mỗi nút (ví dụ: tại sao lại chọn Outlook ở gốc mà không phải Temp)?**
 - Dựa vào một thước đo **heuristic** (~kinh nghiệm) hoặc thống kê.
 - Các thước đo phổ biến là **information gain (độ lợi thông tin)** và **Gini index**.
 - Ý tưởng chung là chọn thuộc tính nào giúp "làm sạch" dữ liệu nhất, tức là sau khi chia, các tập con trở nên "thuần khiết" hơn (**chứa nhiều mẫu của cùng một lớp hơn**).
- **Conditions for stopping partitioning (Điều kiện dừng phân chia): Khi nào thì quá trình xây dựng cây dừng lại ở một nhánh?**
 1. Khi tất cả các mẫu tại một nút đều **thuộc cùng một lớp** (ví dụ: tất cả đều là "Yes"). Nút đó trở thành nút lá.
 2. Khi **không còn thuộc tính nào** để chia nữa, nhưng các mẫu vẫn lẫn lộn.
 3. Khi **không còn mẫu dữ liệu** nào ở nhánh đó.
- **Prediction (Dự đoán):**
 - Khi một nhánh dừng lại nhưng các mẫu vẫn còn lẫn lộn (điều kiện 2), làm thế nào để quyết định nhãn cho nút lá đó?
 - Câu trả lời: Dùng **Majority voting (Bỏ phiếu theo đa số)**. **Lớp nào chiếm đa số trong các mẫu ở nút đó sẽ được chọn làm nhãn cho nút lá**. Ví dụ, nếu có 3 "Yes" và 1 "No" thì nút lá sẽ được gán nhãn là "Yes".

3.5.2. How to Handle Continuous-Valued Attributes? (Xử lý thuộc tính giá trị liên tục)

- Cây quyết định hoạt động tốt với thuộc tính rời rạc (như Outlook có 3 giá trị Sunny, Overcast, Rainy). Vậy với thuộc tính liên tục như Tuổi hay Nhiệt độ thì sao?
- **Method 1: Discretize (Rời rạc hóa):**
 - Chia khoảng giá trị liên tục thành các khoảng rời rạc.
 - Ví dụ: age có thể được chia thành < 20 , $20..30$, $30..40$,...
 - Sau đó, xử lý như một thuộc tính rời rạc thông thường. Cách này đơn giản nhưng có thể làm mất thông tin.
- **Method 2: Find the best split point (Tìm điểm chia tốt nhất):** Đây là cách phổ biến và hiệu quả hơn.
 - **Bước 1:** Sắp xếp tất cả các giá trị của thuộc tính A theo thứ tự tăng dần. e.g. 15, 18, 21, 22, 24, 25, 29, 31, ...
 - **Bước 2:** Các điểm chia (split point) tiềm năng là điểm giữa của hai giá trị liên tiếp.
 - **$(a_i + a_{i+1})/2$**
 - Ví dụ, giữa 15 và 18, điểm chia có thể là $(15+18)/2 = 16.5$.

- **Bước 3:** Với mỗi điểm chia tiềm năng, ta tính **information gain** (phần sau).
- **Bước 4:** Chọn điểm chia nào cho **information gain lớn nhất (maximum)** làm điểm chia tốt nhất cho thuộc tính A.
- Sau đó, dữ liệu sẽ được chia thành hai nhóm: **$A \leq P$ và $A > P$** (với P là điểm chia tốt nhất).

- **Ưu điểm của cây quyết định**

- **Easy to explain (Dễ giải thích):** Ngay cả người không có chuyên môn cũng có thể hiểu được logic của cây. Các quy tắc if-then rất trực quan.
- **Easy to implement (Dễ cài đặt):** Có nhiều thư viện và phần mềm hỗ trợ.
- **Efficient (Hiệu quả):** Tốc độ dự đoán nhanh.
- **Can tolerant missing data (Chịu được dữ liệu thiếu):** Có các phương pháp để xử lý khi một vài giá trị bị thiếu.
- **White box (Hộp trắng):** Ngược lại với "hộp đen" (black box) như mạng nơ-ron, chúng ta có thể **nhìn vào bên trong và hiểu tại sao mô hình lại đưa ra quyết định như vậy**.
- **No need to normalize data (Không cần chuẩn hóa dữ liệu):** Không cần phải đưa dữ liệu về cùng một thang đo (ví dụ: từ 0 đến 1).
- **Non-parametric (Phi tham số):** **Không cần giả định trước về dạng phân phối** của dữ liệu.
- **Can work on various attribute types (Làm việc được với nhiều loại thuộc tính):** Cả số và chữ.

- **Nhược điểm của cây quyết định**

- **Unstable. Sensitive to noise (Không ổn định. Nhạy cảm với nhiễu):** Một **thay đổi nhỏ** trong dữ liệu huấn luyện có thể dẫn đến một **cấu trúc cây hoàn toàn khác**.
- **Accuracy may be not good enough (Độ chính xác có thể không đủ tốt):** So với các mô hình phức tạp hơn, cây quyết định đơn lẻ đôi khi không phải là mô hình chính xác nhất.
- **The optimal splitting is NP (Việc tìm cây tối ưu là bài toán NP-khó):** Tìm ra cây quyết định "tốt nhất tuyệt đối" là một bài toán cực kỳ phức tạp về mặt tính toán. Do đó, **các thuật toán thực tế chỉ dùng phương pháp "tham lam" (greedy) - tức là chỉ chọn cái tốt nhất ở mỗi bước**, điều này không đảm bảo sẽ tạo ra cây tốt nhất tổng thể.
- **Overfitting (Quá khớp):** Đây là nhược điểm lớn nhất. Mô hình học "quá kỹ" các chi tiết và nhiễu trong tập huấn luyện, đến nỗi nó hoạt động rất tệ trên dữ liệu mới. Giống như học vẹt.

- **Underfitting (Mô hình học chưa đủ)**

- Underfitting xảy ra khi **mô hình quá đơn giản, không học được quy luật cơ bản của dữ liệu**.
- **Đặc điểm:**
 - Sai số (error) **cao trên cả tập huấn luyện và tập kiểm tra**.
 - Mô hình không đủ "phức tạp" để mô tả dữ liệu.
- **Ví dụ:** Giả sử ta muốn dự đoán mối quan hệ giữa điểm kiểm tra và số giờ học. Nếu ta dùng **mô hình tuyến tính (đường thẳng)** trong khi dữ liệu thực tế có dạng **đường cong** → mô hình không theo kịp xu hướng thật → **underfitting**.
- **Cách khắc phục:**
 - Dùng **mô hình phức tạp hơn** (ví dụ: thêm đặc trưng, dùng mô hình phi tuyến).
 - Huấn luyện **lâu hơn**.
 - Giảm **regularization (ràng buộc)** nếu đang dùng quá mạnh.

- **Overfitting (Mô hình học quá mức)**

- Overfitting xảy ra khi **mô hình học quá chi tiết** — kể cả **noise** và các **điểm ngoại lai (outliers)** — trong dữ liệu huấn luyện.

- **Đặc điểm:**

- Sai số trên **tập huấn luyện rất nhỏ (học thuộc lòng)**.
- Nhưng sai số trên **tập kiểm tra lại rất cao** → không tổng quát tốt.

- **Ví dụ:** Cũng với bài toán trên, nếu ta dùng **đa thức bậc 10 (rất phức tạp)**, mô hình có thể **uốn lượn theo từng điểm dữ liệu huấn luyện**. Nhìn qua thì rất khớp dữ liệu học, nhưng khi gặp dữ liệu mới → dự đoán sai lệch nhiều.

- **Cách khắc phục:**

- Thu thập **nhiều dữ liệu hơn**.
- Dùng **mô hình đơn giản hơn**.
- Áp dụng **regularization (L1, L2)**.
- Dùng **kỹ thuật dừng sớm (early stopping)**.
- Dùng **cross-validation** để chọn mô hình phù hợp.

3.5.3. Splitting Measures: Information Gain (Entropy) (Thước đo phân chia: Độ lợi thông tin)

3.5.3.1 Entropy (Information Theory)

- A measure of uncertainty associated (**impurity**) (sự hỗn loạn/bất định) with a random number
- **Entropy** giống như **độ hỗn loạn** của một nhóm dữ liệu.
 - Nếu tất cả mẫu đều cùng 1 lớp → **Entropy = 0** → **chắc chắn 100%**
 - Nếu mẫu chia đều 50-50 giữa 2 lớp → **Entropy = 1** → **bất định cao nhất**
→ **Low impurity is the best**
- **Một số quy ước:**
 - **Y = Nhãn lớp (class label) cần dự đoán** → Cái bạn muốn biết: "Có mua hàng?", "Spam hay không?", "Bệnh hay không?"
 - Ví dụ:
 - Y = {Có, Không} → 2 lớp
 - Y = {Setosa, Versicolor, Virginica} → 3 lớp
 - **X = Thuộc tính (feature) dùng để chia dữ liệu** → Cái bạn dùng để hỏi: "Tuổi < 30?", "Thu nhập cao?", "Diện tích > 100m²?"
 - Ví dụ: X = Tuổi, X = Thu nhập, X = Giới tính
- **Độ bất định của nhãn Y (trước khi chia)/ Công thức Entropy:**

$$H(Y) = - \sum_{i=1}^m p_i \log(p_i)$$

Tham số	Ý nghĩa
Y	Biến ngẫu nhiên cần dự đoán (ví dụ: lớp "Có mua" hay "Không mua")
m	Số lớp khác nhau (ví dụ: 2 lớp → m = 2)
p_i	Xác suất của lớp thứ i → p_i = số mẫu lớp i / tổng số mẫu Ví dụ, trong tập dữ liệu "Play Golf" có 14 ngày, trong đó có 9 "Yes" và 5 "No". Vậy p _{yes} = 9/14 và p _{no} = 5/14.
log₂(p_i)	Logarit cơ số 2 (vì ta dùng bit để đo thông tin)
-	Dấu trừ để entropy luôn dương

○ Ví dụ minh họa:

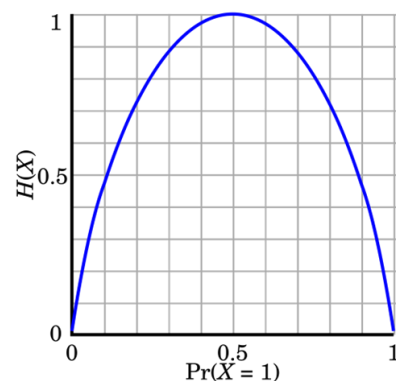
Dữ liệu	Số mẫu	Entropy
10 mẫu: 10 Có	10 Có	$H = 0$ (chắc chắn)
10 mẫu: 5 Có, 5 Không	50-50	$H = 1$ (bất định nhất)
10 mẫu: 8 Có, 2 Không	80-20	$H \approx 0.72$

$$H = -(0.8 \log_2 0.8 + 0.2 \log_2 0.2) \approx 0.72$$

Đồ thị entropy (m=2):

→ Đỉnh cao nhất tại $p = 0.5$ → bất định cao nhất

→ Xuống 0 khi $p = 0$ hoặc 1 → chắc chắn



m = 2

• **Conditional Entropy – Entropy "sau khi biết X"**

- Tính entropy riêng cho từng nhóm thuộc tính. Công thức phía trên là dữ liệu chưa được phân loại theo thuộc tính

$$H(Y|X) = \sum_x p(x) H(Y|X = x)$$

Ký hiệu	Ý nghĩa	Ví dụ minh họa
$H(Y X)$	Entropy của Y khi biết X – độ bất định trung bình của nhãn Y sau khi đã chia dữ liệu theo X	Sau khi chia theo “Tuổi”, còn bao nhiêu bất định trong “Có mua hàng?”
$\sum_x \dots$	Tổng trên tất cả các giá trị có thể của X	Duyệt qua từng nhóm: Tuổi <30, Tuổi ≥30
$p(x)$	Xác suất của giá trị x trong tập dữ liệu	$p(\text{Tuổi} < 30) = \frac{5}{14} \approx 0.36$
$H(Y X = x)$	Entropy của Y trong nhóm có X = x Sử dụng công thức Entropy (nhưng chỉ dành cho 1 nhóm thuộc tính)	Entropy của “Có mua hàng?” chỉ trong nhóm Tuổi <30

○ Ví dụ:

Dữ liệu: 14 người	Tuổi	Mua hàng?
5 người <30	4 Có, 1 Không	→ $H(Y \text{Tuổi} < 30) = 0.72$
9 người ≥30	5 Có, 4 Không	→ $H(Y \text{Tuổi} \geq 30) = 1.0$

Tính $H(Y | \text{Tuổi})$:

$$\begin{aligned}
 H(Y|X) &= p(<30) \cdot H(<30) + p(\geq 30) \cdot H(\geq 30) \\
 &= \frac{5}{14} \cdot 0.72 + \frac{9}{14} \cdot 1.0 \\
 &= 0.26 + 0.64 = \mathbf{0.90}
 \end{aligned}$$

• **Tính Information Gain:**

$$IG(Y|X) = H(Y) - H(Y|X)$$

- Cây quyết định chọn X có IG cao nhất → giảm bất định nhiều nhất!

3.5.3.2. Định nghĩa chính thức cách tính Information Gain để chọn thuộc tính tốt nhất

- Select the attribute with the **highest information gain** (used in typical decision tree induction algorithm: **ID3/C4.5**)

- Giả sử:

- **D**: tập huấn luyện
- **$C_{i,D}$** : tập các mẫu của D thuộc lớp C_i với $i = \{1, \dots, m\}$, **m** số lớp
- **$|C_{i,D}|, |D|$** : lực lượng của tập $C_{i,D}$ và D
- **p_i** : xác suất để một mẫu bất kỳ của D thuộc lớp C_i
- **$Info(D)$** : Thông tin kỳ vọng để phân lớp một mẫu trong D:

$$Info(D) = - \sum_{i=1}^m p_i \log_2 p_i \quad (1) \quad p_i = \frac{|C_{i,D}|}{|D|} \quad (2)$$

(Expected information (entropy) needed to classify a tuple in D)

- Độ lợi thông tin dựa trên phân chia theo 1 thuộc tính (A)

- Thuộc tính A có **v** các giá trị: $\{a_1, a_2, \dots, a_v\}$
- Dùng A để phân chia tập train D thành **v** tập con $\{D_1, D_2, \dots, D_v\}$
- **$Info_A(D)$** : Thông tin cần thiết để phân chia D theo thuộc tính A

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} * Info(D_j) \quad (3)$$

- **$Gain(A)$** : Độ lợi thông tin dựa trên phân chia theo thuộc tính A

$$Gain(A) = Info(D) - Info_A(D) \quad (4)$$

VD2: Cho tập dữ liệu với thuộc tính quyết định là buy_computer.

Giả sử:

- Lớp P: buy_computer = "yes"
- Lớp N: buy_computer = "no"

Tính độ lợi thông tin dựa trên phân chia theo thuộc tính "age"

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

- Thông tin kỳ vọng để phân lớp một mẫu trong D:

Ta có: $|D| = 14; |P| = 9; |N| = 5$

$$\Rightarrow \text{Info}(D) = I(9,5)$$

$$= -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = \mathbf{0.940}$$

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

- Thông tin cần thiết để phân chia D theo Age:

age	p_j	n_j	$I(p_j, n_j)$
<=30	2	3	0.971
31...40	4	0	0
>40	3	2	0.971

$$I(2,3) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5}$$

$$= 0.971$$

$$I(4,0) = -\frac{4}{4} \log_2 \frac{4}{4} - \frac{0}{4} \log_2 \frac{0}{4} = 0$$

$$I(3,2) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5}$$

$$= 0.971$$

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

$$\text{Info}_{age}(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} * \text{Info}(D_j)$$

$$= \frac{5}{14} I(2,3) + \frac{4}{14} I(4,0) + \frac{5}{14} I(3,2)$$

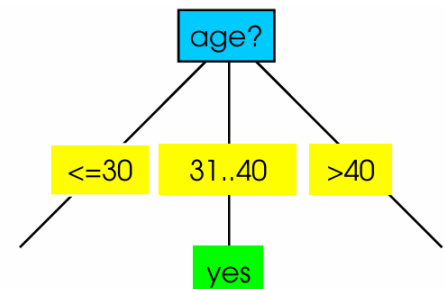
$$= \frac{5}{14} * 0.971 + \frac{4}{14} * 0 + \frac{5}{14} * 0.971 = \mathbf{0.694}$$

- Độ lợi thông tin khi phân chia D theo Age:

$$\text{Gain}(\text{age}) = \text{Info}(D) - \text{Info}_{age}(D)$$

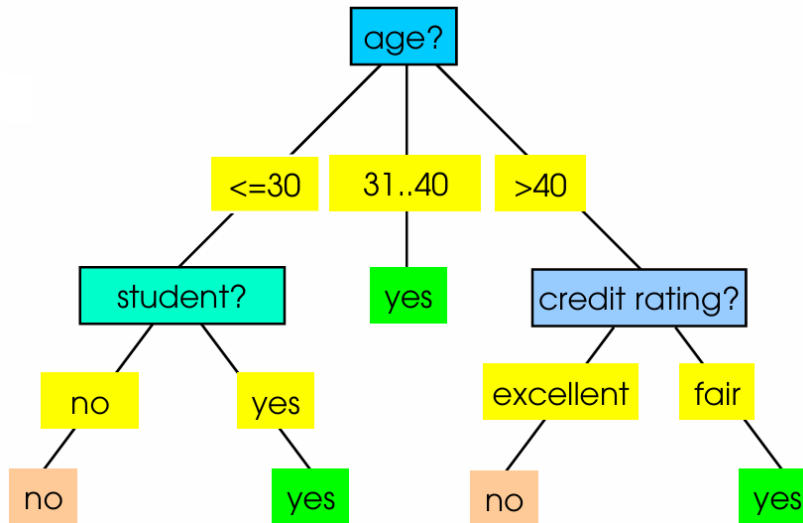
$$= 0.94 - 0.694 = \mathbf{0.246}$$

- Tiếp tục tính độ lợi thông tin dựa trên phân chia theo thuộc tính: "income", "student", "credit_rating"
 - o $\text{Gain}(\text{age}) = \mathbf{0.246}$
 - o $\text{Gain}(\text{income}) = 0.029$
 - o $\text{Gain}(\text{student}) = 0.151$
 - o $\text{Gain}(\text{credit_rating}) = 0.048$
- => Chọn thuộc tính **age**



Với age = "31..40", $I(p_j, n_j) = 0$
=> buy_computer = "yes"

- Thực hiện tương tự với ≤ 30 và > 40 .
 - **Xét nhánh Age ≤ 30 :** thực hiện tương tự, tính:
 - $\text{Gain}(\text{income}): 0.571$
 - $\text{Gain}(\text{student}): \mathbf{0.971}$
 - $\text{Gain}(\text{credit_rating}): 0.952$
 - => Chọn thuộc tính **student**
 - **Xét nhánh Age > 40 :** thực hiện tương tự, tính:
 - $\text{Gain}(\text{income}): 0.019$
 - $\text{Gain}(\text{student}): 0.003$
 - $\text{Gain}(\text{credit_rating}): \mathbf{0.971}$
 - => Chọn thuộc tính **credit_rating**



3.5.4. Gain Ratio: A Refined Measure for Attribute Selection (Tỷ lệ độ lợi: Một thước đo cải tiến)

- **Vấn đề của Information Gain:** Nó có xu hướng **thiên vị (biased)**, ưu tiên các thuộc tính có **nhiều giá trị khác nhau** (ví dụ: thuộc tính "ID khách hàng", mỗi khách hàng một giá trị). Thuộc tính ID sẽ chia dữ liệu thành rất nhiều nhánh nhỏ, mỗi nhánh chỉ có 1 mẫu -> mỗi nhánh đều "tinh khiết" tuyệt đối -> Information Gain sẽ rất cao. **Nhưng việc chia này không có ý nghĩa gì cho việc tổng quát hóa.**
- **Giải pháp: Gain Ratio (Tỷ lệ độ lợi):**
 - Gain Ratio khắc phục vấn đề này bằng cách "phạt" các thuộc tính chia dữ liệu thành quá nhiều mảnh nhỏ.
 - Nó thực hiện điều này bằng cách chia Information Gain cho một giá trị gọi là SplitInfo.
 - **SplitInfo:** Đo lường mức độ "phân mảnh" của dữ liệu do thuộc tính đó gây ra. Một thuộc tính chia dữ liệu thành nhiều nhánh nhỏ sẽ có SplitInfo cao.
 - **Gain Ratio = Information Gain + 1 bước "phạt" thêm → để quyết định chính xác hơn thuộc tính nào làm nút cha (root node).**

- Công thức:

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo_A(D)} \quad (5)$$

$$SplitInfo_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} * \log_2 \left(\frac{|D_j|}{|D|} \right) \quad (6)$$

- **Cách hoạt động:** Nếu một thuộc tính có Gain(A) cao nhưng cũng có SplitInfo(A) rất cao (vì nó chia thành nhiều nhánh), thì GainRatio của nó sẽ bị giảm xuống.
- **Thuật toán C4.5** (một phiên bản cải tiến của ID3) sử dụng Gain Ratio thay vì Information Gain.
- For all the predictor variables, **the one that gives the highest Gain Ratio is chosen for the split.**

VD3: Tính Gain Ratio dựa trên phân chia theo thuộc tính "income"

$$Gain(income) = 0.029$$

$$SplitInfo_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} * \log_2 \left(\frac{|D_j|}{|D|} \right)$$

$$= - \frac{4}{14} \log_2 \frac{4}{14} - \frac{6}{14} \log_2 \frac{6}{14} - \frac{4}{14} \log_2 \frac{4}{14}$$

$$= 1.557$$

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo_A(D)}$$

$$= \frac{0.029}{1.557} = 0.019$$

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

3.5.5. Gini index

- Áp dụng trong thuật toán CART, SLIQ, SPRINT
- Đo mức độ không thuần nhất (impurity) (hỗn loạn) của một tập dữ liệu (giống Entropy), nhưng dễ tính toán hơn (Faster calculate)
- Chọn thuộc tính có Gini index nhỏ nhất

- **Giả sử:**

- **D:** tập huấn luyện
- **$C_{i,D}$:** tập các mẫu của D thuộc lớp C_i với $i = \{1, \dots, m\}$, **m** số lớp
- **$|C_{i,D}|$, $|D|$:** lực lượng của tập $C_{i,D}$ và D
- **p_i :** xác suất để một mẫu bất kỳ của D thuộc lớp C_i
- **Gini(D):** Gini index của tập D:

$$gini(D) = 1 - \sum_{i=1}^m (p_i)^2 \quad (7)$$

$$p_i = \frac{|C_{i,D}|}{|D|} \quad (2)$$

- Gini index dựa trên phân chia theo 1 thuộc tính (A)

- Thuộc tính A có v các giá trị: $\{a_1, a_2, \dots, a_v\}$
- Dùng A để phân chia tập train D thành v tập con $\{D_1, D_2, \dots, D_v\}$
- Gini index của phân chia D theo thuộc tính A:

$$gini_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} * gini(D_j) \quad (8)$$

VD4: Tính Gini index của tập D

Ta có: $|D| = 14$; $|P| = 9$; $|N| = 5$

$$\Rightarrow Gini(D) = 1 - \left[\left(\frac{9}{14} \right)^2 + \left(\frac{5}{14} \right)^2 \right] = 0.459$$

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Tính Gini index cho thuộc tính "age":

age	p_j	n_j	Gini(p_j, n_j)
<=30	2	3	0.48
31...40	4	0	0
>40	3	2	0.48

$$Gini(2,3) = 1 - \left[\left(\frac{2}{5} \right)^2 + \left(\frac{3}{5} \right)^2 \right] = 0.48$$

$$Gini(4,0) = 1 - \left[\left(\frac{4}{4} \right)^2 + \left(\frac{0}{4} \right)^2 \right] = 0$$

$$Gini(3,2) = 1 - \left[\left(\frac{3}{5} \right)^2 + \left(\frac{2}{5} \right)^2 \right] = 0.48$$

$$\begin{aligned}
 gini_{age}(D) &= \sum_{j=1}^v \frac{|D_j|}{|D|} * gini(D_j) \\
 &= \frac{5}{14} gini(2,3) + \frac{4}{14} gini(4,0) + \frac{5}{14} gini(3,2) \\
 &= \frac{5}{14} * 0.48 + \frac{4}{14} * 0 + \frac{5}{14} * 0.48 = 0.343
 \end{aligned}$$

- Thực hiện tương tự ta có:
 - Gini(age) = **0.343**
 - Gini(income) = 0.44
 - Gini(student) = 0.367
 - Gini(credit_rating) = 0.429
 => Chọn thuộc tính **age**
- Thực hiện tương tự với trường hợp <=30 và >40, ta cũng có cây quyết định như ví dụ ở 3.5.3.1.1.

3.5.6. So sánh các độ đo chọn thuộc tính

- **Information Gain:** Thiên vị thuộc tính có nhiều giá trị
- **Gain Ratio:** Thiên về việc phân chia không cân bằng giữa các vùng
- **Gini index:**
 - Thiên vị thuộc tính có nhiều giá trị
 - Gặp vấn đề khi số lớp lớn
 - Có xu hướng ưu tiên cho các thử nghiệm tạo ra kết quả phân chia có cùng kích thước và cùng sự đồng nhất

3.5.7. Các thuật toán và độ đo khác của cây quyết định

3.5.7.1. ID3 (Iterative Dichotomiser 3)

- **Tổng quan:** Được phát triển bởi Ross Quinlan năm 1986. Đây là thuật toán cơ bản để xây dựng cây quyết định đa nhánh (multiway tree), chủ yếu dùng cho phân loại (classification). Nó là mã nguồn mở (open source) và thường được áp dụng trong xử lý ngôn ngữ tự nhiên hoặc học máy cơ bản.
- **Quy trình:**
 - Sử dụng cách tiếp cận tham lam (greedy approach): Tại mỗi nút, chọn đặc trưng categorical mang lại **Information Gain (IG) cao nhất**, tính dựa trên **Entropy**.
 - Xây dựng cây đến kích thước tối đa (grow to max size), sau đó áp dụng **pruning** (tỉa cây) để tránh **overfitting**.
 - **Overfitting:** when model is too complex, it **models the details of the training set and fails on the test set**.
- **Ưu điểm và hạn chế:**
 - **Ưu:** Đơn giản, dễ hiểu, tạo cây nhỏ và hiệu quả cho dữ liệu categorical (ví dụ: chiều cao với giá trị "short, normal, tall").
 - **Hạn chế:**
 - Không có cơ chế pruning tích hợp sẵn (thường phải làm thủ công sau);
 - chỉ xử lý tốt categorical features (không hỗ trợ numerical/continuous trực tiếp, cần discretize trước);
 - dễ overfitting;
 - thiên vị với features có nhiều giá trị (bias towards multi-valued attributes).
- **Ví dụ:** Với dữ liệu về chiều cao (short, normal, tall), ID3 sẽ chọn feature giảm entropy nhiều nhất để chia.
- ID3 **không xử lý missing data tốt và không hỗ trợ regression**. Nó là nền tảng cho các phiên bản sau.

3.5.7.2. C4.5

- **Tổng quan:** Là phiên bản kế thừa của ID3, cũng bởi Ross Quinlan, dùng cho classification và mã nguồn mở (triển khai dưới tên J48 trong Weka). Nó khắc phục nhiều hạn chế của ID3, trở thành một trong những thuật toán decision tree phổ biến nhất.
- **Quy trình:**
 - Chọn đặc trưng dựa trên **Gain Ratio** (tỷ lệ gain, là IG được chuẩn hóa để tránh bias với features có nhiều giá trị).
 - Xử lý dữ liệu numerical bằng cách discretize động (tìm điểm chia tối ưu, ví dụ: chiều cao $> 1m71$ hoặc $\leq 1m71$, tạo khoảng interval như 1m65-1m7, 1m75-1m8,...).
 - Xây dựng cây, sau đó áp dụng **post-pruning** (tỉa sau) tích hợp sẵn để loại bỏ nhánh không cần thiết, cải thiện tổng quát hóa.
- **Ưu điểm và hạn chế:**
 - **Ưu:** Xử lý cả numerical và categorical; xử lý missing data (bỏ qua missing trong tính toán gain/entropy); có thể chuyển cây thành quy tắc if-then để dễ diễn giải hơn; giảm bias so với ID3.
 - **Hạn chế:** Tốn tài nguyên hơn ID3; pruning có thể làm mất một số thông tin chi tiết.
- **Ví dụ:** Với dữ liệu numerical như chiều cao, C4.5 tự động tìm ngưỡng chia (threshold) để tối ưu hóa gain ratio.
- C4.5 còn xử lý attributes có cost khác nhau và có thể tạo rule sets từ cây, giúp dễ áp dụng hơn ID3.

3.5.7.3. C5.0

- **Tổng quan:** Phiên bản cải tiến mới nhất của C4.5 bởi Quinlan, dùng cho classification. Không hoàn toàn open source mà là proprietary (thương mại), nhưng có phiên bản GPL cho Linux single-thread. Nó tối ưu hóa hiệu suất so với C4.5.
- **Quy trình:**
 - Tương tự C4.5: Sử dụng **Entropy để tính Gain Ratio** làm tiêu chí chia.
 - Bao gồm pruning tích hợp và các cải tiến như boosting (kết hợp nhiều cây để tăng độ chính xác).
- **Ưu điểm và hạn chế:**
 - **Ưu:** Chạy nhanh hơn C4.5 (nhanh gấp nhiều lần), sử dụng ít bộ nhớ hơn, tạo cây nhỏ hơn nhưng độ chính xác tương đương hoặc cao hơn; hỗ trợ weighting (trọng số cho cases/misclassifications), winnowing (tự động loại attributes không hữu ích).
 - **Hạn chế:** Proprietary nên không miễn phí hoàn toàn; ít linh hoạt cho tùy chỉnh so với open source.
- **Ví dụ:** Giống C4.5 nhưng hiệu quả hơn với dữ liệu lớn.
- C5.0 thường được dùng trong môi trường thương mại, và nó nhỏ gọn hơn, phù hợp cho thiết bị hạn chế tài nguyên.

3.5.7.4. CART (Classification and Regression Trees)

- **Tổng quan:** Được phát triển bởi Breiman et al. năm 1984, dùng cho cả classification và regression, mã nguồn mở (triển khai trong scikit-learn). Nó xây dựng cây nhị phân (binary tree), luôn chia thành 2 nhánh.
- **Quy trình:**
 - Chọn **feature** và **threshold** mang lại **best split** dựa trên **Gini Impurity** cho classification. Đối với regression, dùng variance reduction (giảm phương sai).
 - Xây dựng cây nhị phân, sau đó pruning để tránh overfitting.
- **Ưu điểm và hạn chế:**
 - **Ưu:** Xử lý cả numerical và categorical; hỗ trợ regression (dự đoán giá trị liên tục); Gini nhanh hơn entropy (vì không dùng log); cây cân bằng hơn nhờ binary splits.
 - **Hạn chế:** Có thể bias với features multi-level; nhạy cảm với thay đổi nhỏ trong dữ liệu; không tạo rule sets như C4.5/C5.0.
- **Ví dụ:** Với classification, chia nhị phân như "Height > 1m71: Yes/No"; với regression, dự đoán giá nhà dựa trên features.
- CART là nền tảng cho các ensemble như Random Forest; nó không hỗ trợ categorical trực tiếp trong một số triển khai (như scikit-learn, cần one-hot encoding).

3.5.7.4. CHAID (Chi-squared Automatic Interaction Detection)

- Một thuật toán decision tree phổ biến, sử dụng **chi-square test (χ^2)** để kiểm tra sự độc lập giữa predictors và response.
- Nó tạo multiway splits dựa trên mức ý nghĩa thống kê (p-value), thường dùng cho categorical data.
- So với ID3/CART, CHAID ít bị overfitting hơn vì dừng sớm nếu không có sự khác biệt ý nghĩa, nhưng chậm hơn với dữ liệu lớn.

3.5.7.5. C-SEP

- Một measure splitting thay thế, hoạt động tốt hơn information gain hoặc Gini trong một số trường hợp cụ thể (như dữ liệu imbalanced), nhưng ít phổ biến hơn.

3.5.7.6. MDL (Minimal Description Length)

- Nguyên tắc chọn cây đơn giản nhất (simple is preferred), đo bằng số bit cần để mã hóa cây và exceptions.
- Nó ưu tiên cây nhỏ để tránh phức tạp không cần thiết.

3.5.7.7. Multivariate splits

- Chia dựa trên tổ hợp nhiều variables (ví dụ: linear combination trong CART), thay vì univariate (một variable).
- Điều này phức tạp hơn nhưng chính xác hơn trong một số trường hợp.

3.5.7.8. Measure nào tốt nhất?

- Không có measure nào vượt trội hoàn toàn
- hầu hết (Info Gain, Gini, Gain Ratio, χ^2) đều cho kết quả tốt tùy dữ liệu.
- Gini nhanh hơn entropy, nhưng entropy xử lý tốt hơn với dữ liệu probabilistic.
- Trong thực tế, chọn dựa trên công cụ (ví dụ: scikit-learn dùng Gini cho CART).

3.6. Bayes Classification Methods (Các phương pháp phân loại Bayes)

3.6.1. Bayes' Theorem

- Phân lớp theo mô hình xác suất
- Chúng ta có một **đối tượng X** (ví dụ: một email, một bệnh nhân, một ảnh...), gồm nhiều **đặc trưng (features)**: $x_1, x_2, \dots, x_n$. Chúng ta muốn **phân loại X vào một lớp (class) H** nào đó (ví dụ: "spam" hay "không spam").

- Theo **định lý Bayes**, **xác suất để X thuộc lớp H** là:
$$P(H|X) = \frac{P(X|H) \cdot P(H)}{P(X)}$$
 - **X**: là dữ liệu của chúng ta (bằng chứng - "evidence"). Ví dụ: một email.
 - **H**: là một giả thuyết (hypothesis), ở đây là một lớp. Ví dụ: email này là "Spam" (nhãn).
 - **P(H|X)** (**Xác suất hậu nghiệm - posteriori probability**): **Xác suất để giả thuyết H là đúng, khi đã biết bằng chứng X**. Đây là thứ chúng ta muốn tìm. Ví dụ: Xác suất email là "Spam", khi đã biết nội dung của nó. (**What we should choose**)
 - **P(X|H)** (**Likelihood**): **Xác suất để thấy bằng chứng X, nếu giả định giả thuyết H là đúng**. Ví dụ: Xác suất để một email chứa từ "mua ngay", nếu giả định nó là "Spam". (**What we just see**)
 - **Xác suất xảy ra X khi đã biết nó thuộc lớp H.**
 - **P(H)** (**Xác suất tiên nghiệm - prior probability**): **Xác suất của giả thuyết H, trước khi có bất kỳ bằng chứng nào**. Ví dụ: Tỷ lệ email "Spam" nói chung trong tất cả các email. (**What we knew previously**)
 - **Xác suất lớp H xuất hiện (tần suất lớp).**
 - **P(X)**: Xác suất thấy bằng chứng X.
- **Mục tiêu phân loại**: Với một dữ liệu X, ta sẽ tính P(H|X) cho tất cả các lớp H có thể có (ví dụ: "Spam" và "Không Spam"). **Lớp nào có xác suất P(H|X) cao nhất thì ta sẽ gán X vào lớp đó**.
 - Vì P(X) giống nhau cho mọi lớp $\rightarrow$ ta chỉ cần so sánh $P(X|H) \times P(H)$ để chọn lớp có giá trị lớn nhất.
- **Ví dụ**:
 - **Bài toán**: **Sáng nay trời nhiều mây (Cloud)**. **Vậy xác suất trời sẽ mưa (Rain) là bao nhiêu?** Tức là tính $P(\text{Rain} | \text{Cloud})$.
 - **Thông tin đã biết (Prior & Likelihood)**:
 - $P(\text{Cloud} | \text{Rain}) = 50\%$: 50% các ngày mưa thì bắt đầu bằng trời mây. (Likelihood)
 - $P(\text{Cloud}) = 40\%$: Trời mây vào buổi sáng là phổ biến, chiếm 40% các ngày.
 - $P(\text{Rain}) = 10\%$: Đây là tháng khô, chỉ có 10% các ngày là có mưa. (Prior)
 - **Áp dụng công thức Bayes**:
$$P(\text{Rain} | \text{Cloud}) = \frac{P(\text{Cloud} | \text{Rain}) \cdot P(\text{Rain})}{P(\text{Cloud})}$$
$$= (50\% * 10\%) / 40\% = 5\% / 40\% = 12.5\%$$

- **Kết luận:** Mặc dù trời mây, nhưng xác suất mưa chỉ là 12.5%, không cao như ta tưởng. Lý do là vì xác suất tiên nghiệm của việc trời mưa ($P(\text{Rain})$) vốn đã rất thấp (chỉ 10%).
- **Ý nghĩa:** Định lý Bayes giúp chúng ta cập nhật niềm tin của mình ($P(\text{Rain})$) dựa trên bằng chứng mới (Cloud) để có được một nhận định chính xác hơn ($P(\text{Rain} | \text{Cloud})$).

3.6.2. Naïve Bayes Classifier

- **Vấn đề:** $P(X|H)$ rất khó tính, vì $X = (x_1, x_2, \dots, x_n) \rightarrow$ có rất nhiều đặc trưng. Ví dụ x_1 là "có từ 'mua'", x_2 là "có từ 'ngay'", v.v...
 - Theo lý thuyết: $P(X|H) = P(x_1|H) \times P(x_2|x_1, H) \times P(x_3|x_1, x_2, H) \times \dots \times P(x_n|x_1, \dots, x_{n-1}, H)$
 - $\rightarrow$ **Quá phức tạp!** Vì phải tính xác suất có điều kiện giữa hàng loạt đặc trưng, các đặc trưng có thể phụ thuộc lẫn nhau (ví dụ: từ "mua" và "ngay" thường đi liền nhau).
- **Giả định Ngây thơ (Naïve Assumption):**
 - Naive Bayes **giả định rằng tất cả các đặc trưng là độc lập có điều kiện với nhau, khi đã biết lớp H.**
 - Nói một cách đơn giản: sự xuất hiện của từ "mua" không ảnh hưởng đến sự xuất hiện của từ "ngay", miễn là chúng ta đã biết email này thuộc lớp "Spam".
 - **Tại sao lại "ngây thơ"?** Vì giả định này hầu như luôn sai trong thực tế. Các từ ngữ trong một câu rõ ràng là có liên quan đến nhau.
- Công thức $P(X|H)$ được đơn giản hóa đi rất nhiều:

$$P(X|H) \approx P(x_1|H) \times P(x_2|H) \times \dots \times P(x_n|H)$$

 - Thay vì phải tính một xác suất đồng thời phức tạp, giờ đây ta chỉ cần tính các xác suất riêng lẻ của từng đặc trưng rồi nhân chúng lại với nhau.
 - Về cơ bản chỉ là **"đếm" tần suất xuất hiện của từng đặc trưng** trong mỗi lớp.
 - $\rightarrow$ **Để phân loại**, ta tính cho mỗi lớp H_i , rồi chọn lớp có giá trị lớn nhất.

• If feature x_k is categorical

- Dùng **tần suất đếm**: $P(x_i = v | C) = (\text{số mẫu trong } C \text{ có } x_i = v) / (\text{tổng số mẫu trong } C)$
- **Ví dụ thực tế:**
 - Lớp "spam": 100 email
 - Trong đó 60 email có từ "miễn phí" $\rightarrow P(\text{"miễn phí"} | \text{spam}) = 60/100 = 0.6$
- **Lưu ý:** Nếu một giá trị chưa từng xuất hiện $\rightarrow$ xác suất = 0 $\rightarrow$ lỗi! $\rightarrow$ Dùng **Laplace Smoothing**:

$$P(x_i = v | C) = \frac{\text{đếm}(x_i = v \text{ trong } C) + 1}{\text{tổng mẫu trong } C + \text{số giá trị có thể của } x_i}$$

- Giả sử bạn đang phân loại email: **Spam** hay **Không Spam**

Từ "miễn phí"	Lớp	$\rightarrow$ Dữ liệu huấn luyện: Spam: 2 email, Không: 1 email
Có	Spam	Tính xác suất:
Có	Spam	$P(\text{"miễn phí"}=\text{Có} \text{Spam}) = 2/2 = 1.0$
Không	Không	$P(\text{"miễn phí"}=\text{Có} \text{Không Spam}) = 0/1 = 0.0$

- Dù email có thể là "Không spam", có từ "miễn phí", nhưng vì từ "miễn phí" chưa từng thấy ở lớp Không $\rightarrow$ xác suất = 0 $\rightarrow$ bị loại hoàn toàn!
- **RẤT NGUY HIỂM!** $\rightarrow$ Gọi là **zero probability problem**

○ **Áp dụng Laplace:**

- Từ "miễn phí": có 2 giá trị → Có, Không → số giá trị có thể = 2
- Xét lớp Spam

Giá trị	Đếm thật	Sau Laplace	Mẫu số
Có (trong Spam)	2	$2+1 = 3$	$2+2 = 4$
Không (Spam)	0	$0+1 = 1$	4

→ $P(\text{"miễn phí"}=\text{Có} \mid \text{Spam}) = 3/4 = 0.75$

→ $P(\text{"miễn phí"}=\text{Không} \mid \text{Spam}) = 1/4 = 0.25$

- Xét lớp Không Spam

Giá trị	Đếm thật	Sau Laplace	Mẫu số
Có (trong Không)	0	$0+1 = 1$	$1+2 = 3$
Không (Không)	1	$1+1 = 2$	3

→ $P(\text{"miễn phí"}=\text{Có} \mid \text{Không}) = 1/3 \approx 0.333$

→ $P(\text{"miễn phí"}=\text{Không} \mid \text{Không}) = 2/3 \approx 0.667$

Không còn xác suất = 0 nữa!

• **If feature x_k is continuous-valued**

- Ví dụ: nhiệt độ, độ ẩm. Chúng ta không thể đếm được vì có vô số giá trị.
- **Giải pháp:** Giả định rằng giá trị của đặc trưng này trong mỗi lớp tuân theo một **phân phối xác suất** nào đó, phổ biến nhất là **Phân phối Gaussian (Phân phối chuẩn - hình chuông)**.
- **Cách làm:**
 1. Với mỗi lớp C_i , tính giá trị trung bình (μ_{C_i}) và độ lệch chuẩn (σ_{C_i}) của đặc trưng x_k từ dữ liệu huấn luyện.
 2. Sử dụng công thức của hàm mật độ xác suất Gaussian:

$$p(\mathbf{x}_k = \mathbf{v}_k \mid C_i) = N(\mathbf{x}_k \mid \mu_{C_i}, \sigma_{C_i}) = \frac{1}{\sqrt{2\pi}\sigma_{C_i}} e^{-\frac{(x-\mu_{C_i})^2}{2\sigma^2}}$$

- Về cơ bản, ta xem xét giá trị v_k có gần với "trung tâm" (mean) của lớp C_i hay không. Càng gần thì xác suất càng cao.

- Example

Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

- $P(\text{Yes} \mid \text{Outlook}=\text{Sunny})$
 - $P(\text{Yes} \mid \text{Sunny}) \propto P(\text{Sunny} \mid \text{Yes}) * P(\text{Yes})$.
 - (Ta bỏ qua $P(\text{Sunny})$ ở mẫu số vì chỉ cần so sánh).

$$P(x \mid c) = P(\text{Sunny} \mid \text{Yes}) = 3 / 9 = 0.33$$

Frequency Table		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3



Likelihood Table		Play Golf		
		Yes	No	
Outlook	Sunny	3/9	2/5	5/14
	Overcast	4/9	0/5	4/14
	Rainy	2/9	3/5	5/14
		9/14	5/14	

$$P(x) = P(\text{Sunny}) = 5 / 14 = 0.36$$

$$P(c) = P(\text{Yes}) = 9 / 14 = 0.64$$

Posterior Probability: $P(c \mid x) = P(\text{Yes} \mid \text{Sunny}) = 0.33 \times 0.64 \div 0.36 = 0.60$



- **Likelihood $P(x \mid c) = P(\text{Sunny} \mid \text{Yes})$:**
 - Nhìn vào cột "Yes", có tổng cộng 9 mẫu.
 - Trong 9 mẫu "Yes" đó, có 3 mẫu có Outlook=Sunny.
 - Vậy $P(\text{Sunny} \mid \text{Yes}) = 3/9 = 0.33$.
- **Prior $P(c) = P(\text{Yes})$:**
 - Trong tổng số 14 mẫu, có 9 mẫu là "Yes".
 - Vậy $P(\text{Yes}) = 9/14 = 0.64$.

- **Tính $P(x) = P(\text{Sunny})$:** Trong 14 mẫu, có 5 mẫu Outlook=Sunny. Vậy $P(\text{Sunny}) = 5/14 = 0.36$.
- **Kết quả:** $P(\text{Yes} | \text{Sunny}) = (0.33 * 0.64) / 0.36 = 0.60$.
- Tính toán tương tự cho trường hợp **$P(\text{No} | \text{Outlook}=\text{Sunny})$** .

$P(x | c) = P(\text{Sunny} | \text{No}) = 2 / 5 = 0.4$

Frequency Table		Play Golf		
		Yes	No	
Outlook	Sunny	3	2	5
	Overcast	4	0	4
	Rainy	2	3	5
		9	5	14

$P(x) = P(\text{Sunny}) = 5 / 14 = 0.36$

$P(c) = P(\text{No}) = 5 / 14 = 0.36$

Posterior Probability: $P(c | x) = P(\text{No} | \text{Sunny}) = 0.40 \times 0.36 \div 0.36 = 0.40$

- Vì $P(\text{Yes} | \text{Sunny}) > P(\text{No} | \text{Sunny})$, nếu một ngày mới có Outlook=Sunny, mô hình sẽ dự đoán là **Yes** (đi chơi golf).
- **Likelihood Tables (Bảng Likelihood)**
 - Để chuẩn bị cho việc phân loại một mẫu dữ liệu đầy đủ (có cả 4 đặc trưng), chúng ta cần tính trước tất cả các xác suất có điều kiện $P(\text{feature_value} | \text{class})$.
 - **Frequency Table (Bảng tần suất):** Đếm số lần xuất hiện. Ví dụ, cho Humidity, có 3 lần High đi với Yes và 4 lần High đi với No.
 - **Likelihood Table (Bảng Likelihood):** Chuyển đổi từ số đếm sang xác suất.
 - Ví dụ, cho Humidity, $P(\text{High} | \text{Yes}) = 3/9$ (vì có 9 mẫu "Yes"). $P(\text{High} | \text{No}) = 4/5$ (vì có 5 mẫu "No").

Frequency Table				Likelihood Table			
		Play Golf				Play Golf	
		Yes	No			Yes	No
Outlook	Sunny	3	2	Outlook	Sunny	3/9	2/5
	Overcast	4	0		Overcast	4/9	0/5
	Rainy	2	3		Rainy	2/9	3/5
Humidity	High	3	4	Humidity	High	3/9	4/5
	Normal	6	1		Normal	6/9	1/5
Temp.	Hot	2	2	Temp.	Hot	2/9	2/5
	Mild	4	2		Mild	4/9	2/5
	Cool	3	1		Cool	3/9	1/5
Windy	False	6	2	Windy	False	6/9	2/5
	True	3	3		True	3/9	3/5

- Sử dụng các bảng likelihood đã tính để phân loại một mẫu dữ liệu mới.

Outlook	Temp	Humidity	Windy	Play
Rainy	Cool	High	True	?

$$P(Yes | X) = P(Rainy | Yes) \times P(Cool | Yes) \times P(High | Yes) \times P(True | Yes) \times P(Yes) / P(x)$$

$$P(Yes | X) = \frac{2/9 \times 3/9 \times 3/9 \times 3/9 \times 9/14}{P(x)} = 0.2 = \frac{0.00529}{0.02057 + 0.00529}$$

$$P(No | X) = P(Rainy | No) \times P(Cool | No) \times P(High | No) \times P(True | No) \times P(No) / P(x)$$

$$P(No | X) = \frac{3/5 \times 1/5 \times 4/5 \times 3/5 \times 5/14}{P(x)} = 0.8 = \frac{0.02057}{0.02057 + 0.00529}$$

$$P(x) = P(x | yes) * P(yes) + P(x | no) * P(no)$$

- **Kết luận:** P(No|X) lớn hơn, vì vậy mô hình dự đoán là **No** (không đi chơi golf).
- **P(x) = P(x|yes)*P(yes) + P(x|no)*P(no)** được gọi là công thức xác suất đầy đủ.

3.6.3. Strength vs. Weakness (Điểm mạnh và điểm yếu)

• Strength

- **Performance (Hiệu suất):** Mặc dù giả định "ngây thơ", Naive Bayes thường có hiệu suất tốt, có thể so sánh được với các mô hình phức tạp hơn như cây quyết định hay mạng nơ-ron trong nhiều bài toán.
- **Incremental (Tính gia tăng):**
 - **Naive Bayes là một mô hình "học online"** rất tốt. Khi có thêm một mẫu dữ liệu huấn luyện mới, ta không cần phải huấn luyện lại từ đầu. Ta chỉ cần cập nhật lại các số đếm (và các xác suất tương ứng).
 - Điều này cho phép kết hợp kiến thức có sẵn (prior knowledge) với dữ liệu quan sát được một cách linh hoạt.

• Weakness

- **Điểm yếu lớn nhất:** Chính là **giả định độc lập có điều kiện** của các thuộc tính.
 - Trong thực tế, các thuộc tính thường có sự phụ thuộc lẫn nhau. Ví dụ, trong y học, Patient's Profile (tuổi, tiền sử gia đình), Patient's Symptoms (sốt, ho), và Patient's Disease (ung thư phổi, tiểu đường) rõ ràng là có liên quan.
 - Việc bỏ qua các mối quan hệ này có thể dẫn đến **mất mát độ chính xác (loss of accuracy)**.
- **Cách khắc phục:**
 - Nếu các mối quan hệ phụ thuộc này là quan trọng và không thể bỏ qua, chúng ta cần sử dụng các mô hình phức tạp hơn có thể mô hình hóa được chúng.
 - Một ví dụ điển hình là **Bayesian Belief Networks (Mạng tin tưởng Bayes)**. Chúng là một sự mở rộng của Naive Bayes, cho phép định nghĩa các mối quan hệ phụ thuộc giữa các biến.

3.7. Lazy vs. Eager Learning (Học lười và Học háo hức)

- **Eager Learning (Học háo hức):**
 - Đây là tất cả các phương pháp chúng ta đã học từ đầu chương đến giờ (Cây quyết định, Naive Bayes).
 - **Cách hoạt động:** Khi nhận được tập dữ liệu huấn luyện, chúng "háo hức" xây dựng ngay một mô hình tổng quát (ví dụ: một cái cây, một bộ các xác suất). Quá trình này gọi là **training**.
 - Khi có dữ liệu mới cần dự đoán, chúng chỉ cần áp dụng mô hình đã có sẵn.
 - **Thời gian:** Tốn nhiều thời gian training, nhưng dự đoán rất nhanh.
- **Lazy Learning (Học lười):**
 - Ví dụ: k-Nearest Neighbor (k-Hàng xóm gần nhất).
 - **Cách hoạt động:** Khi nhận được tập dữ liệu huấn luyện, chúng chẳng làm gì cả, chỉ đơn giản là... **lưu trữ toàn bộ dữ liệu đó**. "Quá trình training" gần như bằng không.
 - **Công việc thực sự chỉ bắt đầu khi có một dữ liệu mới cần dự đoán.** Lúc này, nó mới bắt đầu lôi tập dữ liệu đã lưu ra để so sánh, tính toán.
 - **Thời gian:** Training gần như tức thì, nhưng **tốn nhiều thời gian dự đoán** (vì phải so sánh với toàn bộ tập dữ liệu).
- **Accuracy (Độ chính xác):**
 - Mô hình "háo hức" phải cam kết vào một giả thuyết duy nhất cho toàn bộ không gian dữ liệu (must commit to a single hypothesis that covers the entire instance space).
 - Mô hình "lười" linh hoạt hơn. Nó **đưa ra quyết định dựa trên các "hàng xóm"** cục bộ, cho phép nó tạo ra các ranh giới quyết định phức tạp hơn.

3.7.1. Lazy Learner: Instance-Based Methods (Các phương pháp dựa trên mẫu)

- **Instance-based learning (Học dựa trên mẫu):** Tên gọi khác của học lười, nhấn mạnh việc nó lưu trữ các mẫu (instances) huấn luyện và trì hoãn việc xử lý.
- **Các cách tiếp cận tiêu biểu:**
 1. **k-nearest neighbor approach (k-Hàng xóm gần nhất - kNN):**
 - Đây là phương pháp phổ biến nhất.
 - Dữ liệu được biểu diễn như các điểm trong không gian nhiều chiều.
 - **Để phân loại một điểm mới, ta tìm k điểm gần nó nhất** trong tập huấn luyện và để chúng "bỏ phiếu".
 2. **Locally weighted regression (Hồi quy với trọng số cục bộ):** Xây dựng một mô hình hồi quy nhỏ chỉ dựa trên các điểm lân cận.
 3. **Case-based reasoning (CBR) (Suy luận dựa trên tình huống/trường hợp):**
 - Sử dụng trong các hệ thống phức tạp hơn, nơi mỗi "mẫu" là một "tình huống" (case) được mô tả chi tiết (không chỉ là các con số).
 - Ví dụ: hệ thống chẩn đoán lỗi, hệ thống tư vấn pháp lý.

3.7.2. The k-Nearest Neighbor Algorithm (Thuật toán k-NN)

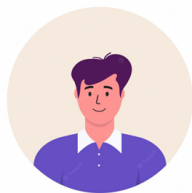
- Ý tưởng cốt lõi: **"Bạn giống ai nhất? → Bạn thuộc nhóm đó!"**
- Mỗi mẫu dữ liệu được coi là một điểm trong không gian n-chiều (n là số đặc trưng).
- **Khoảng cách (Distance):** "Sự gần gũi" giữa các điểm được đo bằng một hàm khoảng cách, phổ biến nhất là **Euclidean distance (khoảng cách Euclid)**.

- Khoảng cách Euclide giữa $X = (x_1, x_2, \dots, x_n)$ và $Y = (y_1, y_2, \dots, y_n)$:

$$D(X, Y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (16)$$

Tức là độ dài đường thẳng nối hai điểm (như khi bạn dùng thước kẻ đo).
 Khi thực hiện so sánh, có thể bỏ qua căn bậc 2.

VD10: Tính khoảng cách giữa Ricky và Steven



Steven:

Age: 35

Income: 95K

No. of credit card: 3



Ricky:

Age: 41

Income: 215K

No. of credit card: 2

$$D(\text{Steven}, \text{Ricky}) = \sqrt{(35 - 41)^2 + (95 - 215)^2 + (3 - 2)^2}$$

- Ta cũng có thể dùng **khoảng cách Manhattan**:

$$d_M(A, B) = |x_1 - y_1| + |x_2 - y_2| + \dots + |x_n - y_n|$$

Tức là **tổng khoảng cách theo từng trục** (như khi bạn đi dọc theo các con đường vuông góc trong thành phố Manhattan).

- Các thuộc tính có giá trị lớn sẽ ảnh hưởng nhiều đến khoảng cách, hoặc chúng có miền giá trị khác nhau → Cần chuẩn hóa giá trị thuộc tính.

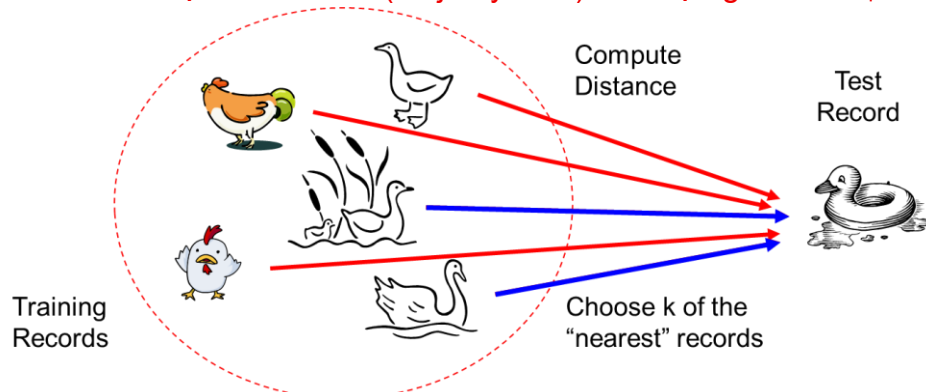
- Chuẩn hóa dữ liệu:** Ánh xạ các giá trị vào đoạn $[0, 1]$ theo công thức:

$$a_i = \frac{v_i - \min v_i}{\max v_i - \min v_i} \quad (17)$$

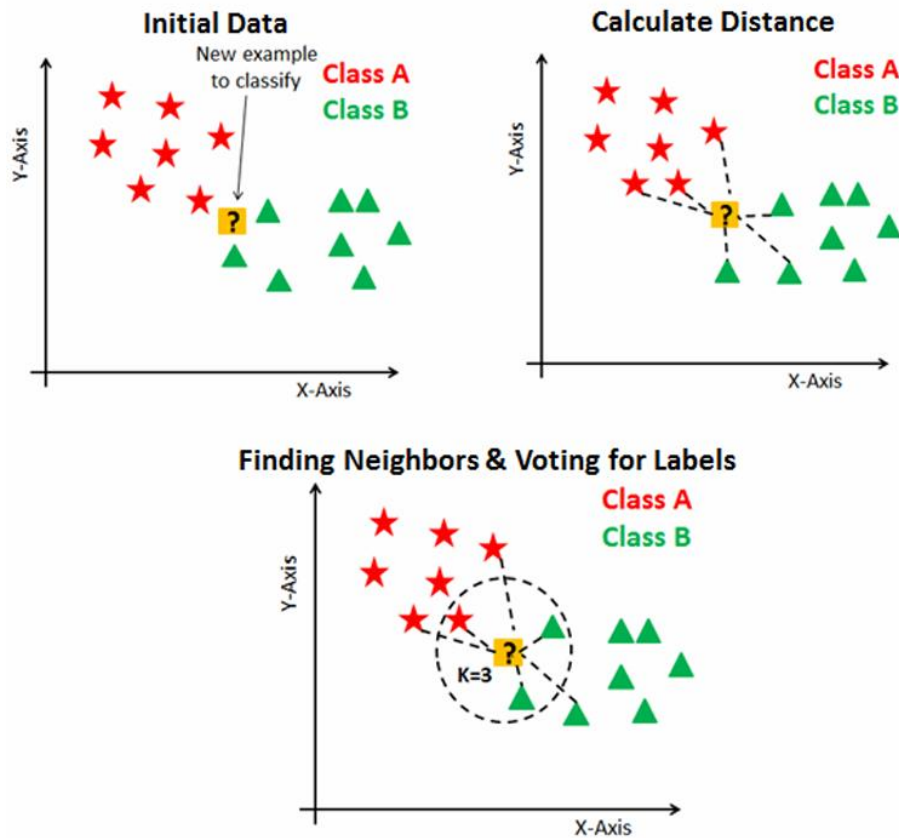
- v_i : giá trị thực tế của thuộc tính i
 - a_i : giá trị đã chuẩn hóa của thuộc tính i

- Cách hoạt động (cho bài toán phân loại):**

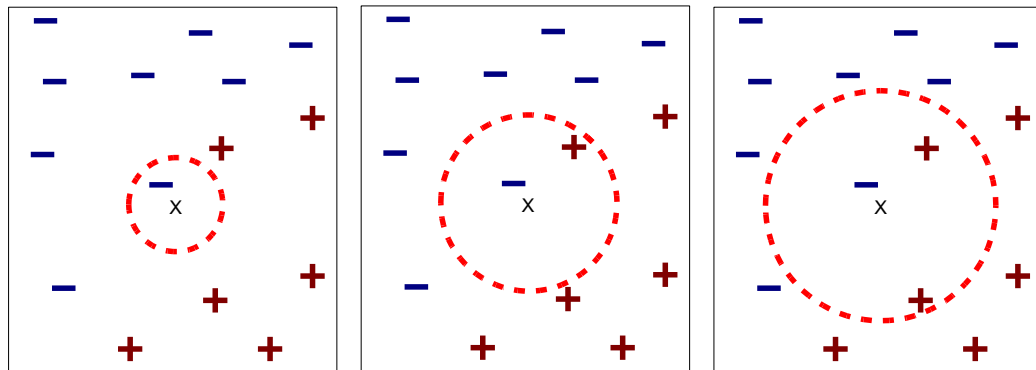
- Cho một điểm dữ liệu mới x_q cần phân loại.
 - Tính khoảng cách từ x_q đến tất cả các điểm trong tập huấn luyện.
 - Xác định k điểm gần x_q nhất.
 - Xem nhãn của k "hàng xóm" này.
 - Nhãn nào xuất hiện nhiều nhất (majority vote) sẽ được gán cho x_q .



- Basic idea: "If it walks like a duck, quacks like a duck, then it's probably a duck"

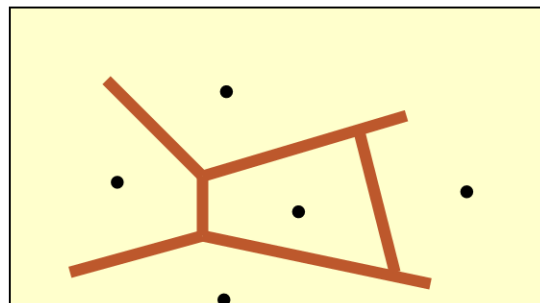


- **K-nearest neighbors** of a record x are data points that have the k smallest distance to x



(a) 1-nearest neighbor (b) 2-nearest neighbor (c) 3-nearest neighbor

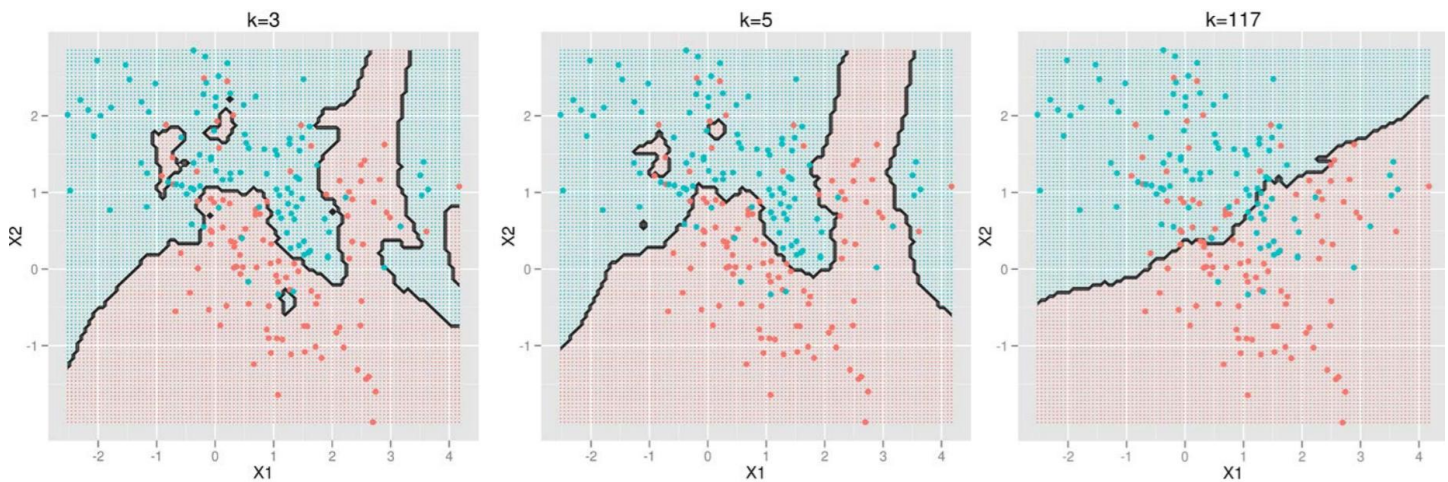
- **Voronoi diagram (Biểu đồ Voronoi):**



- Minh họa ranh giới quyết định được tạo ra bởi thuật toán **1-NN** (chỉ xét 1 hàng xóm gần nhất).

- Mỗi vùng trong biểu đồ chứa tất cả các điểm gần với một điểm huấn luyện cụ thể hơn bất kỳ điểm nào khác. Ranh giới giữa các vùng chính là ranh giới phân loại.
- **k-NN for real-valued prediction (k-NN cho dự đoán số):**
 - k-NN cũng có thể dùng cho bài toán hồi quy. Thay vì bỏ phiếu, ta sẽ lấy **giá trị trung bình (mean)** của k hàng xóm gần nhất làm kết quả dự đoán.
- **Distance-weighted nearest neighbor (k-NN có trọng số khoảng cách):**
 - Một cải tiến so với bỏ phiếu thông thường.
 - Ý tưởng: Hàng xóm càng gần thì ý kiến của nó càng có "trọng lượng" hơn.
 - Cách làm:
 - Gán một trọng số w cho mỗi hàng xóm, được tính như sau:

$$w = \frac{1}{d(x_q, x_i)^2}$$
 - Khi bỏ phiếu, phiếu của mỗi hàng xóm sẽ được nhân với trọng số của nó.
- **Robust to noisy data (Chống nhiễu tốt):** Bằng cách lấy trung bình hoặc bỏ phiếu từ k hàng xóm (với $k > 1$), k-NN có thể **giảm bớt ảnh hưởng của các điểm dữ liệu nhiễu (outlier)**.
- **Curse of dimensionality (Lời nguyền của không gian nhiều chiều):**
 - Đây là một vấn đề lớn của các thuật toán dựa trên khoảng cách như k-NN.
 - Khi **số chiều (số đặc trưng) tăng lên rất cao**, khái niệm "gần" và "xa" trở nên vô nghĩa. Mọi điểm dường như **cách xa nhau như nhau**.
 - Khoảng cách có thể bị "thống trị" bởi các thuộc tính không liên quan, làm cho việc tìm kiếm hàng xóm thực sự trở nên khó khăn.
 - **Cách khắc phục:** Lựa chọn đặc trưng (feature selection), hoặc "kéo dãn" các trục quan trọng và "co lại" các trục không quan trọng (axes stretch or elimination of the least relevant attributes).
- **Choosing the value of k:**
 - If k is **too small**, sensitive to noise points
 - Nếu một điểm dữ liệu nhiễu tình cờ là hàng xóm gần nhất, dự đoán sẽ bị sai
 - Điều này dẫn đến **overfitting (quá khớp)**.
 - Trong thuật ngữ thống kê, đây là mô hình có **high variance (phương sai cao)** và **low bias (độ chệch thấp)**.
 - If k is **too large**, neighborhood may **include points from other classes**
 - Mô hình có thể trở nên quá đơn giản và không nắm bắt được cấu trúc phức tạp của dữ liệu. Điều này dẫn đến **underfitting (chưa khớp)**.
 - **The value of k is the complexity of the model.**
 - **k luôn chọn là một số lẻ. Không chọn số chẵn.**



- **Cách chọn k tốt nhất?** Không có câu trả lời duy nhất.

Phương pháp	Mô tả
Thử nhiều k + Cross-validation (khuyến nghị nhất)	Chia dữ liệu, thử $k=1,3,5,\dots$ → chọn k cho độ chính xác cao nhất
Quy tắc ngón tay cái: $k \approx \sqrt{n}$	n = số mẫu → ví dụ: 100 mẫu → thử $k \approx 10$
Dùng Elbow Method	Vẽ đồ thị lỗi theo k → chọn điểm "khuỷu tay"

- **Chọn k vừa đủ:** đủ để loại nhiễu, nhưng không quá để lẫn lớp

3.7.3. Case-Based Reasoning (CBR) (Suy luận dựa trên tình huống)

- **CBR** là một dạng "k-NN nâng cao".
- Uses a database of problem solutions to solve new problems → Từng gặp vấn đề giống vậy rồi → Làm lại như cũ (hoặc điều chỉnh chút).
- **Sự khác biệt chính với k-NN:**
 - k-NN làm việc với các điểm trong không gian Euclid (**các vector số**).
 - **CBR làm việc với các "cases" (tình huống)** được mô tả một cách **symbolic (tượng trưng, có cấu trúc)**, ví dụ như đồ thị, chuỗi văn bản, các quy tắc logic.

Tiêu chí	k-NN	CBR
Dữ liệu	Điểm số (numerical) trong không gian	Trường hợp (case): mô tả bằng ký hiệu, ngôn ngữ tự nhiên, đồ thị...
Lưu trữ	Vector số	Tuple, câu chuyện, quy trình, phán quyết pháp lý...
Tính toán	Khoảng cách Euclidean	Độ tương đồng ngữ nghĩa (semantic similarity)
Ứng dụng	Phân loại ảnh, dự đoán giá	Chẩn đoán, tư vấn, pháp lý, hỗ trợ khách hàng

- **Ứng dụng:**
 - **Customer-service:** Chẩn đoán lỗi sản phẩm. Khi có lỗi mới, hệ thống tìm kiếm trong cơ sở dữ liệu các "tình huống" lỗi tương tự đã được giải quyết trong quá khứ và đề xuất giải pháp.
 - **Legal ruling (Tư vấn pháp lý):** Tìm kiếm các vụ án tương tự trong quá khứ để tham khảo.

- **Thách thức (Challenges):**

- **Find a good similarity metric (Tìm thước đo độ tương đồng tốt):** Làm sao để định nghĩa "sự tương đồng" giữa hai bản mô tả phức tạp (ví dụ: hai bản tóm tắt vụ án)? Đây là bài toán khó hơn nhiều so với việc tính khoảng cách Euclid.
- **Indexing (Đánh chỉ mục):** Làm sao để tìm kiếm nhanh chóng trong một cơ sở dữ liệu khổng lồ các "tình huống"?

3.8. Linear Classifiers

3.8.1. Giới thiệu bài toán Hồi quy (Regression)

- **Mục tiêu của Hồi quy:** **Tìm một hàm** để ánh xạ từ **các** thuộc tính đầu vào x đến **một** giá trị **liên tục y**.

- Hồi quy = dùng dữ liệu cũ để tìm công thức dự đoán **con số** trong tương lai

- **Ví dụ:**

- {diện tích nhà} => Giá nhà
- {trường đại học; ngành học; GPA} => Thu nhập tương lai
- {Nhiệt độ, độ ẩm} => Lượng mưa (mm)
- {Cân nặng, chiều cao} => Chỉ số BMI

- **Hồi quy hoạt động thế nào?**

- **Bước 1: Có dữ liệu huấn luyện**

Diện tích (m ²)	Giá nhà (triệu)
50	300
70	420
90	550
120	720

- **Bước 2: Tìm đường thẳng (hoặc đường cong) phù hợp nhất**

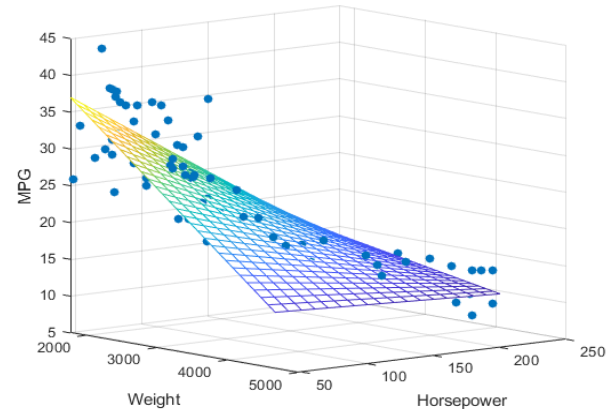
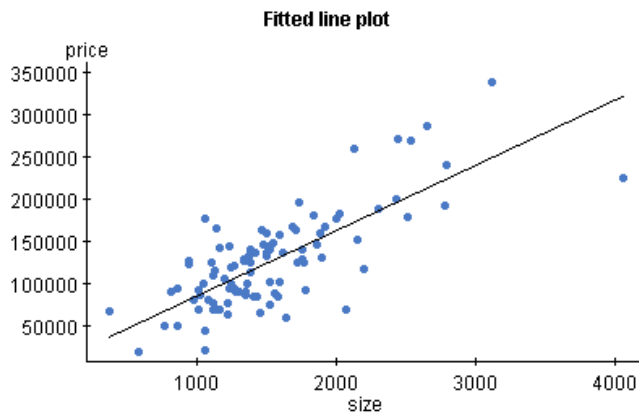
- Giá nhà = $a \times \text{Diện tích} + b$
- → Máy tính tự tìm **a** và **b** sao cho **đường thẳng gần các điểm nhất**

- **Các loại hồi quy phổ biến**

Loại	Dùng khi
Hồi quy tuyến tính	Mối quan hệ gần thẳng
Hồi quy đa thức	Mối quan hệ cong (ví dụ: tăng nhanh rồi chậm lại)
Hồi quy Logistic	Dự đoán xác suất (0 đến 1) → dù tên có "regression" nhưng là phân loại
Ridge, Lasso	Dữ liệu nhiều đặc trưng, tránh overfitting

3.8.2. Hồi quy tuyến tính (Linear Regression)

- Giả định rằng mối quan hệ giữa x và y là một đường thẳng (trong không gian 2D), một mặt phẳng (trong không gian 3D), hoặc một siêu phẳng (trong không gian nhiều chiều hơn).



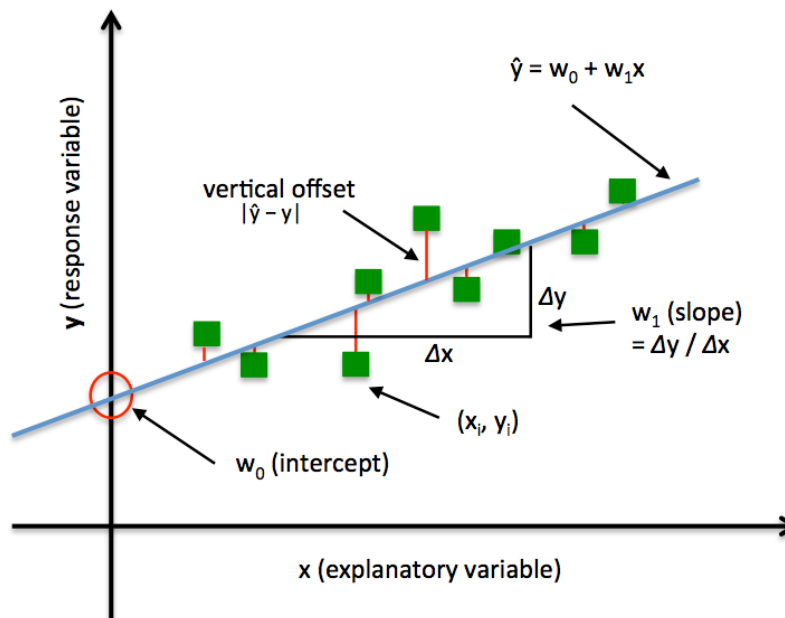
- Bên trái: Hồi quy tuyến tính đơn biến.** Tìm một đường thẳng phù hợp nhất với đám mây điểm dữ liệu (Living Area, Price of houses).
 - Bên phải: Hồi quy tuyến tính đa biến.** Tìm một mặt phẳng phù hợp nhất với đám mây điểm dữ liệu trong không gian 3 chiều.
- Dữ liệu:**
 - y_i : **giá trị** thực tế cần dự đoán (ví dụ: giá nhà thực tế).
 - $y_i, i = 1, 2, 3, \dots, n$
 - x_i : vector các thuộc tính của mẫu thứ i (ví dụ: (diện tích, số phòng ngủ,...)).
 - p -dimensional attributes: $x_i = (x_{i1}, x_{i2}, \dots, x_{ip}), i = 1, 2, 3, \dots, n$
- Model (Mô hình):** $y_i = w \cdot x_i + b$
 - Hoặc viết rõ hơn: $y_i = w_1 x_{i1} + w_2 x_{i2} + \dots + w_p x_{ip} + b$
 - w : là một **weight vector (vector trọng số) (độ dốc)**

$$w = (w_1, w_2, \dots, w_p)$$

Mỗi thành phần w_j trong w cho biết mức độ quan trọng của thuộc tính x_j .
 - b : là **bias (hệ số chặn)**, hay còn gọi là intercept, là điểm cắt trục tung.
 - Mục tiêu của việc học:**
 - The weight vector w and bias b are the model parameter learnt by data
 - Từ dữ liệu huấn luyện, tìm ra các giá trị w và b "tốt nhất" để **về phải xấp xỉ về trái tốt nhất có thể.**
 - Ví dụ dễ hiểu:**

Diện tích (m ²)	Số phòng	Giá nhà (triệu)
50	2	300 $\rightarrow y_1$
70	3	420 $\rightarrow y_2$

Mô hình: Giá = $w_1 \times$ Diện tích + $w_2 \times$ Số phòng + b



- **Cách tìm ra w và b tốt nhất:**

- **Least Square Method (Phương pháp bình phương tối thiểu):**

- "Tốt nhất" nghĩa là gì? Nghĩa là tổng **sai số giữa giá trị dự đoán và giá trị thực tế là nhỏ nhất.**

- **Cost / Loss Function (Hàm mất mát):** $L(w, b) = \sum_{i=1}^n (y_i - wx_i - b)^2$

- Đây là tổng bình phương của các sai số.
 - Ta dùng bình phương để đảm bảo sai số luôn dương và "phạt" nặng các sai số lớn.
 - n là số vector (số mẫu dữ liệu)

- **Optimization Goal (Mục tiêu tối ưu):** Tìm w và b sao cho **hàm $L(w, b)$ đạt giá trị nhỏ nhất** (argmin).

- **Closed-form solution (Ng nghiệm tường minh):**

- Đối với hồi quy tuyến tính đơn giản, có một công thức toán học trực tiếp để tính ra w và b tối ưu mà không cần các phương pháp lặp.

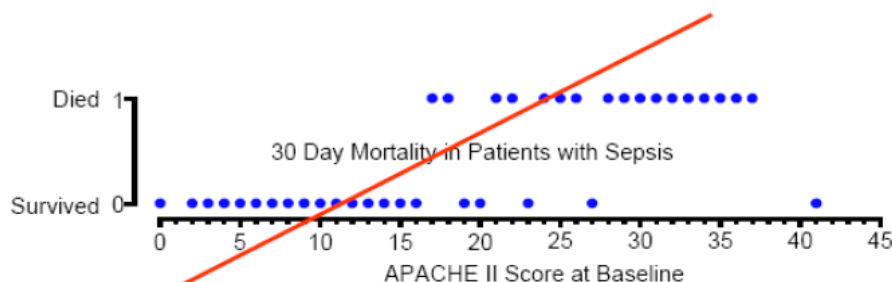
$$w = \frac{\sum_{i=1}^n x_i (y_i - \bar{y})}{\sum_{i=1}^n x_i^2 - n(\sum_{i=1}^n x_i)^2}$$

$$b = \frac{1}{n} \sum_{i=1}^n (y_i - wx_i)$$

- Hoặc gọn hơn:

$$w = \frac{\sum_{i=1}^n x_i (y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \quad b = \bar{y} - w\bar{x}$$

- **A linear function is not good** → It may produce negative probabilities, or probabilities that are greater than 1.

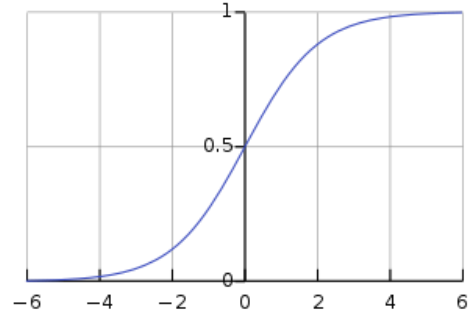


3.8.3. Logistic Regression (Hồi quy Logistic)

- **Câu hỏi:** Làm sao dùng **một mô hình hồi quy** (đầu ra là một số thực từ $-\infty$ đến $+\infty$) để **giải bài toán phân loại** (đầu ra là một lớp)?
- **Ý tưởng chính:** Chúng ta cần một hàm để **"ép" giá trị đầu ra** của mô hình tuyến tính vào khoảng $[0, 1]$. Giá trị này sau đó có thể được diễn giải là **xác suất** để mẫu đó thuộc về lớp 1.
- **Sử dụng Hàm Sigmoid (differentiable function):**

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{e^z + 1}$$

- Đây là một hàm có dạng chữ "S", có đặc điểm là:
 - Nhận **đầu vào** z là một số thực bất kỳ.
 - Cho **đầu ra** là một số trong khoảng $(0, 1)$.
 - Khi z rất lớn, $\sigma(z)$ tiến tới 1.
 - Khi z rất nhỏ (âm), $\sigma(z)$ tiến tới 0.
 - Khi $z = 0$, $\sigma(z) = 0.5$.



- **Cách kết hợp:**

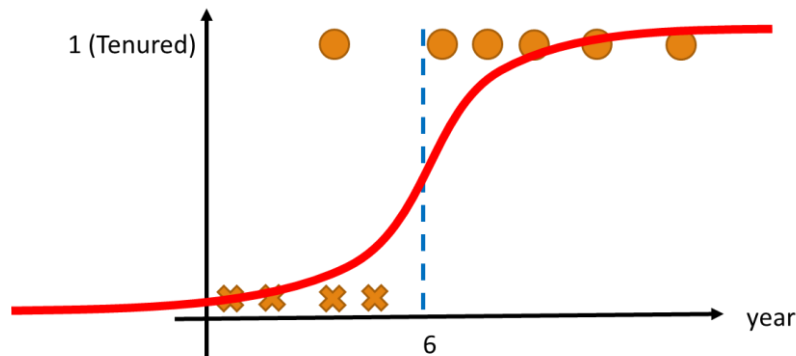
- Tính giá trị tuyến tính: $z = \mathbf{w}x + \mathbf{b}$.
- Đưa z qua hàm Sigmoid: $\mathbf{p} = \sigma(z)$.
- p chính là xác suất dự đoán $P(Y=1 | X)$.

- **Quy tắc phân loại:** Dự đoán = $\begin{cases} \text{Lớp 1} & \text{nếu } \sigma(z) > 0.5 \\ \text{Lớp 0} & \text{nếu } \sigma(z) \leq 0.5 \end{cases}$

- Notice that $\ln \frac{y}{1-y} = \mathbf{w}^T \mathbf{x} + b$

- **An Example**

- **Bài toán:** Dự đoán một giáo sư có được "biên chế" (Tenured) hay không, dựa trên số năm kinh nghiệm (year). Đây là bài toán phân loại nhị phân (0: không biên chế, 1: biên chế).



- **Dữ liệu:**
 - Các điểm dữ liệu được đánh dấu 'x' (ở dưới) là những người không được biên chế.
 - Các điểm dữ liệu hình tròn (ở trên) là những người đã được biên chế.
- **Mô hình:**
 - Chúng ta không thể dùng một đường thẳng (hồi quy tuyến tính) để phù hợp với dữ liệu này.
 - Tuy nhiên, đường cong Sigmoid (màu đỏ) lại rất phù hợp. Nó thể hiện rằng:

- Khi số năm kinh nghiệm còn ít, xác suất được biên chế (giá trị trên trục y) rất thấp, gần 0.
- Khi số năm kinh nghiệm tăng lên, xác suất cũng tăng dần.
- Có một "ngưỡng" (ở đây là khoảng 6 năm), mà tại đó xác suất tăng vọt.
- Khi số năm kinh nghiệm rất cao, xác suất được biên chế tiến gần tới 1.

• Mô hình Hồi quy Logistic

- Xác suất để $Y=1$ (thuộc lớp 1) là: $p(Y = 1 | X = x; \mathbf{w}) = \text{Sigmoid}(w_0 + \sum_{i=1}^n w_i \cdot x_i)$
 - $\mathbf{w} = (w_0, w_1, w_2, \dots, w_n)$ are the parameters
 - w_0 = hệ số tự do (bias) = giá trị dự đoán khi tất cả đặc trưng bằng 0. Điểm (0, w_0) là nơi mà tại đó đường cong Sigmoid đổi hướng (lên/xuống) (tại đó xác suất = 0.5).

• Maximum Likelihood Estimation (Ước lượng hợp lý cực đại - MLE):

- Đây là một nguyên tắc thống kê để tìm tham số cho mô hình.
- **Ý tưởng:** Tìm bộ tham số $\mathbf{w}$ sao cho **xác suất xảy ra của toàn bộ tập dữ liệu huấn luyện là lớn nhất có thể**.
 - Tức là: xác suất dự đoán đúng toàn bộ dữ liệu là cao nhất
- **Xây dựng hàm Likelihood (L):**
 - Xét một điểm dữ liệu (x_i, y_i) .
 - Nếu $y_i=1$ (nhãn thực tế là 1), ta muốn xác suất dự đoán $p(\hat{y}_i=1)$ càng cao càng tốt.
 - Nếu $y_i=0$ (nhãn thực tế là 0), ta muốn xác suất dự đoán $p(\hat{y}_i=0) = 1 - p(\hat{y}_i=1)$ càng cao càng tốt.
 - Viết gọn: $p(\hat{y}_i = y_i) = p_i^{y_i} (1 - p_i)^{1-y_i}$
 - Hàm Likelihood L là tích của các xác suất này trên toàn bộ tập dữ liệu. Chúng ta muốn **tối đa hóa (maximize)** L.

$$L = \prod_i p_i^{y_i} (1 - p_i)^{1-y_i} = \prod_i \left(\frac{\exp(w^T x_i)}{1 + \exp(w^T x_i)} \right)^{y_i} \left(\frac{1}{1 + \exp(w^T x_i)} \right)^{1-y_i}$$

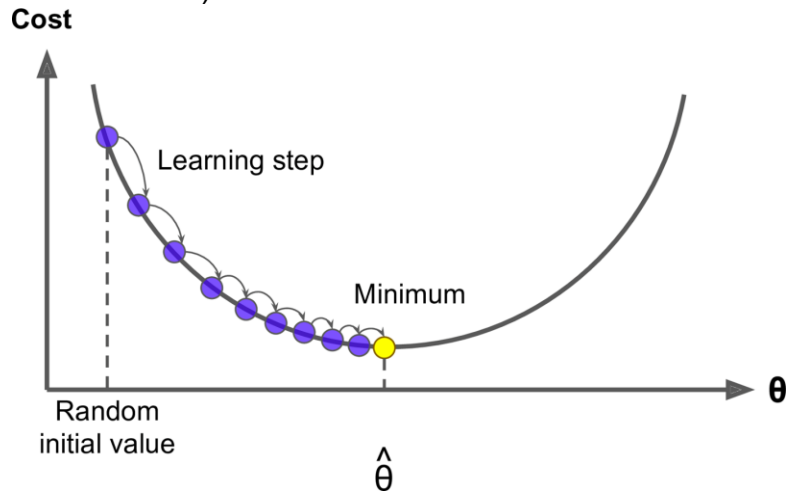
- **Việc tối đa hóa tích L rất khó về mặt toán học.**
 - **Giải pháp:** Lấy logarit của L. Vì logarit là hàm đồng biến, việc tối đa hóa L tương đương với việc **tối đa hóa log(L)**. log(L) được gọi là **Log-likelihood l(w)**

$$\begin{aligned} l(\mathbf{w}) &= \sum_{i=1}^N y_i \log p(Y = 1 | X = x_i; \mathbf{w}) + (1 - y_i) \log(1 - p(Y = 1 | X = x_i; \mathbf{w})) \\ &= \sum_{i=1}^N [y_i x_i^T \mathbf{w} - \log(1 + \exp(\mathbf{w}^T x_i))] \end{aligned}$$

- **Vấn đề:** Không giống như Hồi quy tuyến tính, **không có "ng nghiệm tường minh"** (closed-form solution) cho Hồi quy Logistic. **Tức là không có công thức trực tiếp để tính ra $\mathbf{w}$.**
 - **Giải pháp:** Phải sử dụng các phương pháp tối ưu hóa lặp đi lặp lại. Phương pháp phổ biến nhất là **Gradient Descent/Ascent** (Giảm/Tăng theo Gradient).

3.8.4. Gradient Descent (Giảm theo Gradient)

- Gradient Descent is an iterative optimization algorithm (Thuật toán tối ưu hóa lặp lại) for finding **the minimum of a function** (e.g., the negative log likelihood)
- Mục tiêu:** **Tìm điểm cực tiểu (minimum) của một hàm** (ví dụ: hàm Cost/Loss).
- Trực quan hóa:** Hãy tưởng tượng bạn đang đứng trên một ngọn đồi (hàm Cost) và muốn đi xuống thung lũng (điểm minimum) nhanh nhất có thể.



- Tại mỗi điểm, bạn sẽ nhìn xung quanh và **tìm hướng dốc nhất để đi xuống**.
- Hướng dốc nhất đó chính là **ngược chiều của gradient** (negative gradient). **Gradient là một vector chỉ hướng dốc lên**.
- Thuật toán:**
 - 1/ Bắt đầu từ một điểm a ngẫu nhiên (random initial value).
 - 2/ Tính gradient của hàm F tại điểm a.
 - 3/ Cập nhật vị trí mới: **$a_{\text{new}} = a_{\text{old}} - \gamma \times \nabla F(a_{\text{old}})$** .
 - $\nabla F(a_{\text{old}})$: là gradient (hướng dốc lên).
 - γ (gamma): là **learning rate (tốc độ học)** hay **step size (kích thước bước)**. Nó kiểm soát việc bạn đi một bước dài hay ngắn theo hướng dốc xuống.
 - Lặp lại bước 2 và 3 cho đến khi vị trí không còn thay đổi nhiều nữa (**tức là khi gradient gần bằng 0**, bạn đã đến đáy thung lũng).
- Gradient Ascent:** Tương tự nhưng để tìm điểm cực đại (maximum). Thay vì đi ngược chiều gradient, ta đi cùng chiều: **$a_{\text{new}} = a_{\text{old}} + \gamma \times \nabla F(a_{\text{old}})$** .
- Ta không đi sâu vào toán học**, vì khá phức tạp. **Thư viện của Python** sẽ hỗ trợ những phần này.

```
from sklearn.linear_model import LogisticRegression
```

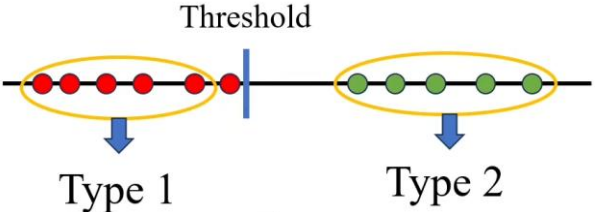
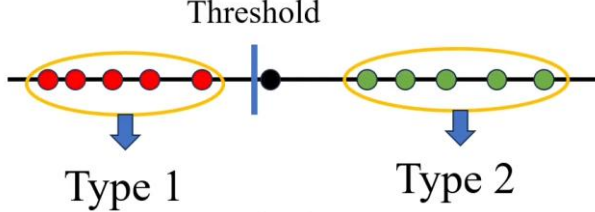
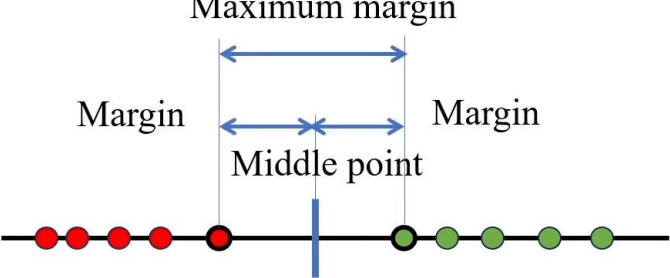
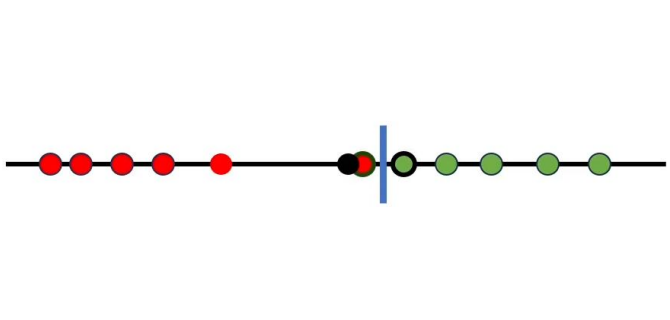
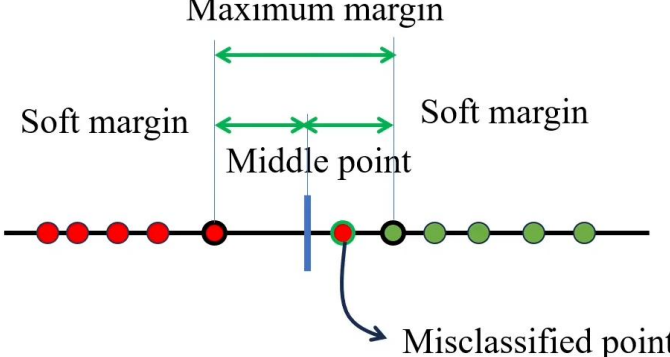
```
model = LogisticRegression()    # Tự động làm hết: MLE + Gradient!  
model.fit(X, y)                 # Học w tự động  
print(model.predict([[5, 8]]))  # Dự đoán
```

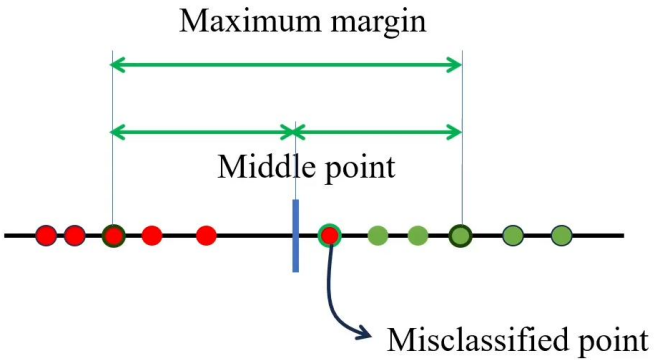
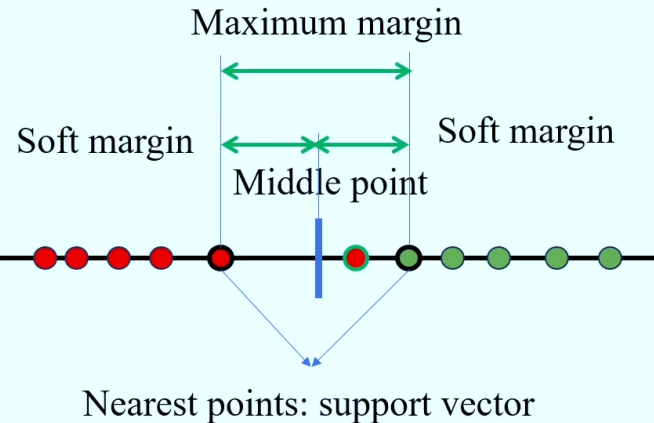
3.9. Support Vector Machine (SVM) (Máy Vector Hỗ trợ)

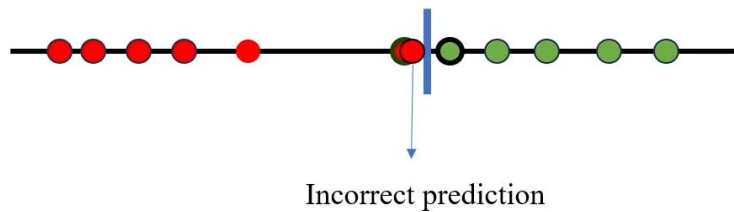
- Là một thuật toán học **có giám sát** (supervised learning), được dùng cho **phân loại** (classification) và đôi khi cả **hồi quy** (regression).

3.9.1. Mở đầu

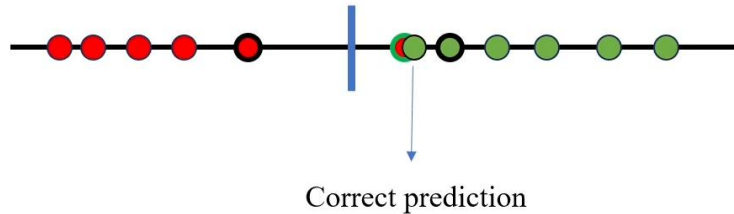
- Xét trường hợp cho dữ liệu tuyến tính – **Support Vector Classification**.

	✓ Dựa vào ngưỡng, khi một điểm dữ liệu nằm ở bên phía nào của ngưỡng thì nó sẽ thuộc vào lớp tương ứng.
	✓ Điểm dữ liệu này nằm ở phía Type 2, nên nó được phân vào lớp Type 2. ✓ Tuy nhiên điểm này lại nằm gần Type 1 hơn. → Ngưỡng sử dụng này nó không công bằng (unfair) → Có thể phân loại SAI.
	✓ Để khắc phục, cần đặt điểm ngưỡng ở chính giữa 2 điểm dữ liệu biên thuộc 2 lớp. ✓ Đây được gọi là Maximum Margin Classification. ✓ Tuy nhiên phương pháp này vẫn có khuyết điểm.
	✓ Nếu tồn tại một điểm dữ liệu coi là ngoại lai của loại 1 (vì nó sát với điểm biên của loại 2), thì khi có một điểm dữ liệu cần phân loại nằm gần điểm ngoại lai (màu đen), thì vô tình nó được phân thành loại 1. → Có thể phân loại SAI. Và tất cả điểm dữ liệu quanh khu vực điểm ngoại lai này cũng có thể bị phân loại sai.
	✓ Một cách tiếp cận khác là ta cho phép phân loại sai (Allowing misclassification). ✓ Khi đó ta xác định điểm phân loại sai, sau đó chọn margin. ✓ Vấn đề: Chọn soft margin như thế nào?

	✓ Ta có thể liên tục mở rộng margin. ✓ Công việc chọn maximum margin nào phù hợp, tốt nhất → Sử dụng Cross-validation → Thay đổi λ tại những khoảng khác nhau, sau đó đánh giá λ nào cho kết quả tốt nhất giữa tập train và tập test. ✓ Phương pháp sử dụng soft margin để tạo ngưỡng phân lớp 2 nhóm dữ liệu được gọi là Support Vector Classification (SVC). ✓ SVC là trường hợp đơn giản nhất của SVM, dùng cho dữ liệu tuyến tính. ✓ Vậy mọi khái niệm liên quan đến SVC đều đúng với SVM.
	✓ Choosing soft margin allows us to control bias-variance tradeoff. (Việc chọn “soft margin” (biên mềm) cho phép ta điều chỉnh sự đánh đổi giữa độ chệch (bias) và phương sai (variance) trong mô hình học máy). ✓ Bias là mức sai lệch giữa dự đoán của mô hình và giá trị thực tế (truth). ✓ Variance trong học máy là mức dao động của mô hình khi ta thay đổi dữ liệu huấn luyện. ✓ Bias thấp → Mô hình nhận diện đúng hơn với training set. ✓ Variance thấp → Mô hình nhận diện đúng hơn với testing set.
✓ Support Vectors: những điểm dữ liệu nằm gần nhất với siêu phẳng. Chúng nằm ngay trên các đường biên của margin. ✓ Chúng được gọi là “vector hỗ trợ” vì chúng là những điểm duy nhất “chống đỡ” và quyết định vị trí của siêu phẳng. Nếu bạn di chuyển các điểm dữ liệu khác (không phải support vector), siêu phẳng sẽ không thay đổi. Nhưng nếu bạn di chuyển một support vector, siêu phẳng sẽ phải thay đổi theo.	✓ Margin (Biên): Là khoảng cách từ siêu phẳng đến các support vector gần nhất của mỗi lớp.



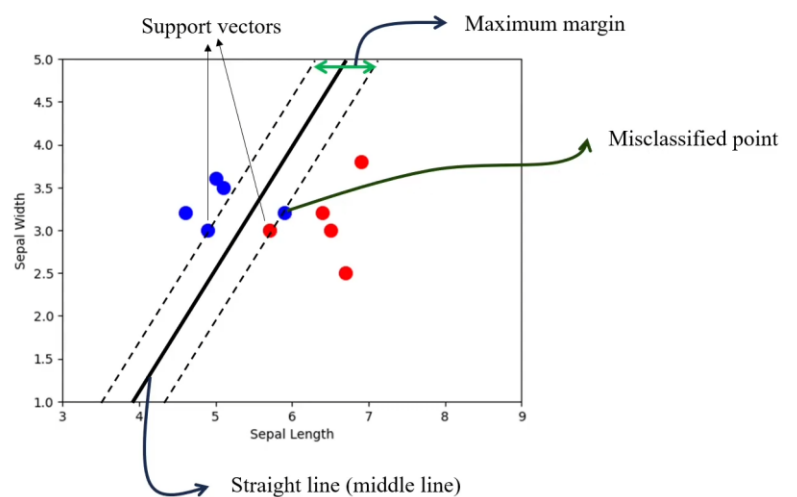
Less bias, higher variance

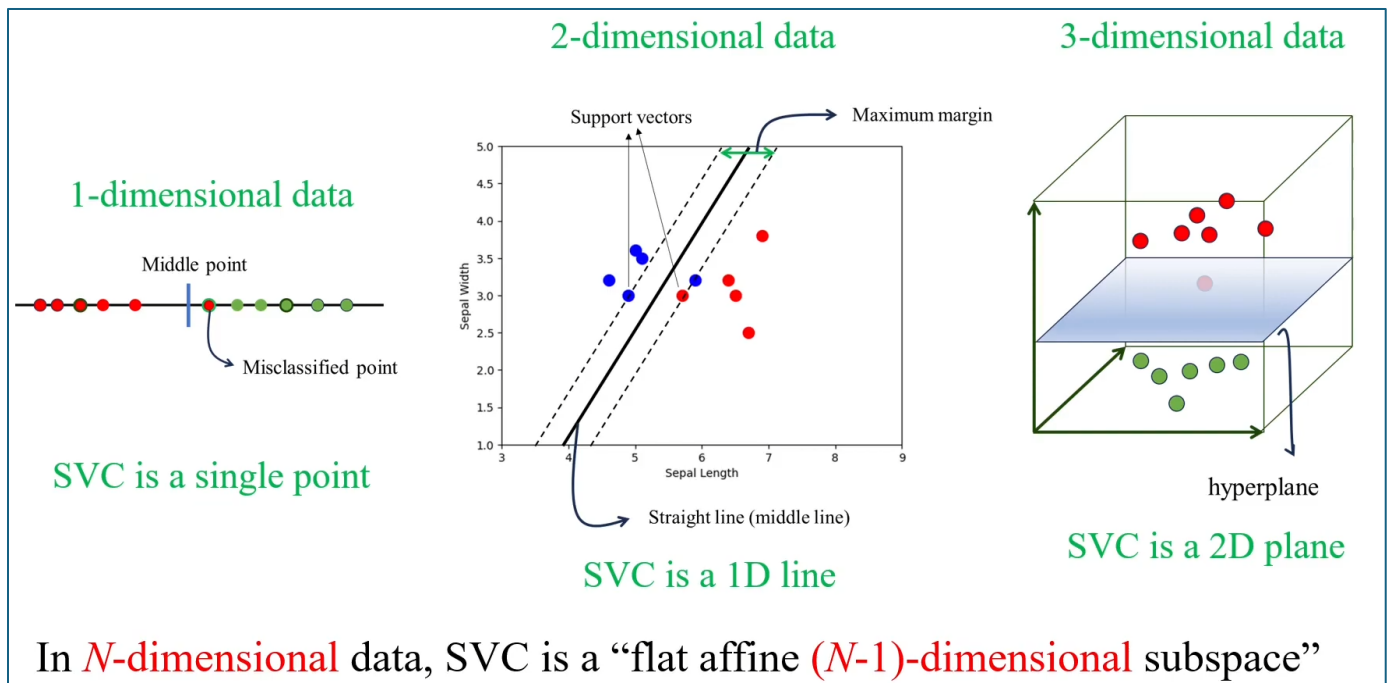


Higher bias, low variance



	Sepal length	Sepal width	Class
0	5.1	3.5	0
1	4.9	3.0	0
2	5.9	3.2	0
3	4.6	3.2	0
4	5.0	3.6	0
5	6.4	3.2	1
6	6.9	3.8	1
7	6.5	3.0	1
8	6.7	2.5	1
9	5.7	3.0	1



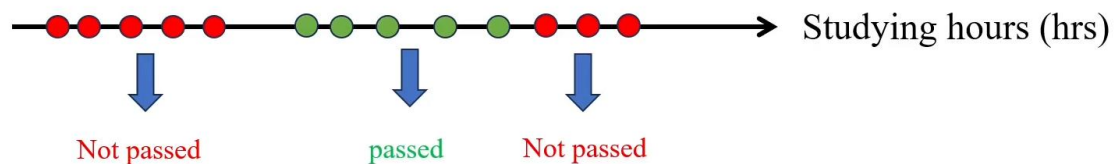
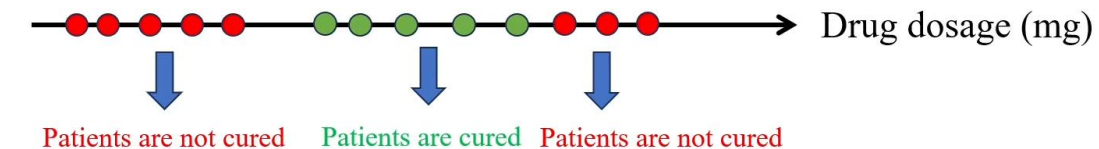


Or: In N -dimensional data, SVC is a $(N-1)$ -dimensional hyperplane

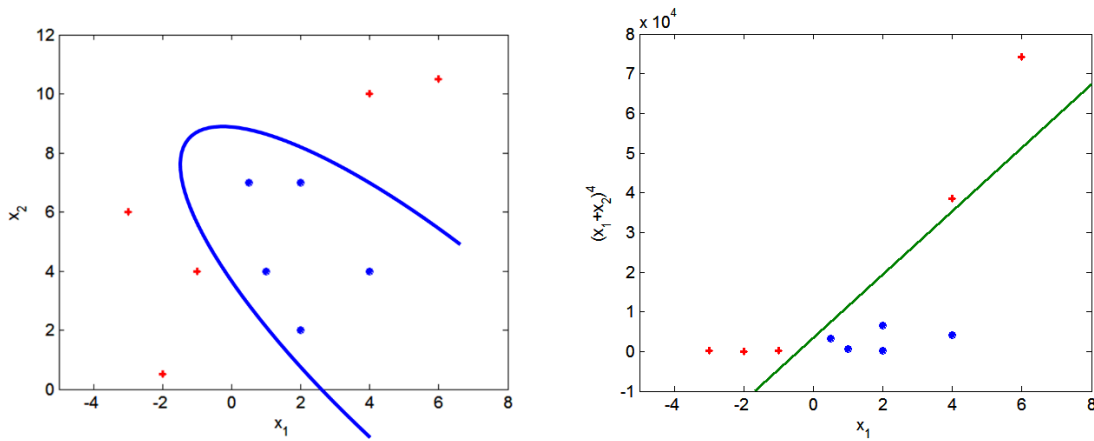
- Advantages of **linear** support vector classifier
 - Effective in **high dimensional** spaces
 - Robust to Overfitting by using soft margin
 - Effective in **binary** classification
 - Easy Optimization

○ **SVC is limited to non-linear data.**

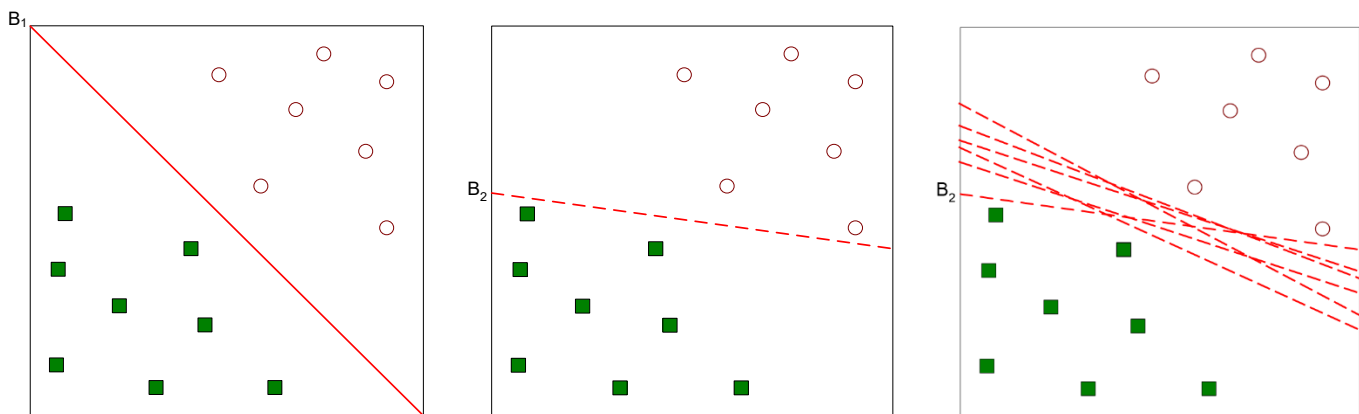
○ Ví dụ:



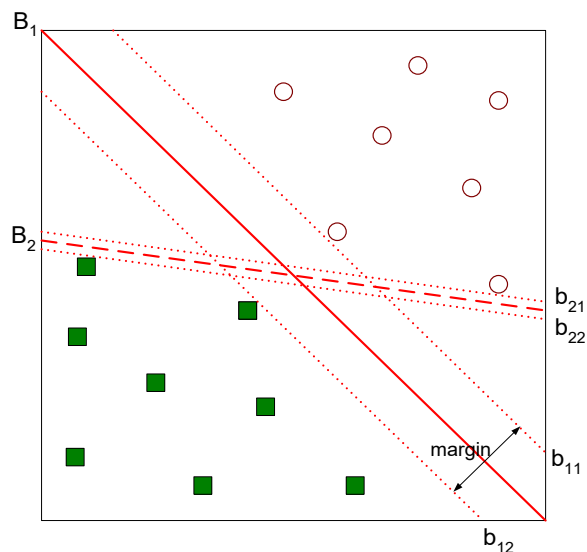
- Đối với dạng dữ liệu như trên, ta không có cách nào để đặt hyperplane cho phù hợp.
- Ta cần biến đổi dữ liệu này sang một dạng khác bằng cách dùng các hàm kernel.



- Ý tưởng chính của SVM là: **Tìm ra một đường biên (hyperplane) tốt nhất để phân tách các lớp dữ liệu trong không gian đặc trưng (feature space).**

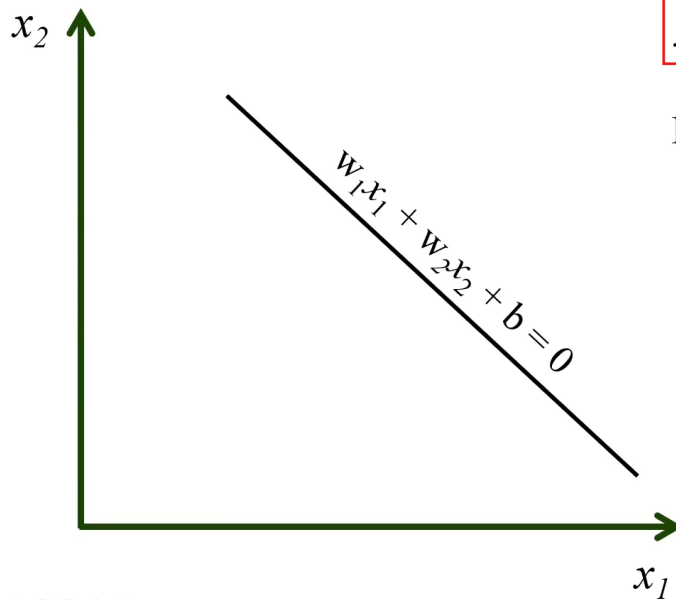


- Siêu phẳng tốt nhất là siêu phẳng tạo ra một "khoảng đệm" hay **biên (margin) lớn nhất** giữa hai lớp dữ liệu.
- Một margin rộng cho thấy mô hình có độ tự tin cao khi phân loại. Nó ít nhạy cảm với nhiễu và có khả năng **tổng quát hóa (generalize)** tốt hơn trên dữ liệu mới chưa từng thấy.



- Find hyperplane **maximizes** the margin => B1 is better than B2

3.9.2. Tính toán margin trên Hyperplane



$$y = ax + b \quad \longrightarrow \quad ax - y + b = 0$$

Linear equation

$$w_1x_1 + w_2x_2 + b = 0$$

$-w_1/w_2$ is slope of line

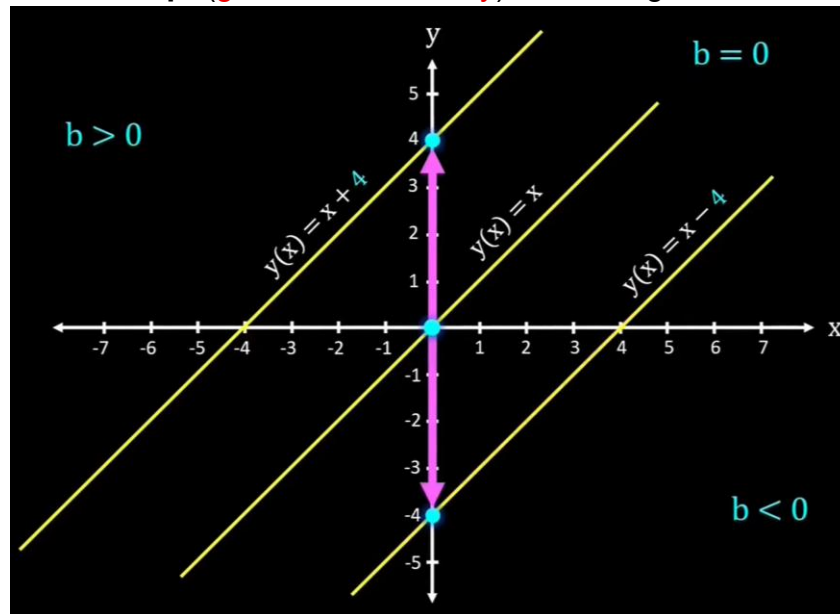
Equation

$$\mathbf{w}^T \mathbf{x} + b = 0$$

$$\mathbf{w} = (w_1, w_2)$$

$$\mathbf{x} = (x_1, x_2)$$

- **Slope** (độ dốc) thể hiện **mức độ nghiêng của đường thẳng** — tức là khi **x** thay đổi 1 đơn vị thì **y** thay đổi bao nhiêu.
 - Nói cách khác: **Slope** = "mức thay đổi của y trên mỗi thay đổi của x".
- Trong phương trình $y = ax + b$
 - **a** chính là **slope** (độ dốc)
 - **b** là **intercept** (giao điểm với trục y), điểm tung độ

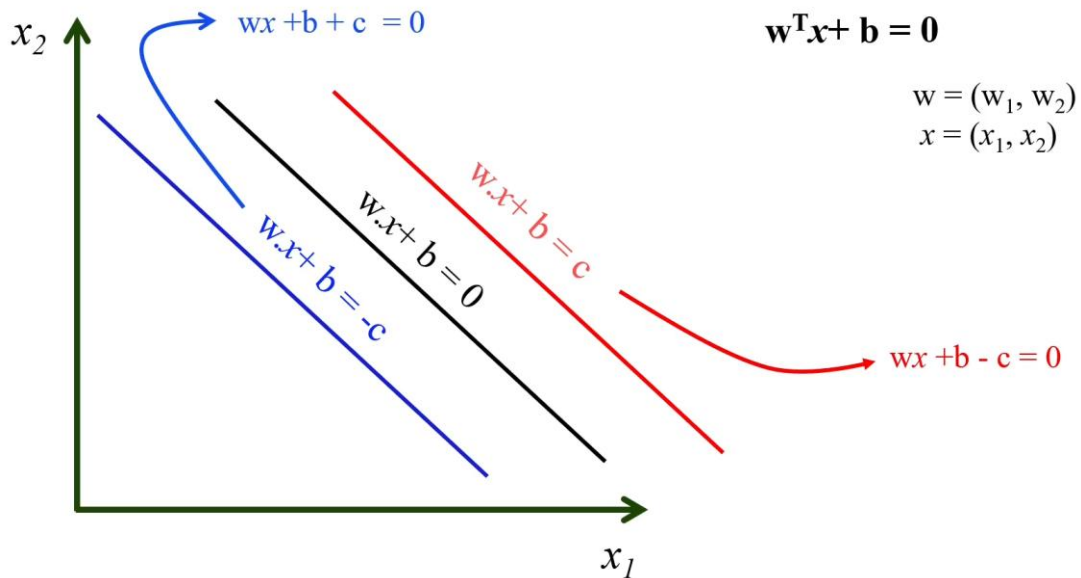


- Ta có phương trình tổng quát: $w_1x_1 + w_2x_2 + b = 0$
- Muốn viết lại theo dạng $y = ax + b$, ta làm như sau:

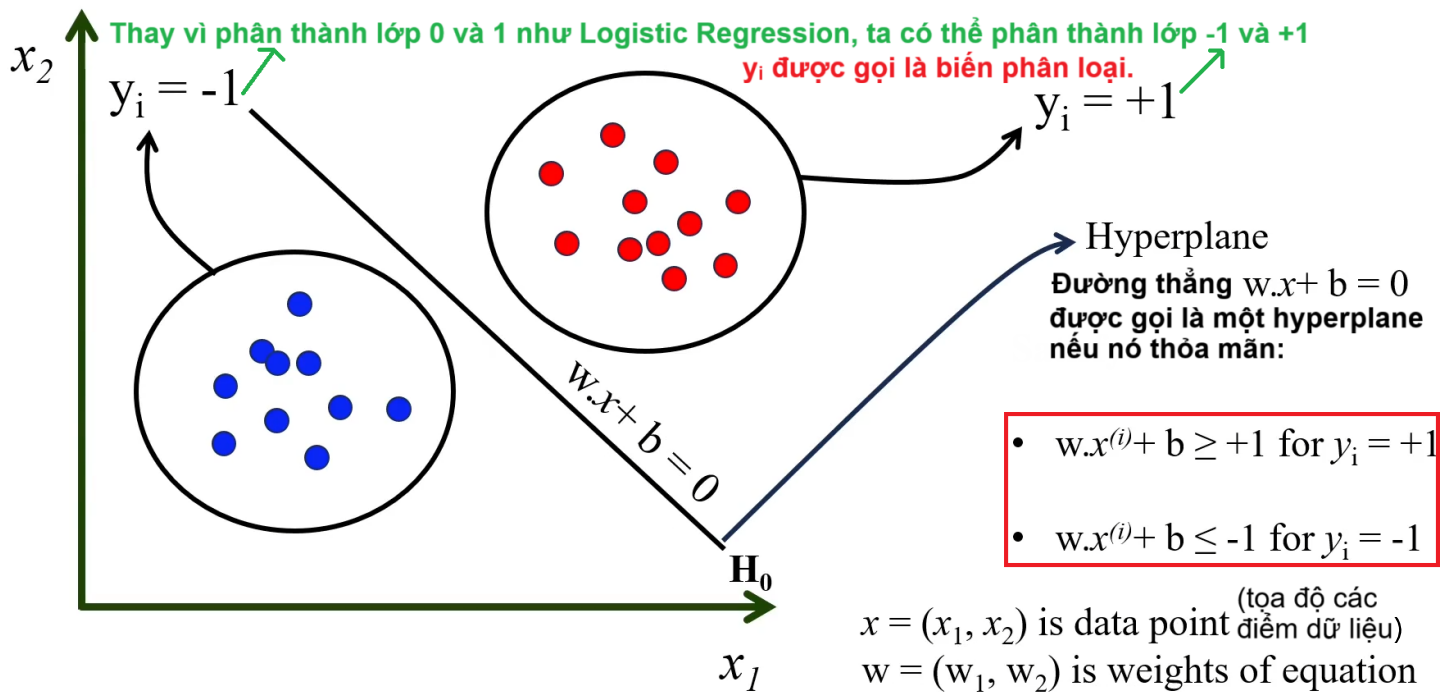
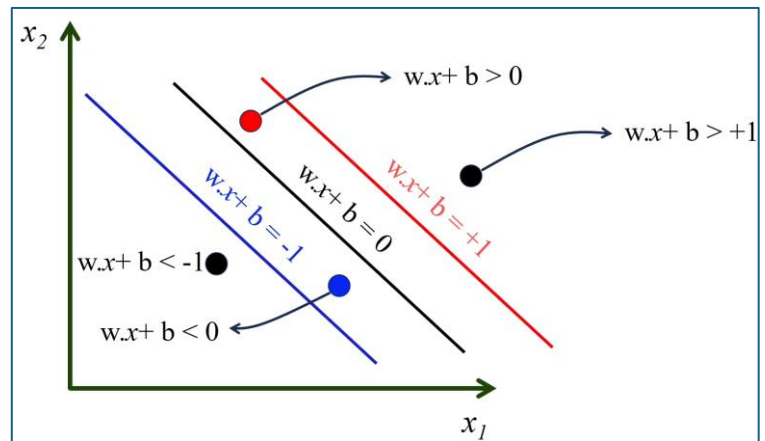
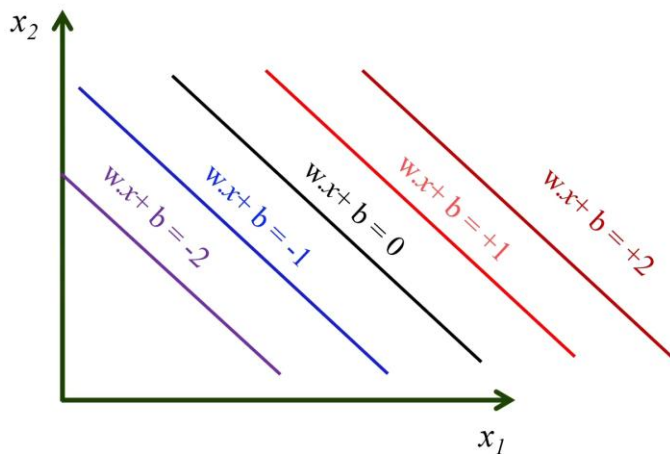
$$w_1x_1 + w_2x_2 + b = 0 \Rightarrow w_2x_2 = -w_1x_1 - b \Rightarrow \mathbf{x_2} = -\frac{w_1}{w_2}x_1 - \frac{b}{w_2}$$

=> Khi đó:

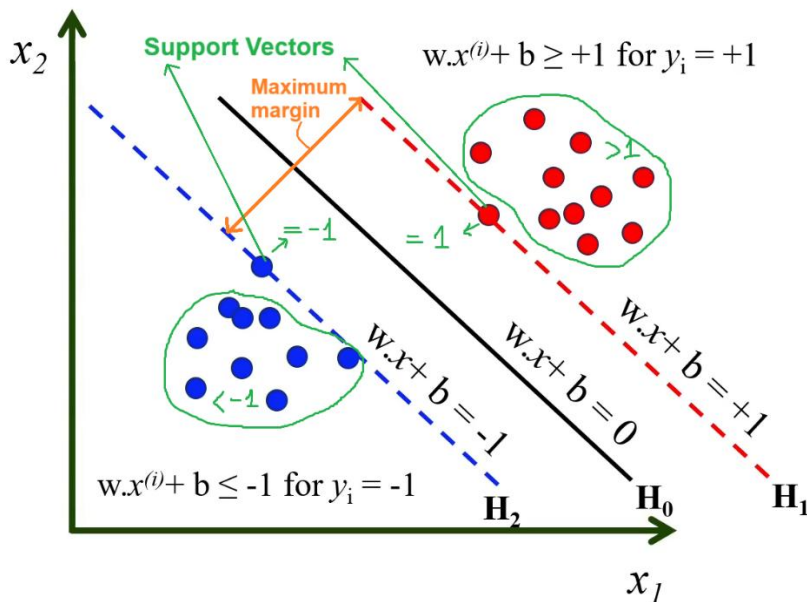
- **Độ dốc (slope)** của đường thẳng = $-w_1 / w_2$
- **Giao điểm với trục x_2** = $-b / w_2$



- Ta xét khi c là một số. Khi thay đổi giá trị c , ta luôn thu được một đường thẳng song song với $w \cdot x + b = 0$, vì độ dốc của đồ thị không thay đổi (độ dốc chỉ phụ thuộc w_1, w_2).



- Tổng quát, trong SVM, ta muốn mỗi điểm x_i nằm **đúng phía của siêu phẳng**, tức là:
 $y_i(w^T x_i + b) \geq 1$ với $y_i = +1$ hoặc -1 .



H₁: $w.x + b = +1$ positive hyperplane
H₂: $w.x + b = -1$ negative hyperplane

Trong mặt phẳng tọa độ Oxy , khoảng cách từ điểm $M(x_0; y_0)$ đến đường thẳng $\Delta: Ax + By + C = 0$ là:

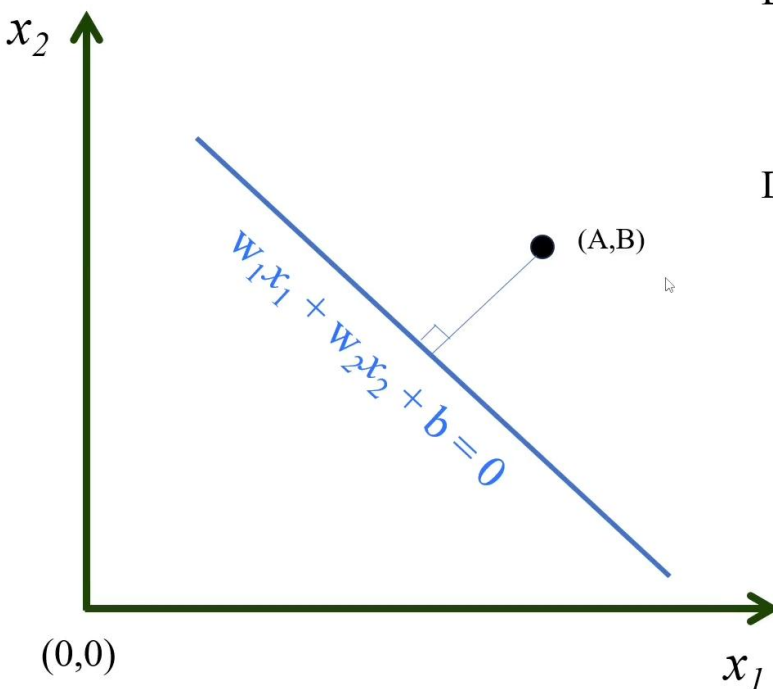
$$d(M, \Delta) = \frac{|Ax_0 + By_0 + C|}{\sqrt{A^2 + B^2}}$$

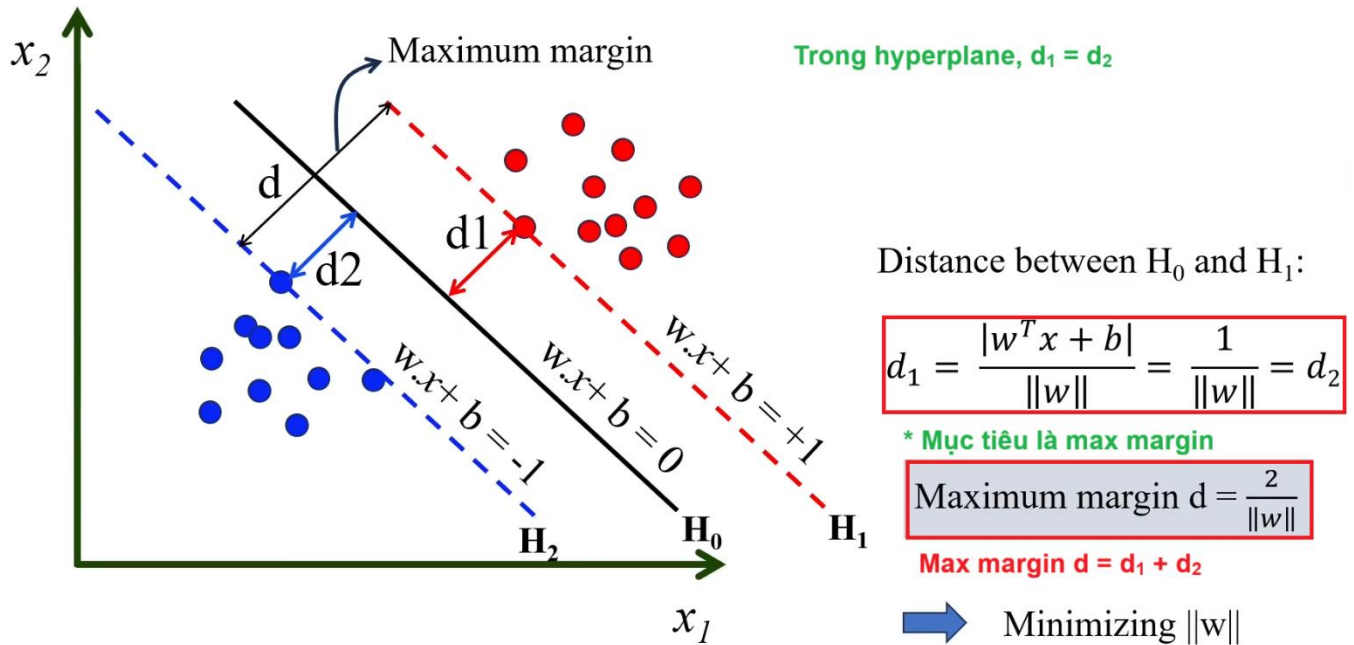
Linear Equation

$$w_1 x_1 + w_2 x_2 + b = 0$$

Distance from a point (A, B) to line:

$$\frac{|w_1 A + w_2 B + b|}{\sqrt{w_1^2 + w_2^2}} = \frac{|w_1 A + w_2 B + b|}{\|w\|}$$



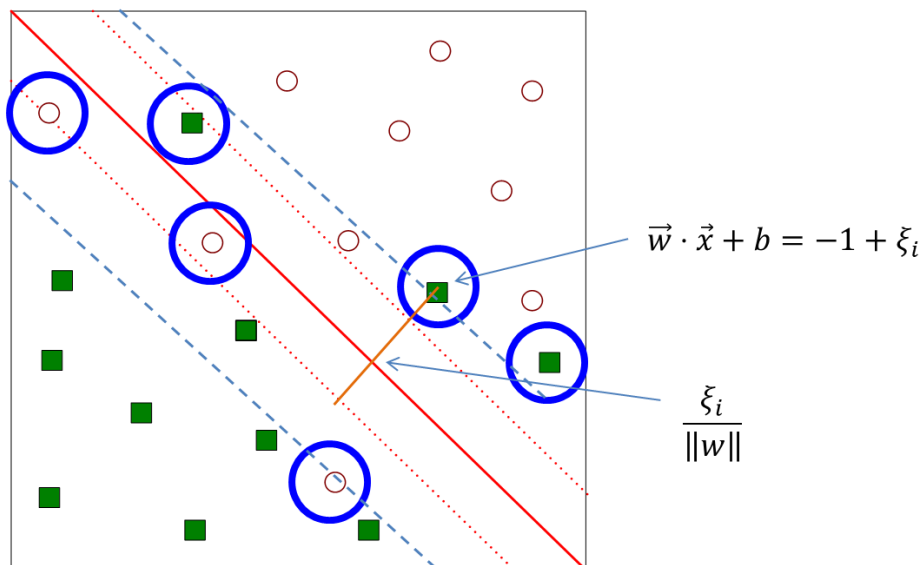


- Tối đa hóa $\frac{2}{\|w\|}$ tương đương với **tối thiểu hóa** $\|w\|$ (vì nghịch đảo của nó). Để tiện cho tính toán, ta thường bình phương biểu thức (vì nó luôn dương, không làm thay đổi nghiệm):

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

- Trở lại với Soft Margin SVM (C-SVM)**

- Trường hợp dữ liệu không phân lớp tuyến tính được → Không tồn tại hyperplane nào phân biệt hoàn toàn 2 lớp mà không có lỗi.



- Biến slack $\xi_i \geq 0$:
 - ξ_i là độ lệch (slack) của chính mẫu i so với margin lý tưởng.
 - $y_i(w \cdot x_i + b) \geq 1 - \xi_i$

- $\xi_i = 0$: điểm nằm đúng phía hyperplane.
- $0 < \xi_i \leq 1$: điểm nằm trong margin.
- $\xi_i > 1$: điểm bị phân lớp sai.
- Khi dữ liệu không hoàn toàn phân tách được (có nhiễu, chồng chéo), SVM cho phép một số điểm nằm sai biên — gọi là Soft Margin SVM. **Tham số C sẽ kiểm soát mức độ cho phép sai số này:**
 - **C lớn** → phạt mạnh các điểm sai → biên hẹp, variance cao (dễ overfit).
 - **C nhỏ** → chấp nhận nhiều sai hơn → biên rộng, bias cao (dễ underfit).
- **Đề margin max:**

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

- $\sum_{i=1}^n \xi_i$ có nghĩa là: Tổng các biến slack ξ_i của tất cả n điểm dữ liệu.

- **SVM là mô hình gì?**
 - là một **mô hình phân biệt (Discriminative Model)**
 - **Mô hình sinh (Generative Model - VD: Naïve Bayes)**: Cố gắng **học sự phân phối xác suất của từng lớp $P(x|y)$** . Sau đó dùng định lý Bayes để tính $P(y|x)$. Nó học "cách dữ liệu được tạo ra".
 - **Mô hình phân biệt (Discriminative Model - VD: SVM, Logistic Regression)**: **Không quan tâm đến việc dữ liệu được tạo ra như thế nào**. Nó chỉ tập trung vào một mục tiêu duy nhất: **tìm ra ranh giới** để phân biệt trực tiếp giữa các lớp. Nó học $P(y|x)$ một cách trực tiếp.
- Do đó, SVM thường cho kết quả phân loại tốt hơn trong nhiều trường hợp vì nó tập trung thẳng vào bài toán cần giải quyết.

3.10. Model Evaluation and Selection (Đánh giá và lựa chọn mô hình)

- *Sử dụng tập validation, test* của các bộ dữ liệu được gán nhãn lớp để đánh giá thay vì sử dụng tập train
- *Sử dụng các độ đo đánh giá (evaluation metrics)*
 - Tập trung vào khả năng dự đoán của mô hình hơn là tốc độ phân lớp hay xây dựng mô hình, khả năng mở rộng,
 - Một số Evaluation metrics:
 - Confusion Matrix
 - Accuracy, Error Rate, Sensitivity, Specificity
 - Precision, Recall, F-measures
- *Sử dụng các phương pháp ước tính độ chính xác:*
 - Cross-validation
 - Bootstrap
 - Comparing classifiers
 - Confidence intervals
 - Cost-benefit analysis and ROC Curves

3.10.1. Confusion Matrix (Ma trận nhầm lẫn)

- Ma trận nhầm lẫn là một bảng tổng kết kết quả dự đoán của mô hình so với kết quả thực tế.
- Các thành phần (cho bài toán 2 lớp):
 - C1 (Positive class), -C1 (Negative class)**. Ví dụ: C1 = "có bệnh", -C1 = "không bệnh".
 - True Positives (TP)**: Thực tế là Positive, dự đoán là Positive (Đúng).
 - True Negatives (TN)**: Thực tế là Negative, dự đoán là Negative (Đúng).
 - False Positives (FP)**: Thực tế là Negative, dự đoán là Positive (Sai, "báo động giả", Type I error).
 - False Negatives (FN)**: Thực tế là Positive, dự đoán là Negative (Sai, "bỏ sót", Type II error).

		← LỚP DỰ ĐOÁN →	
LỚP THỰC TẾ ↑ ↓	LỚP THỰC TẾ \ LỚP DỰ ĐOÁN	C ₁	¬ C ₁
	C ₁	True Positives (TP)	False Negatives (FN)
	¬ C ₁	False Positives (FP)	True Negatives (TN)

- May have extra rows/columns to provide totals
- Example of Confusion Matrix:

Actual class \ Predicted class	play_golf = yes	play_golf = no	Total
play_golf = yes	6954	46	7000
play_golf = no	412	2588	3000
Total	7366	2634	10000

- TP = 6954: 6954 lần thực tế là yes, mô hình dự đoán yes.
- FN = 46: 46 lần thực tế là yes, mô hình dự đoán no.
- FP = 412: 412 lần thực tế là no, mô hình dự đoán yes.
- TN = 2588: 2588 lần thực tế là no, mô hình dự đoán no.
- Về confusion matrix nhiều lớp (n lớp)

	Dự đoán: C ₁	Dự đoán: C ₂	Dự đoán: C ₃
Thực tế: C ₁	CM[1,1]	CM[1,2]	CM[1,3]
Thực tế: C ₂	CM[2,1]	CM[2,2]	CM[2,3]
Thực tế: C ₃	CM[3,1]	CM[3,2]	CM[3,3]

→ CM_{i,j}: số mẫu thực tế lớp i được dự đoán là lớp j.

- **Các thước đo được tính từ Ma trận nhầm lẫn**

- **Accuracy (Độ chính xác)** = $\frac{TP + TN}{All} = \frac{TP + TN}{TP + TN + FP + FN}$

- Tỷ lệ tổng số dự đoán đúng.
- **Vấn đề:** Accuracy có thể gây hiểu lầm trong trường hợp **class imbalance (dữ liệu mất cân bằng)**. Ví dụ, nếu 99% email là không spam, một mô hình ngu ngốc luôn dự đoán "không spam" sẽ có accuracy là 99%, nhưng nó hoàn toàn vô dụng.

- **Error Rate (Tỷ lệ lỗi)** = $\frac{FP + FN}{All} = 1 - \text{Accuracy}$.

A \ P	C	¬ C	<i>sum</i>
C	TP	FN	P
¬ C	FP	TN	N
<i>sum</i>	P'	N'	All

- **Các thước đo cho dữ liệu mất cân bằng:**

- **Sensitivity (Độ nhạy) / Recall (Độ phủ) / True Positive Rate:**

- **Sensitivity (Recall) = R** = $\frac{TP}{P} = \frac{TP}{TP + FN}$

- Trong số tất cả các trường hợp Positive thực tế, mô hình nhận diện được bao nhiêu phần trăm?
- Đo lường khả năng "không bỏ sót" các trường hợp Positive. Rất quan trọng trong y học (không muốn bỏ sót bệnh nhân).
- **Range: [0, 1]**

- **Specificity (Độ đặc hiệu) / True Negative Rate:**

- **Specificity** = $\frac{TN}{N} = \frac{TN}{TN + FP}$

- Trong số tất cả các trường hợp Negative thực tế, mô hình nhận diện đúng bao nhiêu phần trăm?

- **Precision (Độ chính xác):**

- **P = Percision** = $\frac{TP}{TP + FP}$

- Trong số tất cả các trường hợp mà mô hình dự đoán là Positive, có bao nhiêu phần trăm thực sự là Positive?
- Đo lường mức độ "chắc chắn" của các dự đoán Positive. Quan trọng khi chi phí cho False Positive là cao (ví dụ: gán nhầm email quan trọng vào mục spam).

- **"Inverse" relationship (Mối quan hệ nghịch đảo):** Thường thì Precision và Recall có sự đánh đổi. Một mô hình muốn tăng Recall (cố gắng không bỏ sót) thường sẽ phải chấp nhận giảm Precision (dự đoán "bừa" hơn, dẫn đến nhiều False Positive hơn).
- **F-measure (F-score):**
 - Là một thước đo duy nhất kết hợp cả Precision và Recall.
 - Nó là **harmonic mean (trung bình điều hòa)** của Precision và Recall. Trung bình điều hòa "phạt" nặng các trường hợp một trong hai chỉ số quá thấp.

$$F_{\beta} = \frac{1}{\alpha \cdot \frac{1}{P} + (1 - \alpha) \cdot \frac{1}{R}} = \frac{(\beta^2 + 1)P * R}{\beta^2 P + R}$$

F_{β} : weighted measure giữa Precision và Recall

- **F_{β} -measure:** Dạng tổng quát, cho phép coi trọng Recall hơn Precision (nếu $\beta > 1$) hoặc ngược lại (nếu $\beta < 1$).
- **F_1 -measure (hoặc F-score) (balanced F-measure):** That is, when $\beta = 1$,

$$F_1 = \frac{2P * R}{P + R}$$

- F1 càng cao classifier càng tốt

VD12: Cho Confusion matrix. Tính Accuracy, Error rate, Sensitivity, Specificity, Precision, Recall và F1-score

Actual class \ Predicted class	buy_computer = yes	buy_computer = no	Total
buy_computer = yes	6954	46	7000
buy_computer = no	412	2588	3000
Total	7366	2634	10000

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} = \frac{6954 + 2588}{10000} = 95.42\%$$

$$Errorrate = 1 - Accuracy = 4.58\%$$

$$Sensitivity = \frac{TP}{TP + FN} = \frac{6954}{7000} = 99.34\%$$

$$Specificity = \frac{TN}{FP + TN} = \frac{2588}{3000} = 86.27\%$$

$$Precision = \frac{TP}{TP + FP} = \frac{6954}{6954 + 412} = 94.41\%$$

$$Recall = \frac{TP}{TP + FN} = \frac{6954}{6954 + 46} = 99.34\%$$

$$F_1 = \frac{2 * Precision * Recall}{Precision + Recall} = 96.81\%$$

3.10.2. Phương pháp Holdout

- **Cách làm:** Chia ngẫu nhiên toàn bộ dữ liệu thành hai phần không giao nhau:
 - **Training set (Tập huấn luyện):** Dùng để xây dựng mô hình (ví dụ: 2/3 dữ liệu).
 - **Test set (Tập kiểm tra):** Dùng để đánh giá mô hình (ví dụ: 1/3 dữ liệu).
- Thích hợp cho tập dữ liệu nhỏ.
- Lấy mẫu sao cho mỗi lớp được phân bổ đều trong train và test
- **Nhược điểm:**
 - Kết quả có thể phụ thuộc nhiều vào cách chia ngẫu nhiên. Nếu bạn "xui", tập test có thể chứa toàn các ca khó, hoặc tập train không đại diện cho toàn bộ dữ liệu.
 - Lãng phí dữ liệu (chỉ dùng 2/3 để học).
- **Cải tiến (Repeated random sub-sampling):** Lặp lại quá trình Holdout nhiều lần với các cách chia ngẫu nhiên khác nhau, sau đó lấy trung bình kết quả.
 - **Lấy mẫu ngẫu nhiên:** Lặp lại holdout k lần, độ chính xác = trung bình của độ chính xác thu được.

3.10.3. Phương pháp Cross-Validation (Xác thực chéo) (k-fold)

- **Cách làm:**
 - Chia ngẫu nhiên dữ liệu thành k phần (folds) bằng nhau.
 - Lặp lại k lần:
 - Trong lần lặp thứ i , lấy phần thứ i làm **test set**.
 - Lấy $k-1$ phần còn lại gộp lại làm **training set**.
 - Huấn luyện mô hình trên training set và đánh giá trên test set.
 - Kết quả cuối cùng là trung bình của k kết quả đánh giá thu được.
- Thường chọn $k=10$.
- **Ưu điểm so với Holdout:**
 - Tất cả các mẫu dữ liệu đều được làm test set đúng một lần.
 - Kết quả ổn định hơn, ít phụ thuộc vào cách chia ngẫu nhiên.
 - Sử dụng dữ liệu hiệu quả hơn.
- **Stratified cross-validation:** Một biến thể quan trọng, đảm bảo rằng tỷ lệ các lớp trong mỗi "fold" xấp xỉ với tỷ lệ trong toàn bộ dữ liệu. Điều này rất cần thiết cho các bộ dữ liệu mất cân bằng.

3.10.4. Phương pháp Bootstrap

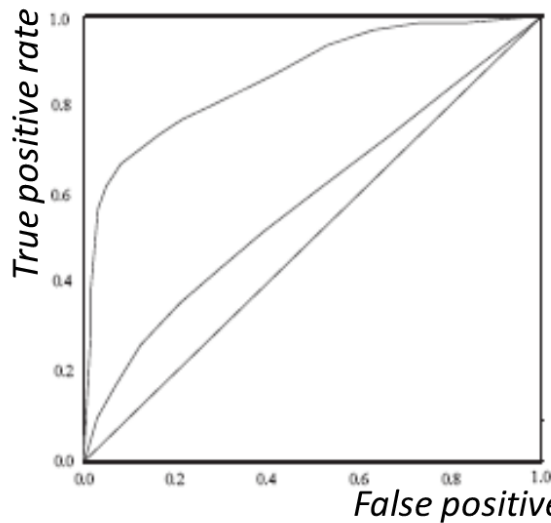
- Hoạt động tốt với các bộ dữ liệu nhỏ.
- Khi một bộ dữ liệu được chọn, nó có khả năng được chọn lại và thêm lại vào tập huấn luyện.

3.10.5. ROC Curves (Đường cong ROC)

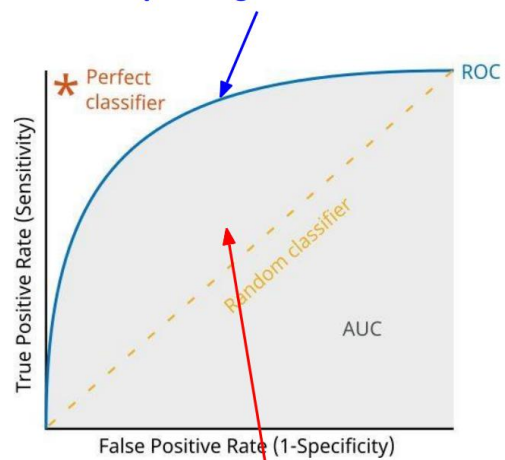
- **ROC (Receiver Operating Characteristics)**

- **Trục tung: True Positive Rate (TPR) = Recall** $= \frac{TP}{TP + FN}$.

- **Trục hoành: False Positive Rate (FPR) = FP / N = 1 - Specificity** $= \frac{FP}{FP + TN}$.



Receiver Operating Characteristic curve



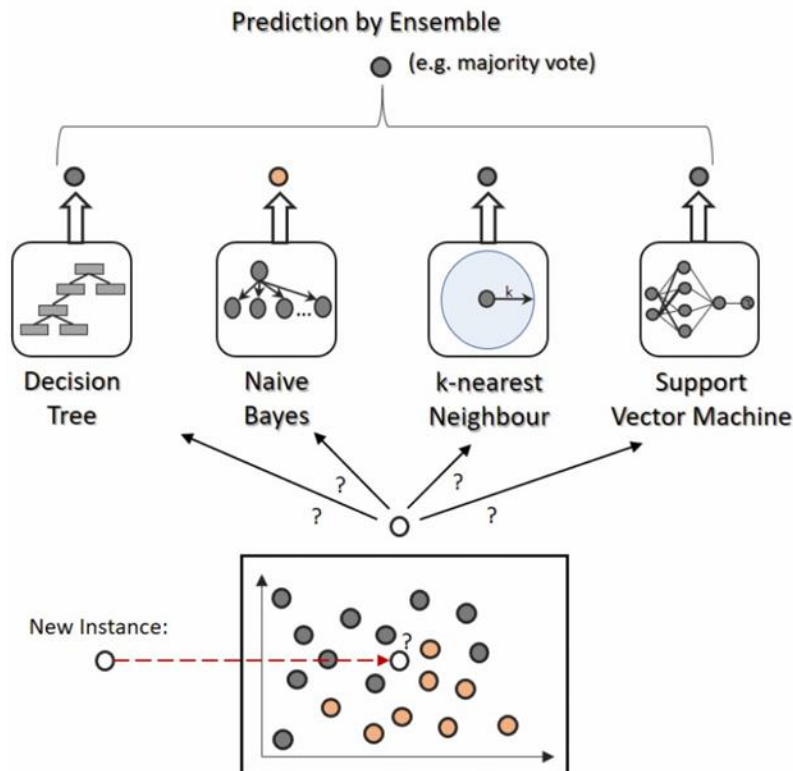
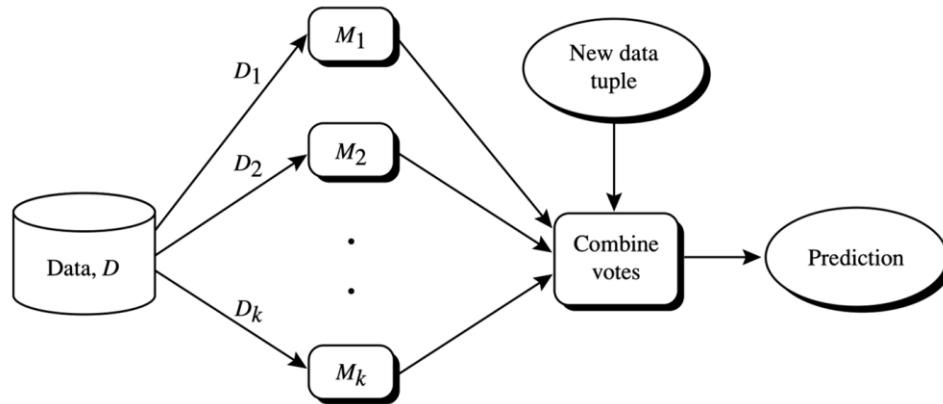
Area under the ROC curve

- ROC biểu diễn **khả năng phân biệt giữa 2 lớp** khi thay đổi ngưỡng phân loại (threshold).
- **Cách vẽ đường cong:**
 - Hầu hết các bộ phân loại không chỉ đưa ra nhãn (0/1) mà còn đưa ra một "điểm số" hay xác suất.
 - Ta thay đổi "ngưỡng quyết định" (threshold) từ 0 đến 1.
 - Với mỗi ngưỡng, ta tính TPR và FPR tương ứng và vẽ một điểm trên biểu đồ.
 - **Nối tất cả các điểm lại ta được đường cong ROC.**
- **Diễn giải đường cong:**
 - Đường **góc 45° (đường chéo)**: mô hình đoán ngẫu nhiên (không phân biệt được 2 lớp).
 - **Điểm (0,1) (góc trên bên trái)**: Là điểm của một bộ phân loại hoàn hảo (TPR=1, FPR=0).
 - **Càng gần góc trên bên trái, mô hình càng tốt.**
- **AUC (Area Under the Curve - Diện tích dưới đường cong)**:
 - **Là một chỉ số duy nhất để tóm tắt hiệu suất của mô hình.**
 - AUC = 1: Mô hình hoàn hảo.
 - AUC = 0.5: Mô hình ngẫu nhiên.
 - AUC > 0.5: Mô hình tốt hơn ngẫu nhiên.
 - AUC cho phép so sánh các mô hình với nhau một cách khách quan.

3.11. Techniques to Improve Classification Accuracy

3.11.1. Ensemble Methods: Increasing the Accuracy (Phương pháp tổ hợp/dàn hợp xướng)

- Thay vì cố gắng xây dựng một mô hình duy nhất thật tốt, chúng ta xây dựng **nhiều mô hình (yếu)** và **kết hợp** dự đoán của chúng lại.
- Kết quả của các mô hình học máy có thể được cải thiện bằng cách kết hợp kết quả của các mô hình khác nhau.
- **Nguyên tắc "Trí tuệ đám đông"**: Một nhóm người đa dạng đưa ra quyết định thường sẽ tốt hơn quyết định của một chuyên gia duy nhất.



• Quy trình:

- Từ bộ dữ liệu gốc D , tạo ra các **bộ dữ liệu con** $D_1, D_2, \dots, D_k$.
- **Huấn luyện các mô hình cơ sở** (base models) $M_1, M_2, \dots, M_k$ trên các bộ dữ liệu tương ứng.
- Khi có một dữ liệu mới, **đưa nó cho tất cả các mô hình M_i để dự đoán**.

- **Combine votes (Kết hợp phiếu bầu):** Tổng hợp kết quả từ tất cả các mô hình (ví dụ: lấy theo đa số - majority voting) để đưa ra dự đoán cuối cùng.
- Mục tiêu là tạo ra một mô hình tổng hợp M^* tốt hơn bất kỳ mô hình thành viên M_i nào.

Câu hỏi: Khi nào việc kết hợp nhiều mô hình lại mang lại hiệu quả?

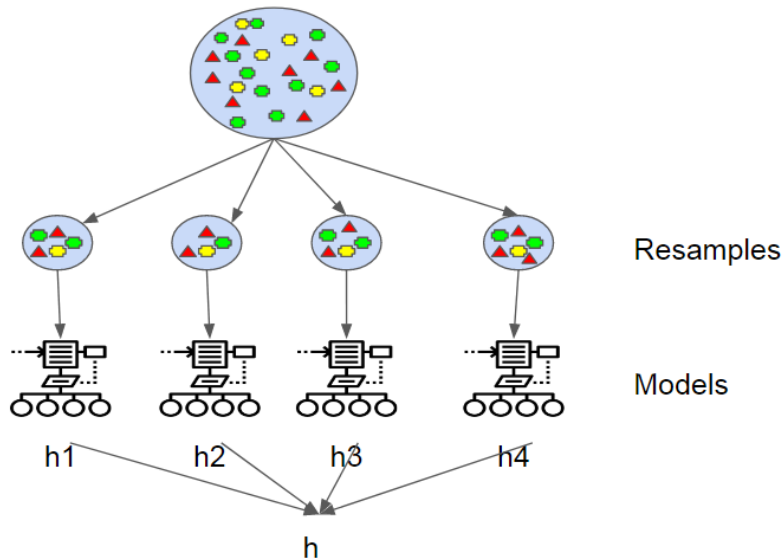
- **Ví dụ (Majority Voting):** Giả sử có 3 mô hình (M_1, M_2, M_3) và 3 mẫu dữ liệu (x_1, x_2, x_3). Dấu tick (✓) là dự đoán đúng, dấu (X) là dự đoán sai.

		x_1	x_2	x_3		x_1	x_2	x_3		x_1	x_2	x_3		
Base model performance	M_1	✓	✓	X	M_1	✓	✓	X	M_1	✓	X	X		
	M_2	X	✓	✓	M_2	✓	✓	X	M_2	X	✓	X		
	M_3	✓	X	✓	M_3	✓	✓	X	M_3	X	X	✓		
Ensemble performance	Voting Ensemble	✓	✓	✓	Voting Ensemble	✓	✓	X	Voting Ensemble	X	X	X		
Case 1: Ensemble has positive effect					Case 2: Ensemble has no effect					Case 3: Ensemble has negative effect				

- **Case 1:** Mỗi mô hình sai ở một chỗ khác nhau. Khi kết hợp lại (Voting), tất cả đều được dự đoán đúng → **Ensemble có hiệu quả tích cực.**
- **Case 2:** M_1 và M_2 luôn đúng, M_3 luôn sai. Khi kết hợp, kết quả vẫn đúng → **Ensemble không có tác dụng (cũng không tệ đi).**
- **Case 3:** Hai trong ba mô hình (M_1, M_3) thường xuyên sai ở cùng một chỗ. Khi kết hợp, kết quả sẽ bị sai theo số đông → **Ensemble có hiệu quả tiêu cực.**
- **Điều kiện để Ensemble hoạt động tốt:**
 - **Accurate (Chính xác):** Các mô hình thành viên phải có độ chính xác tốt hơn là đoán ngẫu nhiên (tốt hơn 50% cho bài toán 2 lớp). Nếu các mô hình thành viên đều kém, kết hợp lại cũng sẽ kém.
 - **Diverse (Đa dạng):** Đây là điều kiện quan trọng nhất. Các mô hình thành viên phải mắc lỗi ở những chỗ khác nhau. Nếu tất cả các mô hình đều giống hệt nhau và cùng sai ở một điểm, việc kết hợp sẽ không mang lại lợi ích gì.
- **Phân loại:**
 - Simple ensemble methods: Max voting, Averaging, Weighted Average
 - Advanced ensemble methods: Bagging, Boosting, Stacking, Blending
- Hai phương pháp Ensemble phổ biến nhất: **Bagging và Boosting.**

3.11.2. Bagging: Bootstrap Aggregation

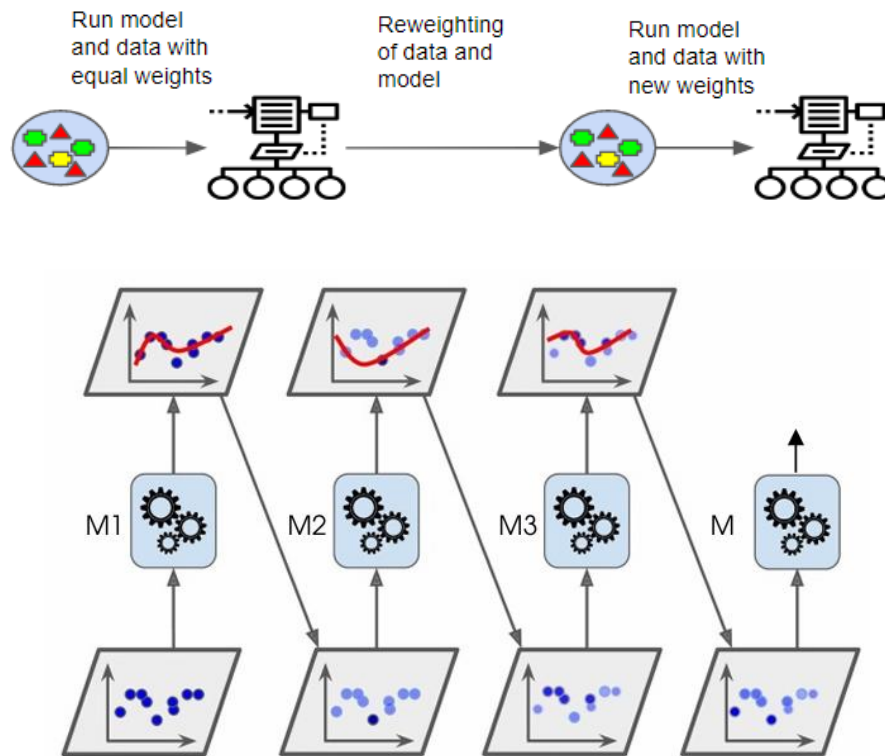
- **Cách tạo sự đa dạng:** Huấn luyện mỗi mô hình trên một bộ dữ liệu con khác nhau. Các bộ dữ liệu con này được tạo ra bằng cách lấy mẫu có hoàn lại (sampling with replacement) từ bộ dữ liệu gốc.
- **Học song song (parallel):** Các mô hình h_1, h_2, h_3, h_4 có thể được huấn luyện độc lập và cùng một lúc.
- Chung 1 thuật toán cho các mode



- Giống như hỏi ý kiến của nhiều bác sĩ khác nhau và lấy quyết định theo đa số.
- **Training (Huấn luyện):**
 - **Lặp k lần** (để tạo k mô hình).
 - Trong mỗi lần lặp, tạo một **bootstrap sample D_i** : **Lấy ngẫu nhiên N mẫu** (bằng kích thước của tập huấn luyện gốc) từ bộ dữ liệu gốc D, nhưng là **lấy mẫu có hoàn lại**. Điều này có nghĩa là *một mẫu có thể được chọn nhiều lần, và một số mẫu có thể không được chọn lần nào*.
 - Huấn luyện mô hình M_i trên bộ dữ liệu D_i .
- **Classification (Phân loại):** classify an unknown sample X
 - Đưa mẫu X cho cả k mô hình.
 - Trả về kết quả được bỏ phiếu nhiều nhất (majority vote).
- **Prediction (Dự đoán số):** Nếu là bài toán hồi quy, thay vì bỏ phiếu, **ta lấy trung bình các kết quả dự đoán từ k mô hình**.
- **Output:**
 - **Hard Voting:** Lớp được nhiều bộ learner dự đoán nhất
 - **Soft Voting:** Nếu output của các learner là số thực (xác suất của class trong classification hoặc giá trị trong regression) thì có thể chọn class có xác suất lớn nhất hoặc lấy trung bình các regression.

3.11.3. Boosting

- **Cách tạo sự đa dạng:** Huấn luyện các mô hình một cách **tuần tự (sequential)**.
- Cố gắng xây dựng một strong classifier từ những weak classifier.
- Giống như tham khảo một hội đồng các bác sĩ, nhưng ý kiến của bác sĩ có "uy tín" (chẩn đoán chính xác trong quá khứ) sẽ có trọng số cao hơn.



- **Học tuần tự (order):**

- Huấn luyện mô hình đầu tiên trên dữ liệu gốc.
- Xác định các mẫu mà mô hình đầu tiên dự đoán sai.
- Trong vòng huấn luyện tiếp theo, **tăng trọng số (reweighting)** cho các mẫu bị **dự đoán sai** đó, khiến mô hình thứ hai phải "tập trung" hơn vào việc sửa những lỗi này.
- Quá trình lặp lại, mỗi mô hình sau sẽ cố gắng sửa lỗi của mô hình trước đó.
- Quy trình này được tiếp tục và các mô hình được thêm vào cho đến khi **tập dữ liệu huấn luyện hoàn chỉnh được dự đoán chính xác hoặc số lượng mô hình tối đa được thêm vào**.
- **Kết hợp:** Mô hình cuối cùng M^* không phải là bỏ phiếu ngang bằng. Phiếu của mỗi mô hình M_i sẽ được **nhân với một trọng số**, trọng số này thường **phụ thuộc vào độ chính xác của chính mô hình M_i** đó. Mô hình nào tốt hơn sẽ có tiếng nói lớn hơn.

3.11.4. Adaboost (Freund and Schapire, 1997)

- một trong những thuật toán **Boosting** nổi tiếng nhất.
- **Input:** Tập huấn luyện $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$.
 - x_i : đặc trưng.
 - y_i : là **nhãn thật** (ground truth) của mẫu đó (thường là +1 hoặc -1).
- **Initialize weights:** Khởi tạo trọng số w của tất cả N mẫu $\{w_n^{(1)}\} = 1/N$
 - $\rightarrow$ Tổng trọng số: $\sum w_n = 1$
- **For round $i = 1$ to k :**
 - **Fit a classifier M_i :** Tìm một mô hình cơ sở (thường là một cây quyết định rất nông, gọi là "decision stump") để tối thiểu hóa **lỗi có trọng số**. Lỗi có trọng số là tổng trọng số của các mẫu bị phân loại sai.
 - **Evaluate error rate ϵ_i :** Tính tỷ lệ lỗi có trọng số của M_i .

$$\epsilon_i = \frac{J_m}{\sum w_n^{(i)}}; \quad J_m = \sum_{n=1}^N w_n^{(i)} I(M_i(x_n) \neq y_n)$$

$$I(M_i(x_n) \neq y_n) = \begin{cases} 1, & \text{nếu } M_i(x_n) \text{ dự đoán sai (khác } y_n) \\ 0, & \text{nếu } M_i(x_n) \text{ dự đoán đúng (bằng } y_n) \end{cases}$$

- **Calculate vote α_i :** Tính trọng số phiếu bầu cho M_i bằng công thức

$$\alpha_i = \frac{1}{2} \ln \left(\frac{1 - \epsilon_i}{\epsilon_i} \right)$$

- Nếu ϵ_i nhỏ (mô hình tốt), α_i sẽ lớn.
- Nếu ϵ_i gần 0.5 (mô hình đoán ngẫu nhiên), α_i sẽ gần 0.

- **Update weights:** Cập nhật trọng số của các mẫu.

$$w_n^{(i+1)} = w_n^{(i)} \exp\{\alpha_i \cdot I(M_i(x_n) \neq y_n)\}$$

- Sau đó, chuẩn hóa lại để tổng các trọng số bằng 1.

- **Final model:** Dự đoán cuối cùng được đưa ra bằng cách lấy **tổng có trọng số của các dự đoán từ các mô hình M_i** (với trọng số là α_i).

$$M^*(x) = \text{sign} \left(\sum_{i=1}^k \alpha_i M_i(x) \right)$$

3.11.5. Gradient Boosting (Tăng cường Gradient)

- **Ý tưởng cốt lõi:** Thay vì điều chỉnh trọng số của các mẫu dữ liệu như AdaBoost, Gradient Boosting sẽ trực tiếp **tối ưu hóa hàm mất mát (loss function)** của bài toán.
- **Cách hoạt động (một cách đơn giản):**

- 1 Khởi tạo ban đầu:

$$\hat{y}_i^{(0)} = 0$$

hoặc trong thực tế, ta có thể khởi tạo bằng một giá trị trung bình:

$$\hat{y}_i^{(0)} = \arg \min_c \sum_i L(y_i, c)$$

(ví dụ: nếu là hồi quy MSE thì c = trung bình của y).

Bắt đầu với một dự đoán ban đầu đơn giản.

- 2 Sau vòng lặp đầu tiên:

$$\hat{y}_i^{(1)} = \hat{y}_i^{(0)} + f_1(x_i)$$

- $f_1(x_i)$: mô hình đầu tiên (ví dụ một cây quyết định nhỏ).
- Nó cố gắng học từ dữ liệu thật để dự đoán gần đúng y_i .
- Vì ban đầu chưa có mô hình nào, nên $\hat{y}_i^{(0)} = 0$, nên $\hat{y}_i^{(1)} = f_1(x_i)$.
 - Huấn luyện một **mô hình yếu mới (new weak learner)**
 - Cộng dự đoán của mô hình yếu mới này vào dự đoán tổng thể.
 - Mỗi mô hình mới được thêm vào để sửa một phần lỗi của các mô hình trước đó.

3 Sau vòng thứ hai:

$$\hat{y}_i^{(2)} = \hat{y}_i^{(1)} + f_2(x_i) = f_1(x_i) + f_2(x_i)$$

- Ở đây, $f_2(x_i)$ không học lại toàn bộ dữ liệu từ đầu, mà chỉ học phần lỗi còn sót lại sau bước 1:

$$residual_i^{(1)} = y_i - \hat{y}_i^{(1)}$$

Mục tiêu của f_2 là ước lượng phần còn sai này.

- Tính toán "**phần dư**" (**residuals**), tức là sai số giữa dự đoán hiện tại và giá trị thực tế.

4 Tổng quát sau t vòng:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

hay tương đương:

$$\hat{y}_i^{(t)} = \sum_{k=1}^t f_k(x_i)$$

→ Nghĩa là mô hình sau mỗi vòng được cập nhật bằng cách cộng thêm một mô hình yếu f_t , giúp giảm sai số dần.

Mô hình cuối cùng (**Additive model - Mô hình cộng tính**) $F(x)$ được biểu diễn là tổng của nhiều hàm con yếu $f_t(x)$:

$$F(x) = f_1(x) + f_2(x) + \dots + f_T(x) = \sum_{t=1}^T f_t(x)$$

- Scalable implementation: XGBoost (Triển khai có khả năng mở rộng: XGBoost):**
 - XGBoost (Extreme Gradient Boosting) là một thư viện/thuật toán triển khai Gradient Boosting cực kỳ hiệu quả và tối ưu.
 - Nó là một trong những thuật toán thành công nhất cho dữ liệu dạng bảng (tabular data) và đã chiến thắng vô số cuộc thi Machine Learning.

3.11.6. Random Forest

- Random Forest** là một cải tiến của phương pháp **Bagging**, nhưng mô hình cơ sở của nó luôn là **Decision Trees (Cây quyết định)**.
- Thay vì chỉ dùng **một cây quyết định (Decision Tree)**, Random Forest **xây dựng nhiều cây (>=1, thực tế ≥ 10 cây mới đáng gọi là "forest") độc lập** với nhau rồi **lấy kết quả "bình chọn" (vote)** của chúng để ra quyết định cuối cùng. Kết quả cuối cùng được quyết định bởi đa số.
- Mỗi **cây quyết định (Decision Tree)** trong rừng là một **bộ phân loại yếu (weak learner)**.
- Càng có **nhiều cây**, mô hình càng:
 - Ổn định hơn** (vì giảm nhiễu, kết quả trung bình tốt hơn).
 - Giảm phương sai (variance)** → ít overfitting hơn.

Nhưng đổi lại:

- Thời gian huấn luyện & dự đoán lâu hơn.
- Dung lượng bộ nhớ lớn hơn.
- **Cách tạo ra sự đa dạng (diversity) - "Randomness":** Random Forest sử dụng hai nguồn ngẫu nhiên:
 - **Data bagging (tương tự Bagging thông thường):** Mỗi cây trong "rừng" được huấn luyện trên một **bootstrap sample** (lấy mẫu có hoàn lại) từ dữ liệu gốc.
 - Với mỗi cây, ta **lấy ngẫu nhiên một tập con** của dữ liệu huấn luyện (cùng kích thước với tập gốc, nhưng có **lặp lại** — *sampling with replacement*).
 - Mỗi cây học trên một tập con khác nhau → giúp giảm **overfitting**.
 - **Feature bagging (ngẫu nhiên về đặc trưng) - Đây là điểm khác biệt chính:**
 - Tại mỗi nút của mỗi cây, khí cần chọn một đặc trưng tốt nhất để phân chia, thuật toán **không xét tất cả các đặc trưng**.
 - Thay vào đó, nó chỉ **chọn ngẫu nhiên một tập con các đặc trưng** để xem xét.
 - Sau đó, nó tìm đặc trưng tốt nhất trong số tập con ngẫu nhiên đó.
 - **Lợi ích của Feature Bagging:**
 - Nó làm cho các cây trong rừng trở nên **ít tương quan (less correlated)** với nhau hơn.
 - Giả sử có một đặc trưng rất mạnh. Nếu không có feature bagging, hầu hết các cây sẽ chọn đặc trưng đó ở nút gốc, làm cho chúng trở nên giống nhau. Feature bagging buộc các cây phải khám phá các đặc trưng khác, tạo ra sự đa dạng lớn hơn.
- **Hai phương pháp xây dựng random forest:**
 - **Forest-RI (random input selection):**
 - Ở mỗi node, **chọn ngẫu nhiên F thuộc tính** làm ứng viên, rồi chia node theo thuộc tính tốt nhất.
 - Đây là cách **thông thường nhất** (được dùng trong thư viện sklearn, R, v.v.).
 - **Forest-RC (random linear combinations):** Một phương pháp phức tạp hơn, **tạo ra các thuộc tính mới bằng cách kết hợp tuyến tính các thuộc tính có sẵn**. Ít phổ biến hơn.
- **Ưu điểm của Random Forest:**
 - **Comparable in accuracy to Adaboost (Độ chính xác tương đương Adaboost):** Thường cho kết quả rất cao.
 - **More robust to errors and outliers (Mạnh mẽ hơn trước lỗi và ngoại lệ):** Do tính chất trung bình hóa của Bagging, nó ít bị ảnh hưởng bởi nhiễu hơn so với Boosting (Boosting có xu hướng tập trung vào các điểm khó/nhiều).
 - **Insensitive to parameters (Ít nhạy cảm với tham số):** Hoạt động khá tốt mà không cần tinh chỉnh quá nhiều tham số.
 - **Faster than typical boosting:** Vì các cây có thể được xây dựng song song.

3.11.7. Tóm tắt lại ưu và nhược điểm của các phương pháp Ensemble hiện đại (Random Forest và XGBoost).

- **Random Forest và XGBoost** là hai thuật toán được sử dụng phổ biến nhất và hiệu quả nhất cho **dữ liệu dạng bảng (tabular data)** (dữ liệu có cấu trúc hàng-cột như trong file Excel/CSV).
- **Pros (Ưu điểm):**
 - **Good performance:** Cho hiệu suất rất cao.
 - **Requires no data scaling:** Không cần chuẩn hóa dữ liệu.
 - **Can scale to large datasets:** Có thể xử lý các bộ dữ liệu lớn.
 - **Can handle missing data:** Cả hai thuật toán đều có cơ chế nội tại để xử lý dữ liệu bị thiếu.
- **Cons (Nhược điểm):**
 - **Can overfit if not tuned properly:** Nếu không được tinh chỉnh tham số cẩn thận (ví dụ: số lượng cây quá nhiều), chúng vẫn có thể bị quá khớp.
 - **Lack of interpretability (Thiếu khả năng diễn giải):** So với một cây quyết định đơn lẻ, việc hiểu tại sao một "rừng" hay một chuỗi các mô hình Boosting lại đưa ra một quyết định cụ thể là rất khó. Chúng giống như những "hộp đen" (black boxes).

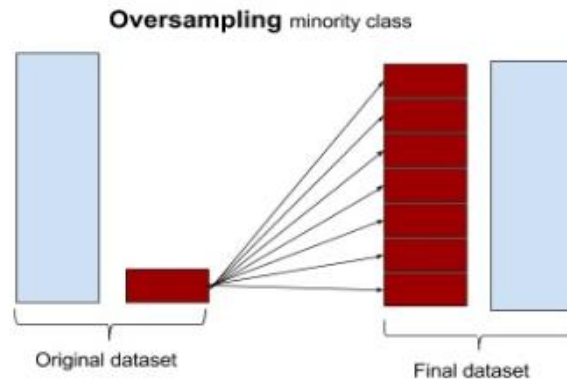
3.11.8. Classification of Class-Imbalanced Data Sets (Phân loại tập dữ liệu mất cân bằng)

- **Giả định trong các phương pháp học máy truyền thống:**
 - Dữ liệu các lớp cân bằng nhau (tỷ lệ giữa các lớp gần bằng nhau).
 - **Chi phí sai lầm (error cost)** của các lớp là **như nhau**.
 - → Ví dụ: dự đoán sai một người "không bị bệnh" thành "bị bệnh" **tốn kém bằng** dự đoán sai chiều ngược lại.
- **Vấn đề:** Trong nhiều bài toán thực tế, số lượng mẫu của các lớp là rất chênh lệch. Lớp chứa ít mẫu được gọi là **minority class (lớp thiểu số)**, lớp chứa nhiều mẫu là **majority class (lớp đa số)**.
- Các thuật toán truyền thống thường giả định các lớp cân bằng, do đó chúng có xu hướng **thiên vị về phía lớp đa số**. Chúng có thể đạt accuracy cao bằng cách dự đoán tất cả là lớp đa số, nhưng lại bỏ qua hoàn toàn lớp thiểu số quan trọng.
- **Ví dụ:**
 - **Medical diagnosis (Chẩn đoán y tế):** Số người bị bệnh (lớp thiểu số) ít hơn rất nhiều so với số người khỏe mạnh.
 - **Fraud detection (Phát hiện gian lận):** Số giao dịch gian lận (thiểu số) cực kỳ ít so với giao dịch hợp lệ.
 - **Product defect (Lỗi sản phẩm):** Số sản phẩm lỗi ít hơn nhiều so với sản phẩm tốt.
- **Tại sao mất cân bằng gây vấn đề?**
 - Giả sử 99% dữ liệu là "âm" (negative). Một mô hình **luôn dự đoán "âm"** vẫn đạt **độ chính xác (accuracy) = 99%**, → nhưng hoàn toàn **vô dụng** vì nó **không bao giờ phát hiện được các trường hợp hiếm** (positive).
 - Vì vậy, khi xử lý **imbalanced data**, ta phải **thay đổi cách học hoặc đánh giá mô hình**.

- Các phương pháp xử lý mất cân bằng ở **cấp độ dữ liệu (Data Level)**.

- **Oversampling (Lấy mẫu quá mức):**

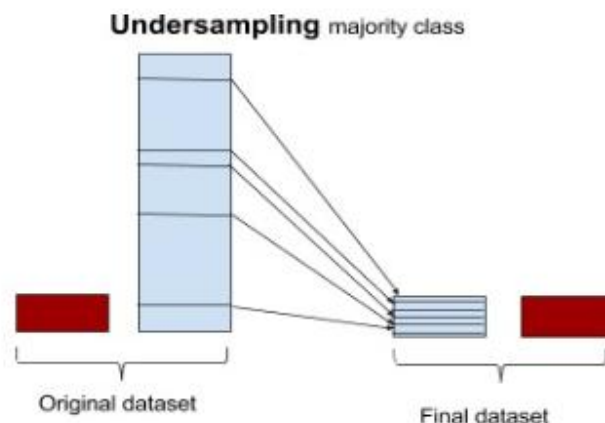
- **Tăng số lượng mẫu của lớp thiểu số.**
- Cách đơn giản nhất là **nhân bản (duplicate)** ngẫu nhiên các mẫu của lớp thiểu số.
- Cách phức tạp hơn (như SMOTE) là **tổng hợp/tạo ra (synthesize)** các mẫu thiểu số mới dựa trên các mẫu đã có.



- Các mẫu màu đỏ (thiểu số) được nhân lên để có số lượng tương đương với màu xanh (đa số).

- **Under-sampling (Lấy mẫu dưới mức):**

- **Giảm số lượng mẫu của lớp đa số.**
- Cách đơn giản nhất là **loại bỏ ngẫu nhiên** các mẫu của lớp đa số.
- **Nhược điểm:** Có thể làm mất thông tin quan trọng chứa trong các mẫu bị loại bỏ.



- Các mẫu màu xanh (đa số) bị loại bỏ bớt để có số lượng tương đương với màu đỏ (thiểu số).

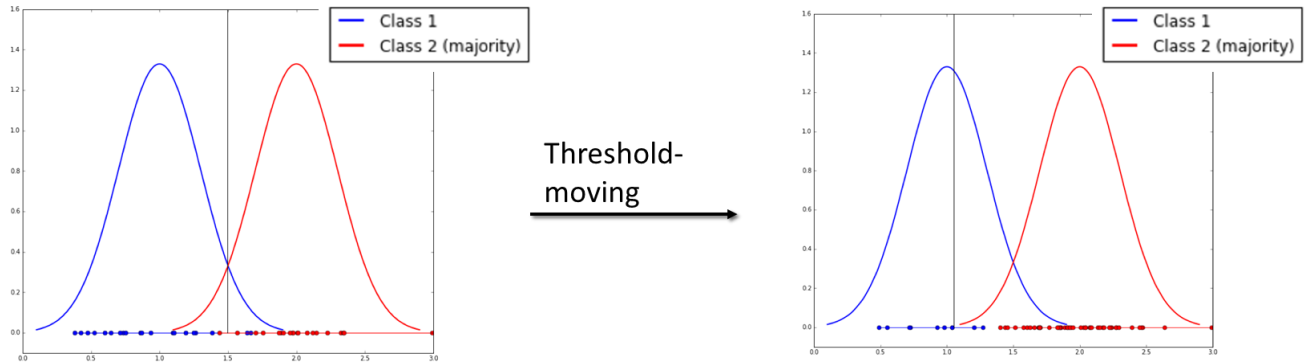
- **Synthesizing (tổng hợp mẫu mới)**

- Tạo **mẫu giả lập (synthetic samples)** của lớp thiểu số bằng cách kết hợp hoặc nội suy từ các mẫu có sẵn.
- Ví dụ: SMOTE, ADASYN.

- Các phương pháp xử lý mất cân bằng ở **cấp độ thuật toán (Algorithm Level)**.

- **Threshold-moving (Di chuyển ngưỡng):**

- Thông thường, ngưỡng quyết định là 0.5 (nếu xác suất > 0.5 thì là lớp 1).
- Với dữ liệu mất cân bằng, ta có thể **giảm ngưỡng này xuống** (ví dụ: 0.2). Điều này làm cho mô hình "dễ dàng" hơn trong việc dự đoán lớp thiểu số, **giúp tăng Recall (giảm False Negative) nhưng có thể làm giảm Precision (tăng False Positive)**.



- **Class weight adjusting (Điều chỉnh trọng số lớp):**

- "Phạt" nặng hơn khi thuật toán phân loại sai một mẫu của lớp thiểu số.
- Nhiều thuật toán (như Logistic Regression, SVM) cho phép gán "trọng số" khác nhau cho các lớp trong quá trình huấn luyện. Ta sẽ **gán trọng số cao hơn cho lớp thiểu số** trong hàm mất mát.
- Ví dụ: trong SVM hoặc Logistic Regression, ta có thể dùng **class_weight = 'balanced'** (trong sklearn)
→ Giúp mô hình "quan tâm" hơn đến các mẫu thiểu số.

- **Ensemble techniques:** Các phương pháp Ensemble cũng rất hiệu quả cho dữ liệu mất cân bằng, vì các mô hình con có thể tập trung vào các khía cạnh khác nhau của dữ liệu.

- **Kết hợp nhiều mô hình** (Random Forest, AdaBoost, BalancedBagging, v.v.)
- Có thể **huấn luyện từng mô hình trên dữ liệu được cân bằng** (bằng oversampling hoặc under-sampling).

- **Evaluate imbalanced data classifier (Đánh giá bộ phân loại dữ liệu mất cân bằng)**
 - **Can we use Accuracy? KHÔNG.**
 - Ví dụ kinh điển: Nếu 98% dữ liệu là lớp negative, một mô hình luôn dự đoán negative sẽ có accuracy là 98% nhưng hoàn toàn vô dụng.
 - **Nên dùng gì?**
 - **ROC Curve (và AUC):**
 - ROC Curve không bị ảnh hưởng bởi sự mất cân bằng của các lớp vì nó xem xét tỷ lệ (TPR, FPR) trong từng lớp riêng biệt.
 - Một mô hình tốt vẫn sẽ có đường cong ROC gần góc trên bên trái, bất kể dữ liệu có cân bằng hay không.

Metric	Ý nghĩa	Khi dùng
Precision	Tỷ lệ dự đoán dương đúng	Khi false positive tốn kém
Recall (Sensitivity)	Tỷ lệ phát hiện đúng lớp dương	Khi false negative tốn kém (như y tế)
F1-score	Trung bình điều hòa giữa precision & recall	Khi muốn cân bằng 2 yếu tố
ROC-AUC	Diện tích dưới đường cong ROC	Đánh giá tổng thể khả năng phân biệt lớp

3.12. Phụ lục

3.12.1. Tham số (Parameters) là gì?

- Trong mô hình học máy, **tham số** là **những giá trị mô hình tự học được** trong quá trình huấn luyện.
 - Đây là những thứ **mô hình tự điều chỉnh** để phù hợp với dữ liệu.
 - Bạn **không chạm vào chúng**, không đặt tay chỉnh chúng.
- **Ví dụ:**
 - Trong **hồi quy tuyến tính**: tham số là **hệ số w và b**.
 - $y = w \cdot x + b$
 - w và b được **học** từ dữ liệu.
 - Trong **mạng neural**: tham số là **trọng số (weights) và bias** trong các lớp.
 - Trong **SVM**: tham số là **vector trọng số của siêu phẳng phân tách**.
 - Trong **KNN**: **KHÔNG có tham số** vì mô hình không học trọng số (lazy learning).
- ➡ **Tham số = cái mô hình tự học.**

3.12.2. Siêu tham số (Hyperparameters) là gì?

- Siêu tham số **không phải** mô hình tự học, mà **người dùng phải chọn trước** khi huấn luyện.
 - Chúng **quy định cách mô hình học**.
 - Ta chỉnh chúng để mô hình hoạt động tốt hơn.
 - Đây chính là phần mà GridSearchCV hoặc RandomSearchCV sẽ tìm giá trị tối ưu.

- **Ví dụ:**

Mô hình	Tên siêu tham số	Ý nghĩa
KNN	n_neighbors	số hàng xóm để phân lớp
SVM	C, kernel, gamma	điều khiển độ phức tạp, loại hàm nhân
Random Forest	n_estimators, max_depth	số cây và độ sâu cây quyết định độ phức tạp
Neural Network	learning rate, batch size	tốc độ học và kích thước mẫu huấn luyện

-  *Siêu tham số = cái bạn (hoặc GridSearch) phải **chọn trước**.*